

claude-windows / 45ea3a91-654d-4972-9cd3-65f35db54caf

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\45ea3a91-654d-4972-9cd3-65f35db54caf\subagents\workflows\wf\_2a64bc88-2a8\agent-a334e8173a624a3d2.jsonl`
- SHA-256: `eeb498484d323dd81e3350ae5ab2d8ac96e0bd89bf0c4b8027cf45b420f8df4c`
- Source modified: `2026-06-10T06:09:44+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf\_2a64bc88-2a8`
- session_id: `45ea3a91-654d-4972-9cd3-65f35db54caf`

Transcript

1. user (2026-06-10T05:59:28.044Z)

CONTEXT ? two sibling repos on a Windows machine:

- INDEXER: C:/Users/avidu/Projects/badminton-highlight-indexer ? local AI badminton(->racket-sports) highlight indexer + rally detector. FastAPI + SQLite + ffmpeg + GPU CV via WSL; Gemini = paid cloud AI. docs/ tree is rich. Current branch docs/next-steps-multisport (master = main branch).

- COLLECTOR: C:/Users/avidu/Projects/sports-data-collector ? golden-label acquisition/curation/ingestion CLI (system-of-record candidate for training corpus).

Project state (June 2026): scaffolding complete; owned-model roadmap agreed (docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md) but NO platform/owned-model implementation started; next committed platform slice = async job model. Licensing guardrail: MIT/Apache-2.0/BSD only for code+data+weights; paid LLMs are inference-only, never training-data sources. ENVIRONMENT RULES: this dev environment has NO GPU/torch/cv2 ? do NOT attempt to run the video pipeline or import torch/cv2 modules. Test suites are offline/fully-mocked and SAFE to run. You are READ-ONLY: never edit/create/delete repo files; running tests and git read commands is allowed.

QUALITY BAR: cite file paths (and line numbers where possible) for every finding. Report what IS in the code/docs, not what you assume. Be skeptical and concrete. Your final structured output is consumed programmatically by an orchestrator ? return raw data, not prose for a human.

TASK: deep RELIABILITY / OPERABILITY / SECURITY audit of the indexer as a system that must one day run as a hosted paid API (see docs/PLATFORM_ARCHITECTURE.md + docs/DEFERRED_HARDENING_PLAN.md ? read both first to know what is ALREADY deferred deliberately; do not re-report deliberately-deferred items as discoveries, instead assess whether the deferral list is COMPLETE).

Deep-read with a failure-mode lens: backend/main.py (API endpoints, sync blocking, global state), backend/infrastructure/database.py (SQLite concurrency, INSERT OR REPLACE cascade, transaction discipline, WAL mode?), backend/pipeline/detectors/ (WSL seam: timeout_sec=0 hangs, staging, healthchecks, cache healing), backend/providers/gemini.py (upload lifecycle, orphaned remote files, API-key handling, error swallowing), backend/pipeline/stitcher/compiler.py (ffmpeg invocation ? command injection via filenames? temp cleanup), backend/reporting/ (never-raises swallow-all), run_*.py entrypoints.

ASSESS: (1) silent-failure inventory ? every path where an error becomes an empty result or swallowed exception, and whether the observability report actually disambiguates each one; (2) concurrency: what happens with 2 simultaneous /api/process calls (SQLite locking, shared globals, GPU contention, same video twice)? (3) security: path/command injection surfaces (video filenames into ffmpeg/WSL bash -c), API key leakage into logs/reports, FastAPI input validation; (4) data integrity: can a crash mid-run corrupt the DB or cache? is re-ingest destructive? (5) resource lifecycle: temp dirs, staged WSL copies, Gemini remote files, frame caches ? what leaks; (6) the DEFERRED_HARDENING_PLAN completeness: name concrete failure modes it does NOT list.

Severity-rate as if a paying customer's video run is on the line.

2. user (2026-06-10T05:59:34.122Z)

```

1 # Deferred Hardening Plan ? Train/Eval Guardrails, Async Jobs, Packaging
2
3 **Status:** ? Design for review ? **no code yet**. Owner-requested follow-up to the
4 "config & identity consistency" tech-debt pass. Each item below becomes its own PR
5 only after this doc is approved.
6
7 **Why these three:** they were cataloged as out-of-scope during the consistency
8 pass because each is larger/design-heavy or needs the owner's machine. They are the
9 remaining high-value hygiene items not already done (config/structure/Result/logging/
10 reports) or in flight (detector abstraction, PR #49).
11
12 **Recommended sequence:** #7 Packaging (lowest risk, unblocks clean installs/CI) ?
13 #13 Train/eval guardrails (correctness of the data flywheel) ? #16 Async jobs
14 (largest, product-facing).
15
16 ---
17
18 ## 1. Packaging modernization (debt #7)
19
20 ### Problem
21 - No `pyproject.toml` on `master`. (PR #49's description lists packaging as "done",
22   but it isn't present ? reconcile with the owner.)
23 - The root `setup.py` is a **diagnostics/dependency-checker**, not a PEP 517 build
24   script ? its name collides with setuptools semantics and will confuse `pip`/`build`.
25 - Deps live only in `requirements.txt`; `obsreport` is a commented soft dep
26   (BACKLOG) and `supervision<0.30.0` is pinned (debt #14).
27
28 ### Approach
29 - Add `pyproject.toml` (PEP 621): project metadata + runtime deps mirroring
30   `requirements.txt`, with optional-dependency groups:
31   - `[project.optional-dependencies] dev` ? `pytest`, coverage tooling.
32   - `reporting` ? `obsreport` pinned to its git tag (`v0.1.3`, per BACKLOG).
33   - `gpu` ? torch/torchvision (note: the cu124 `--extra-index-url` is **not**
34     expressible in standard PEP 621 deps; document the install line or use a
35     `[tool.uv]/pip-conf` approach rather than faking it).
36 - Console entry points: `indexer = backend.main:main` (so `indexer --server` works),
37   optionally eval drivers.
38 - **Rename the diagnostic `setup.py`** (e.g. `scripts/doctor.py`) so it no longer
39   shadows the build system; update README/`run_all_tests.py`/CI references.
40 - Choose a build backend (hatchling recommended) and confirm the `backend/` import
41   root stays valid (keep current layout; no src/ move required).
42
43 ### Risks
44 - `setup.py` rename touches docs and any muscle-memory invocations ? grep first.
45 - Torch CUDA wheels can't be pinned cleanly in PEP 621; must document the
46   `--extra-index-url` install path (the current `setup.py` logic can move into docs).
47 - CI must switch to `pip install -e .[dev]`.
48
49 ### Verification
50 - `pip install -e .[dev]` in a clean venv ? `run_all_tests.py` green.
51 - `indexer --help` resolves via the entry point.
52
53 ---
54
55 ## 2. Train/eval separation guardrails (debt #13)
56
57 ### Problem
58 Separation is **by convention only**. `backend/eval/calibrate_local.py` and
59 `calibrate_wasb.py` do honest leave-one-video-out (LOVO), but nothing *enforces*
60 that a video used to train the learned segmenter
61 (`backend/eval/training.py::build_training_set` ? `distill_local`) is absent from the
62 eval/regression set. Leakage would silently inflate the very metrics the product
63 story rests on (? in BACKLOG).
64
65 ### Approach

```

```

66 - **Split unit = match, never clip** (per `GOLDEN_SET_PHASE1_VIDEOS.md`: split by
67 match, hold ~? of each sport as eval).
68 - Add a `split: "train" | "eval"` field to the collector's `curate export`
69 manifest (cross-repo contract, ADR-002 lineage) ? or a separate eval manifest.
70 - New `backend/eval/splits.py`:
71   - `load_split(manifest) -> (train_ids, eval_ids)` keyed by **match id** (not just
72     file MD5 ? re-encoded duplicates share a match but differ in
73     `compute_video_id`, so group at match level).
74   - `assert_disjoint(train_ids, eval_ids)` ? raise on any overlap.
75 - Wire the guard into the training entrypoint (`distill_local`/`training`) and into
76   `run_regression.py` so the baseline is computed strictly on the eval split.
77
78 ### Risks
79 - Requires a collector-side manifest change (coordinate the cross-repo contract).
80 - Small corpus ? tiny held-out set ? high metric variance; the guardrail is correct
81   but the numbers will be noisy until the golden set grows.
82 - MD5-only dedup is insufficient (re-encodes evade it) ? must track match identity.
83
84 ### Verification
85 - Unit test: overlapping train/eval ids ? `assert_disjoint` raises.
86 - Integration: a deliberately-leaked manifest fails the training entrypoint.
87
88 ---
89
90 ## 3. Async job model (debt #16)
91
92 ### Problem
93 `POST /api/process` runs validate ? segment ? report **inline**, blocking the
94 request for the full duration of a long video. Unworkable as a hosted/product API.
95
96 ### Approach (minimal first slice ? single self-hosted box)
97 - `POST /api/process` returns **202 + `job_id`** ; add `GET /api/jobs/{job_id}`
98   returning `{status, progress, result, reports}`.
99 - Run the pipeline on a background worker ? start with a `ThreadPoolExecutor`
100   (segmenters already use threads internally for AI handoff, so this fits); leave a
101   real queue (RQ/Celery) for later if multi-process is needed.
102 - Persist job state in the existing SQLite DB via the migration ladder
103   (`backend/infrastructure/database.py::init_db`, bump `PRAGMA user_version` 1 ? 2,
104   add a `jobs` table). The observability report becomes the job's artifact.
105 - Update the static UI to poll `GET /api/jobs/{id}` instead of blocking.
106
107 ### Risks
108 - **SQLite thread-safety:** verify `check_same_thread`/connection-per-thread;
109   `_get_connection` already re-issues `PRAGMA foreign_keys=ON` per connection ?
110   confirm it's safe under the executor.
111 - Cancellation/timeout and crash recovery (a job in RUNNING after a restart).
112 - In-process store is not horizontally scalable ? acceptable for the current
113   single-box self-hosted target; call it out so it isn't mistaken for production-HA.
114
115 ### Verification
116 - Submit a job ? poll ? DONE with the same segments the sync path produced.
117 - Server restart mid-job leaves a recoverable/clearly-FAILED job, not a hang.
118
119 ---
120
121 ## Out of this doc
122 - **Detector/WSL abstraction (#6):** in flight (PR #49).
123 - **Unify hybrid segmenters (#7 in LOW_REGRET):** owner-deferred.
124 - **ByteTrack `supervision<0.30.0` migration + real-footage validation (#14/#15):**
125   need a GPU/WSL box and real video ? owner-machine work, tracked separately.
126

```

3. user (2026-06-10T05:59:37.495Z)

```

1   # Platform Architecture ? Multi-User "Assistant Coach"

```

```

2
3  **Status:** ? Strategy / research ? for review. No code yet.
4  **Date:** 2026-06-07 . **Owner:** avidullu
5
6  **Purpose.** Map the path from today's **single-user, local** tool to a **multi-tenant
7  "assistant coach" platform**: users upload videos, get highlights **stored and
8  browsable by history**, and choose **where their data lives** (Google Drive / cloud
9  bucket / local) and **where the compute runs** (our GPU / their VM). The organizing
10 principle is to **decouple the layers** ? identity, ingestion, storage, orchestration,
11 execution/compute, the detection+highlight pipeline, and data-serving ? behind small
12 **pluggable interfaces**, reusing the registry pattern the codebase already uses for
13 segmenters/providers/validators/sports/annotations.
14
15  **Product framing (the north star *and* the v1 line).** This is an **assistant coach**,
16 not just a clipper ? but the strategy is **phased and unambiguous about scope**:
17 - **v1 (concrete, near-term):** **rally detection ? highlight-reel compilation ? per-
user
18 history ? a coach/user correction loop that feeds the golden set** ("assistant-coach
19 *lite"). Essentially today's pipeline made multi-user ? this is the real v1
deliverable.
20 - **Mid-term:** **rich rally/shot semantic metadata** (shot type, rally outcome) and the
21 **owned video-QA model** ($5A; $6 P4?P5).
22 - **North-star vision (strictly future ? *not* v1):** **gait / biomechanics, per-athlete
23 identifying templates, and deep coaching feedback under a supervising coach** ($6 P6).
24 Inspiring, but explicitly deferred ? and the part that raises the **biometric**
privacy
25 bar ($7).
26
27 The buyer is the **academy/coach** (beachhead in
28 [`PMF_MARKET_RESEARCH.md`](PMF_MARKET_RESEARCH.md)); **power users self-serve**. Two
29 consequences ripple through the design: (1) the data model keeps an **optional coach ?
30 athlete overlay over a generic core** ($4.1) ? not baked in; (2) gait/biometrics is
31 **special-category data**, gated and deferred ($7).
32
33 > **Competitive note (validated by research):** none of Veo / Hudl / SwingVision /
34 > Spiideo offer true BYO-storage or BYO-compute ? they're vendor-cloud SaaS
35 > (SwingVision uniquely runs *edge* compute on the user's iPhone). So **"your video,
36 > your storage, your compute" is a genuine differentiator for privacy-sensitive
37 > academies ? and SwingVision proves user-side compute is commercially viable. All four
38 > share the same **teams ? athletes ? clips** data model, which we adopt ($4).
39
40 > Scope: this is a **direction-setting doc**. It does not start implementation; the
41 > first infra slice (an async job model) is the on-hold #16 in
42 > [`DEFERRED_HARDENING_PLAN.md`](DEFERRED_HARDENING_PLAN.md). Commercial constraints
come
43 > from [`COMMERCIALIZATION.md`](COMMERCIALIZATION.md) (C4 paid-LLM no-train; C12 SaaS
44 > foundations) and `archives/MONETIZATION_AUDIT.md`.
45
46 ## 1. Current state & gaps (what we're evolving from)
47
48 | Concern | Today | Gap for multi-user |
49 |---|---|---|
50 | Identity / tenancy | none | no users, orgs, auth, or isolation |
51 | Storage | local `filepath` string; output to `output_dir` | no abstraction; can't BYO
Drive/bucket; raw video must be on the box |
52 | Data model | `videos` keyed by file **MD5**, `play_segments`, `events`,
`candidate_segments` (SQLite) | no owner/athlete link; **highlights aren't a first-class
persisted entity** (compile writes a file and returns a path); no per-user history |
53 | Execution | pipeline runs **in-process, synchronously** in `/api/process` | blocks;
can't dispatch to a user's VM/GPU; no job status |
54 | Pipeline | **already pluggable** (segmenter/provider/validator/sport/annotation
registries) | reuse as-is ? this layer is the model for the rest |
55 | API | `/api/process`, `/api/query`, `/api/compile`, `/api/config`; no auth | needs
auth, per-tenant scoping, async job + history endpoints |
56

```

```

57 Grounding: `backend/infrastructure/database.py` (schema + `PRAGMA user_version`
58 migration ladder, v1), `backend/main.py` (endpoints, sync flow), the registry pattern
59 in `backend/pipeline/segmenters/base.py::SegmenterRegistry` and
60 `backend/providers/base.py`.
61
62 ## 2. Target layered architecture
63
64 Decouple into layers with a clean control-plane vs data-plane split ? the
65 orchestrator coordinates metadata and jobs; the heavy bytes and GPU work can live
66 elsewhere (even in the user's own cloud).
67
68 ...
69
70 CONTROL      ?????????????????????????????????????????????????????????
71 PLANE        ? Presentation / API (auth, per-tenant) ?
72              ? Identity & Tenancy (org?coach?athlete?user) ?
73              ? Orchestration / Jobs (submit?status?result) ?
74              ?????????????????????????????????????????????????????????
75              ? dispatch ? metadata
76 DATA       ?????????????????????????????????????????????????????????
77 PLANE        ? Execution / Compute ? ? Data / Serving ?
78              ? (local | BYO-VM | ? ? (per-tenant DB: ?
79              ? hosted GPU) ? ? videos, segments, ?
80              ? runs the Pipeline: ? ? highlights, jobs, ?
81              ? detect ? highlight ? ? history) ?
82              ?????????????????????????????????????????????????????????
83              ? read input / write artifacts
84              ?????????????????????????????????????????????????????????
85              ? Storage (local upload | Google Drive | S3 | ?
86              ? GCS | Azure) ? raw video + highlight outputs ?
87              ?????????????????????????????????????????????????????????
88
89 The two new pluggable seams are Storage and Compute/Execution; everything
90 else either exists (pipeline) or is net-new infra (identity, orchestration, serving). A
91 model/training layer ? the owned generative video-QA model ($5A) ? sits atop these,
92 fed by Storage (training corpus), Data (labels), and Compute (training + inference), and
93 is delivered through the existing AI-provider registry with a paid fallback**.
94
95 ## 3. Pluggable abstractions (extend the existing registry pattern)
96
97 The codebase already proves the model: an `ABC` + a `Registry` singleton with
98 `register/get/list_available` (e.g. `SegmenterRegistry`). Add two more registries with
99 the same shape so they're plug-and-play and testable in isolation.
100
101 ### 3.1 `StorageBackend` registry
102 Replace the bare `video_path: str` that flows through the pipeline with a
103 StorageRef (a backend id + a key/URI), resolved by a backend.
104
105 ```python
106 class StorageBackend(ABC):
107     name: str
108     def open_read(self, ref: StorageRef) -> BinaryIO: ... # stream large video
109     def download_to_local(self, ref: StorageRef, dst: str): ... # for ffmpeg/cv2/WSL
110     def upload(self, local_path: str, ref: StorageRef): ... # persist
111     def signed_url(self, ref: StorageRef, ttl: int) -> str: ... # hand to LLM / browser
112     def stat(self, ref: StorageRef) -> StorageStat: ...
113     def list(self, prefix: StorageRef) -> list[StorageRef]: ...
114
115
116 - Recommended implementation: build the backend over fsspec (one matrix ?
117 local / S3 (`s3fs`) / GCS (`gcsfs`) / Azure (`adlfs`) + Google Drive via `gdrivefs`,
118 Dropbox via `dropboxdrivefs`, reaching for smart_open as the robust large-MP4
119 byte-streaming primitive. Storage becomes one more registry. (Treat Drive/Dropbox as
120 add-ons ? less battle-tested than `s3fs`/`gcsfs`; cover with integration tests.)

```

```

121 - **Backends:** `local` (upload dir), `gdrive`, `s3`, `gcs`, `azure`. **BYO-storage**:
122 the tenant registers their own bucket/Drive; raw video **never leaves their account**.
123 - **Auth ? never persist users' long-lived keys.** For buckets, have the user create an
124 IAM role you **AssumeRole** into, scoped to one bucket/prefix, and mint **presigned
125 URLs** so raw video moves **browser ? the user's bucket directly, never through our
126 servers**. For Drive, OAuth with the narrow **drive.file** scope; for academies, a
127 **service account with domain-wide delegation**. Store any per-tenant creds encrypted.
128 - **Why it matters for the pipeline:** ffmpeg/OpenCV/WSL need a **local file**, while
the
129 LLM provider can take a **signed URL**. The backend exposes both `download_to_local`
130 (for compute) and `signed_url` (for the provider/browser), so the pipeline stops
caring
131 *where* the bytes live.
132 - **Migration path:** introduce `StorageRef` alongside the current local path; a `local`
133 backend wraps today's behavior so nothing breaks; adapters land incrementally.
134
135 ### 3.2 `ComputeBackend` / `ExecutionBackend` registry
136 Where the pipeline actually runs for a given job.
137
138 - **`local`** ? in-process / local subprocess (today's behavior; dev + self-host).
139 - **`hosted`** ? our GPU VM pool (the SaaS default).
140 - **`byo_worker`** ? the tenant runs a **pull-based worker** (a small agent) on their
own
141 VM/GPU. **Critical security property:** the worker *pulls* jobs scoped to its tenant
142 from the control plane, pulls input from the tenant's **own** storage, runs the
143 pipeline, and writes artifacts + results back ? so **the orchestrator never holds the
144 user's cloud credentials** and needs **no inbound access** to their network. This
145 mirrors the **self-hosted CI runner** pattern (GitHub Actions runners use a 50s
146 outbound-only HTTPS long-poll). This pull-based runner is the **cheapest, most
147 credential-safe v1** for BYO-compute.
148 - For users who want the platform to **provision GPUs in their *own* cloud account on
149 demand** (an academy with an AWS account but no standing GPU box), add **SkyPilot**
150 (BYOC: everything launches in the user's accounts/VPCs; creds stay with them).
151 **Modal** is the managed fallback for users wanting zero infra (accepting that raw
152 video then flows through Modal).
153 - **?? Operational / support burden (a real adoption risk).** For the primary beachhead
?
154 Indian badminton **academies/coaches** ? asking them to *run and maintain a worker on
155 their own VM/GPU* is a meaningful ongoing load (updates, monitoring, firewall/NAT, the
156 "it stopped after our IT change" ticket). So: **default academy users to `hosted`**,
and
157 treat `byo_worker`/SkyPilot as a **power-user / enterprise / privacy-sensitive tier**,
158 not the academy default. If/when BYO ships, plan for **auto-updating, self-healing
159 agents + health heartbeats + clear worker status in the UI**, and budget the support
cost.
160 - **Middle tier to consider ? "zero-infra BYO":** a managed-but-isolated path where the
161 **storage** stays in the user's bucket (signed URLs) while compute runs on **our**
GPUs
162 in a **strictly isolated, per-tenant, ephemeral sandbox** (nothing persists our side).
163 This gives the BYO *privacy* benefit (raw video never persists on our infra)
**without**
164 the user running a worker ? a pragmatic default between full-hosted and full-BYO.
165
166 The pipeline code is unchanged; only *where it's invoked* and *how it gets bytes* vary.
167
168 ### 3.3 Reuse, don't reinvent
169 The existing `segmenter` / `provider` / `validator` / `sport` / `annotation` registries
170 are the proven precedent and stay as the **pipeline** layer. The `annotation` registry +
171 `candidate_segments` table are also the natural seam for the **coach-correction / golden
172 -set** loop in the assistant-coach phase ($6, P4).
173
174 ## 4. Multi-tenant data model & serving layer
175
176 ### 4.1 Entities ? generic core, with the coach model as an optional overlay
177 The foundational schema is deliberately **domain-neutral** so we don't overfit to the

```

```

178     coach mindset ? the coach/athlete framing is a market-fit hypothesis that will
iterate**
179     (see C13 / PMF), not a load-bearing assumption. Generic core:
180     ```
181     User ??< VideoCollection ??< Video ??< Highlight ??< HighlightMetadata
182                               ???< PlaySegment ??< Event
183     Video / Highlight ??< Job (processing record: status, compute backend, timings)
184     ```
185     - Generic & re-targetable: a `VideoCollection` groups a user's videos (a season, an
186       event, a "folder") with no assumption of a coach or academy ? a power user, a coach,
or a
187     parent are all just `User`s with collections.
188     - Highlights are first-class (today compile only writes a file): a `highlights`
table
189     (segment ids + output `StorageRef` + params + created_at) powers "see past videos and
190     highlights by history," and HighlightMetadata carries the rich semantic tags
191     (shot type, rally outcome, etc. ? see §6 metadata stage) plus titles / ratings /
notes.
192     - video_id stays the content MD5 (dedup within a tenant), scoped by `tenant_id`.
193     - Coach overlay (optional, additive): for the academy/coach GTM, layer
194     `Org/Academy ? Coach ? Athlete` *on top* (e.g. a `Membership`/role table + an
`athlete_id`
195     tag on collections/videos) without baking it into the core tables ? so the schema
196     stays usable for solo power users and the coach model can evolve, or be dropped, as
PMF
197     dictates.
198     - History/serving API: list collections/videos/highlights per user (and, with the
199     overlay, by athlete/coach/date); query highlights by `HighlightMetadata` (e.g.
shot
200     type / outcome); re-stitch from stored segments without reprocessing.
201
202     ### 4.2 Tenancy isolation ? options & recommendation
203     | Model | Isolation | Ops cost | Best for |
204     |---|---|---|---|
205     | Shared DB + `tenant_id` (row-level, ideally Postgres RLS) | logical | low |
Hosted SaaS, many small tenants ? recommended default |
206     | Schema-per-tenant | medium | medium | tens/hundreds of mid tenants |
207     | DB-per-tenant (incl. SQLite-per-tenant) | strong/physical | higher at scale |
self-hosted single-academy, or high-isolation enterprise |
208
209     Recommendation: for the hosted product, migrate SQLite ? Postgres with shared-DB +
210     tenant_id + Row-Level Security; keep SQLite-per-tenant as the self-hosted /
211     single-academy deployment (it reuses today's code almost verbatim). Either way,
212     abstract DB access behind a repository layer now so the engine is swappable, and
213     graduate the `PRAGMA user_version` migration discipline to Alembic under Postgres.
214
215     ## 5. Orchestration / async jobs
216
217     This is the serving layer that makes multi-user real. It is the multi-user superset of
218     [DEFERRED_HARDENING_PLAN.md](DEFERRED_HARDENING_PLAN.md) #16 (async job model) ?
build
219     that first, single-tenant, then generalize.
220
221     - Job record (`jobs` table): `id, tenant_id, video_id, type (index|compile|analyze),
222       status (queued|running|done|failed), progress, compute_backend, result_ref, error,
223       timestamps`. `/api/process` returns 202 + job_id; `/api/jobs/{id}` polls.
224     - Idempotency keyed on `(tenant_id, video_id, pipeline_version)` ? reuse the
225       content-hash discipline already in `compute_video_id`.
226     - Dispatch: the orchestrator enqueues; a worker (hosted or `byo_worker`) claims jobs
227       for its tenant, resolves input via the tenant's `StorageBackend`, runs the pipeline,
228       writes artifacts + DB rows. Observability reports (already per-video) become the
229       job artifact surfaced in history.
230     - Queue/worker backend ? a decision to prototype in #16, not a default here. The
231       requirements are firm ? idempotency, resumability (a 40-min GPU job mustn't
232       restart on a worker blip), progress states ? which rule out fire-and-forget

```

Celery/RQ

```
233     for the long jobs. The *choice* should be made by prototyping in the
234     [DEFERRED_HARDENING_PLAN.md](DEFERRED_HARDENING_PLAN.md) #16 slice, weighing:
235     - **Start simplest:** *Arq* (async-native, natural FastAPI fit) or even a **plain
236       Postgres-backed queue + worker** ? likely sufficient for the *first* single-tenant
237       async slice, with the lowest operational complexity and the easiest debugging.
238     - **Durable-execution upgrade if/when needed:** *Hatchet* (Postgres-backed, per-step
239       retries/timeouts/concurrency; pairs neatly with the pull-based worker) ? attractive
240       because it adds **no new datastore** once we're on Postgres, but it *is* a new
241       dependency + operational/debugging surface; adopt only when long-job resumability/
242       observability demands it.
243     - **Heaviest:** *Temporal* ? most mature durable workflows, operationally heavy;
244       reserve for genuinely complex multi-stage pipelines at scale.
245
246     Record the decision criteria in #16: operational complexity, maturity, **debugging
247     experience for long GPU jobs**, and whether durability is worth a new dependency.
248     **Bias: start with the simplest thing that gives retries + status; graduate only on
249     evidence.**
250
251     ## 5A. Owned generative model (the video-QA assistant) & the data flywheel
252
253     The north-star capability: an **owned, self-hostable generative video model** ?
254     fine-tuned on sports video (including **consented, user-contributed** footage) ? that
255     **answers questions about rallies, shots, and technique**. This is the conversational
256     brain of the "assistant coach." Crucially, it does **not** replace the known stack; it
257     is introduced **behind the same abstraction, with the paid API as an always-ready
258     fallback**.
259
260     > **The moat is *not* the model.** Anyone can download Qwen-VL/Gemma. The defensible
261     > assets are *(a) the consented, badminton-specific video?QA dataset* and *(b) the
262     > eval harness**. Build and message *those* ? not "we own an LLM."
263
264     > **? Committed guardrail (legal, non-negotiable).** We respect the licensing findings
265     in
266     > full. **Paid frontier models (Gemini / OpenAI / Anthropic) are used for *inference /
267     > serving / fallback* only ? never as a *training* data source.** Harvesting their
268     outputs
269     > to train our owned model would breach their "no competing model" terms (a
270     terms/copyright
271     > violation), so it is prohibited in both engineering and legal process. **Training
272     teachers
273     > are open, Apache-2.0 models only** (Qwen-VL / InternVL / Gemma 4). Detail in the
274     > distillation constraint below.
275
276     ### Core principle: AI is a pluggable provider with a paid fallback
277     The codebase already abstracts the cloud AI behind `provider_registry` /
278     `AIVideoProvider` (`backend/providers/`). Make that a **first-class platform
279     principle**:
280     - Every generative capability routes through the **provider registry**.
281     - The **owned model** plugs in as a new provider (`owned_vlm`); **Gemini (paid, via
282       Vertex AI)** stays registered as the **fallback** (failover on error/low-confidence,
283       or as the quality ceiling/teacher reference).
284     - This lets us **ship today on the known paid stack** and **swap in the owned model when
285       its quality clears a measured bar** ? with zero architectural churn and the paid path
286       always one config flip away. (Mirrors the COMMERCIALIZATION C4 A/B/C strategy: the
287       owned model is the long-run "(A) eliminate the per-call dependency"; paid is "(C)".)
288
289     ### The model / training layer (new sub-layer, fed by the scaffolding)
290     ```
291     Data curation      Training      Eval      Serving
292     (consented user vid (fine-tune an open, (extend the existing (self-host inference
293     + golden set +      commercially-usable  eval harness to QA   as a provider /
294     rally/segment/event base VLM via      quality + the        ComputeBackend;
295     annotations ?       LoRA/QLoRA)      regression gate)     paid fallback)
296     video-QA pairs)
```



```

292     ``
293     This layer **reuses every other layer**: **Storage** holds the training corpus;
294     **Data** supplies labels (the `annotation` registry + `candidate_segments` +
295     golden set are already the labeling seam); **Compute/Orchestration** runs training and
296     inference jobs (even **BYO-compute** can serve inference on the tenant's GPU); the
297     **eval harness** already in `backend/eval/` extends to grade QA quality. **In other
298     words, the rock-solid scaffolding is exactly what makes the model flywheel possible ** ?
299     which is why scaffolding comes first (see S6).
300
301     ### Base model & training (Apache-2.0 only ? licensing is decisive)
302     **Recommended near-term base: `Qwen2.5-VL-7B-Instruct`** ? Apache-2.0, native long-video
303     (dynamic-FPS sampling, temporal mRoPE), mature LoRA tooling. Upgrade path
304     **Qwen3-VL-4B/8B (Apache-2.0)**; **Gemma 4 multimodal (Apache-2.0, 2026)** is a clean
305     Google-stack alternative to evaluate.
306     - ?? **Lock to Apache-2.0-clean bases** so you can fine-tune, **keep weights private,
307     gate behind subscriptions, and never owe open-sourcing or branding**. (Note Qwen2.5-VL
308     is Apache **only at 7B/32B** ? 3B/72B use the custom Qwen license.)
309     - ? **Reject for the flagship:** **Gemma 3** ? custom terms with a **viral derivative
310     clause** (anything fine-tuned on it, *or even trained on its synthetic outputs*, stays
311     permanently bound to Google's *updatable* Prohibited-Use Policy); **Gemma 4's switch
312     to
313     Apache-2.0 removes this**. **Llama 3.2 Vision** ? custom license forces a "Llama-?"
314     model name + "Built with Llama" badge and a 700M-MAU ceiling. **`gpt-oss`** ?
315     Apache-2.0
316     but **text-only**, so usable only as an optional text-reasoning layer, never the video
317     encoder.
318     - **Training method:** **QLoRA** (? r=64, ?=128) on the 7B ? fits one prosumer GPU and
319     avoids catastrophic forgetting of general video skills. Build the dataset by
320     converting
321     the existing rally/shot/segment annotations + golden set into templated **clip?QA
322     pairs**, expand synthetically with **open-model teachers only** (see constraint
323     below),
324     and hold out a **golden QA eval** that gates every fine-tune. A few hundred gold pairs
325     ?
326     **5k?20k** curated is a realistic band for this narrow domain.
327     - **Serving economics:** a 7B VLM ? 14 GB VRAM (FP16) / ~3.5 GB (INT4); a ?16 GB GPU is
328     a
329     comfortable serve target (vLLM for throughput). Self-hosting wins **only** once you
330     have
331     sustained video-QA volume **+** a defensible consented dataset **+** real per-call API
332     cost pain ? below that, the paid API is cheaper than a standing GPU and the second
333     stack
334     is overhead. Hence: **paid-primary now ? owned-primary / paid-fallback later.**
335
336     Base-model licensing + training-data policy are tracked as **COMMERCIALIZATION C14**.
337
338     ### Training backend ? managed service vs self-managed (the fine-tune "forge")
339     *Where* you run the LoRA/RL fine-tune. Weigh on **video support, weight
340     export/ownership,
341     smallest servable model, RL support, no-train DPA, and infra burden**.
342
343     **Decisive finding (researched 2026-06):** managed fine-tuning services are
344     **image-based** (JSONL + base64 images) ? for video you pre-sample frames and feed them
345     as image sequences. **Only the self-managed `ms-swift` explicitly advertises `native`
346     video (+audio) training**. So for true video-QA, a self-managed framework is the
347     strongest technical fit ? and it also lets you fine-tune the **small 7B** (cheap to
348     serve) and keep full ownership. Managed services trade some of that for zero infra.
349
350     | Option | Type | Video / VLM support | Smallest VL | RL | Export weights | Best for |
351     |---|---|---|---|---|---|---|
352     | **ms-swift** (ModelScope) | Self-managed (rented/own GPU) | **Native video+audio**;
353     Qwen3-VL/Omni, 300+ MLLMs | any incl **7B** | ? DPO/GRPO | ? full ownership | **? video-QA;
354     slots into BYO-compute (§3.2)** |
355     | **LLaMA-Factory** | Self-managed | Qwen3-VL + 100+ VLMs; video datasets | any incl 7B
356     | ? | ? | popular, well-documented |

```

```

345 | **Unsloth** | Self-managed (1-GPU optimized) | Qwen3-VL LoRA (vision+lang; frames) |
any incl 7B | ? GRPO | ? | cheapest first runs |
346 | **Tinker** (Thinking Machines) | Managed API (low-level) | Qwen3-VL **30B-A3B+ only**;
image input (video **unverified**) | 30B-A3B MoE | ? PP0/GRPO, custom loss | ? adapters + ckpt;
no-train ToS | zero-infra RL/research at MoE scale |
347 | **Together AI** | Managed (tune + serve) | Qwen3-VL / Gemma-3 / Llama-4; image data |
verify | partial | ? adapters downloadable | managed Qwen3-VL + serving |
348 | **Fireworks AI** | Managed (tune + serve) | Full Qwen2.5-VL (3B?72B SFT; LoRA@32B +
LoRA-upload) | **7B** | partial | ?? verify export (lock-in risk) | managed *small*-VL + serving
|
349 | **Predibase** (Rubrik) | Managed *** VPC self-host** | VLM instruct-tune; **LoRAX**
multi-adapter | verify | ? (VPC) | ? **RFT/GRPO** flagship | RL "creativity" + multi-adapter +
privacy |
350 | ~~OpenAI GPT-4o / Gemini tuning~~ | Closed | ? | ? | ? | ? no export | **non-fit ?
violates "own the model"** |
351
352 *(Enterprise clouds ? Databricks Mosaic / SageMaker / Vertex on open models ? give full
353 control but are only worth it if you're already on that stack.)*
354
355 **Recommendation:**
356 - **Early iteration / true video-QA ?** self-managed **`ms-swift`** (or LLaMA-Factory /
357 Unsloth) on a **rented GPU** (RunPod / Lambda / Modal / SkyPilot-to-your-cloud):
native
358 video, any size incl. **7B** (cheap to serve), full ownership, and it drops into the
359 **ComputeBackend / BYO-compute** seam ($3.2). **Likely a better fit than Tinker** for
360 the video requirement *and* serving economics.
361 - **Managed, no GPU ops ?** **Together AI** (Qwen3-VL + downloadable adapters + serving)
362 or **Fireworks** (full Qwen2.5-VL incl. 7B + serving) ? confirm frame/video support
and
363 adapter export.
364 - **If RL "coaching creativity" + per-sport multi-adapter is central ?** **Predibase**
365 (RFT + LoRAX, VPC self-host for the biometric/privacy posture) or **Tinker** (lowest-
366 level RL primitives).
367 - **Tinker** remains a strong zero-infra RL-research option with clean weight export ?
just
368 not the cheapest path to a *small video* model (its VL floor is 30B-A3B).
369 - **Portability is the hedge:** you own the **data + recipe + exported weights**, so the
370 backend is swappable ? same ethos as the provider/registry pattern. Don't get locked
in.
371
372 **Verify before committing (any backend):** (1) **native video / frame-sequence**
373 ingestion + max frames per example; (2) **weight export** + a **commercial DPA**
covering
374 consented, minors-excluded video (biometric, $7); (3) the **inference plan** for the
size
375 you train (a self-managed **7B** is far cheaper to serve than a 30B-A3B MoE).
376
377 ### Training harness ? make it agent-runnable (nanochat-style operability)
378 The goal: **an agent (or CI) can run a fine-tune on a fresh batch of golden-labeled
video
379 and get a trustworthy ship / no-ship verdict, unattended.** Karpthy's **nanochat** is
the
380 reference for that *operability* ? one MIT repo, one `speedrun.sh` that runs
381 tokenizer?train?eval?serve, a single complexity dial, and an automatic **report-card**
382 score. It is **not** a usable engine here (text-only, from-scratch GPT-2-class), but its
383 five properties are exactly what make training agent-runnable, and we copy them:
384 ** (1) one command, end-to-end; (2) one complexity dial (sane auto-derived
defaults);
385 ** (3) an automatic **report card** ? a deterministic pass/fail vs a target (the crux);
386 ** (4) reproducible + deterministic + cheap; (5) minimal/readable.
387
388 **Engine (don't reinvent):** **`ms-swift`** already delivers this for VLMs ? CLI-driven,
389 **native video**, Qwen3-VL support, an eval backend (EvalScope), and reproducibility via
390 `args.json` + `full_determinism`. (LLaMA-Factory is the close second; HF **`nanoVLM`** ?
391 ~750 lines ? is the minimal reference to *learn* VLM training, not the production path.)

```

```

392
393     **The harness = thin glue over components we already have** ? one declarative,
394     agent-invokable job (call it `train_speedrun`):
395     1. **Build dataset** from golden labels ? clip?QA pairs (reuse the `annotation` registry
+
396         `candidate_segments` + golden set ? the labeling seam in `backend/`).
397     2. **Fine-tune** via ms-swift on a **ComputeBackend** GPU ($3.2), run as an
orchestration
398         job ($5 / DEFERRED #16): one config, idempotent, logged.
399     3. **Auto-eval + report card** ? extend the existing `backend/eval/` regression harness
to
400         **QA quality**; emit a `report.md` pass/fail vs the **held-out golden QA eval**.
401     4. **Register** the result as the `owned_vlm` provider (Gemini paid fallback stays)
**only
402         if it beats the bar**.
403
404     So: **golden labels ? ms-swift train (on a compute backend) ? eval gate ? register
405     provider**, behind one command an agent or CI runs. nanochat is the inspiration for the
406     wrapper; ms-swift is the engine; the **golden QA eval is the report card**.
407
408     > **Prerequisite (the crux):** the harness is only as trustworthy as the **golden QA
eval
409     > set**. nanochat's speedrun is meaningful *because* its eval (CORE score) is automatic
and
410     > trustworthy; ours is the golden QA eval ? **build that first**, or an agent-run loop
411     > optimizes blind. (Same "moat = dataset + eval", scaffolding-first principle.)
412
413     ### ?? Teacher / distillation constraint (the single biggest legal landmine)
414     You **cannot** train the owned model on outputs from the paid frontier APIs ? a general
415     video-QA assistant is squarely a **competing model**: **Gemini** ("may not use the
416     Services to develop models that compete"), **OpenAI** ("may not use Output to develop
417     models that compete"), **Anthropic** (bars using Outputs to train competing models). Two
418     safe lanes: ** (1) generate synthetic QA with **open-model teachers only** (Qwen-VL /
419     InternVL / Gemma 4 ? Apache-clean); ** (2) keep the paid API strictly as a **runtime
420     fallback** ? serving answers is fine, *harvesting its outputs to train the model that
421     replaces it is not*. Write a hard ***"no training on competitor-API outputs"*** rule into
422     eng + legal process. (This makes the ROADMAP's "Gemini as offline teacher only" precise:
423     the teacher is *open models*, not the paid APIs.)
424
425     ### Data flywheel, personalization & consent tiers
426     Two complementary loops, each with its **own** opt-in (training consent is separate from
427     processing consent ? $7):
428
429     - **Global model (shared ? raises the floor).** Train the base owned model on an
430     **explicitly-contributed, licensed** corpus ? *not* on whatever sits in a tenant's
bucket
431     (preserves "your video, your storage"). Flywheel: *more consented, labeled matches ? a
432     better shared model ? better highlights/answers ? more reason to contribute.* Higher
433     consent bar (pooled data, biometric/minor care ? $7); **minors excluded by default**.
434
435     - **Per-user adapters (personalized ? raises the ceiling) ? owner hunch, under
exploration.**
436     A small **per-user (or per-tenant) LoRA adapter, fine-tuned *only on that user's own
437     consented videos*** on top of the shared base, served via **multi-LoRA** (LoRAX /
438     Together / Predibase). Why it's compelling:
439     - **Precision:** adapts to *that user's* court, camera angle, lighting, and players'
440     styles ? materially better detection/highlights for them.
441     - **Cleanest consent story:** **"improve **your** detection using **your** data" is
far
442     lower-friction and lower-risk than contributing to a global model ? the data
improves
443     **only the user's own adapter**, with **no cross-user mixing**, sidestepping much of
the
444     pooled-corpus biometric/minor entanglement. Reinforces the differentiator: *your
video,

```

```

445     your storage, your compute ? and now **your model**. With BYO-compute the adapter
can be
446     trained/served on the user's own GPU and **never leave their boundary**; **deletion
=
447     drop the adapter** (clean erasure).
448 - **Open questions (still a hunch)** cold-start (need enough of the user's labeled
data
449     before a personal adapter beats the base ? fall back to the shared model meanwhile);
a
450     **per-user held-out eval** to prove the adapter helps (don't ship a regression);
base +
451     optional global-adapter + per-user-adapter stacking; per-user training cost at
scale.
452     Worth a spike once the harness + golden eval exist.
453
454     **Two consent tiers, then:** **a)** "improve **my own** detection with **my** data"
455     (per-user, low friction) and **b)** "contribute to the **shared** model" (global,
higher
456     bar). **Default for both is off.**
457
458     ## 6. Phased roadmap
459
460     | Phase | Deliverable | Build / reuse |
461     |---|---|---|
462     | **P0** | **Async job model, single-tenant** (DEFERRED #16) | `jobs` table via
`user_version` ladder; worker; 202+status endpoints |
463     | **P1** | **Identity + multi-tenant data model + upload + highlight persistence +
history** | auth (build vs Auth0/Clerk/Supabase ? decide); Postgres + `tenant_id`/RLS;
`highlights` table; repository layer; history API + UI |
464     | **P2** | **`StorageBackend` registry + BYO-storage** | new ABC+registry;
`local`>`gdrive`>`s3`>`gcs`; replace `video_path` with `StorageRef`; encrypted per-tenant creds
|
465     | **P3** | **`ComputeBackend` registry + BYO-compute** | `local`>`hosted`>`byo_worker`;
pull-based worker agent; control/data-plane split |
466     | **P4** | **Rich rally/shot semantic metadata & tagging** (the next value stage after
solid rally detection + highlights) | shot type (smash / backhand drive / net), rally
**outcome** (winner / forced / **unforced error**), context (e.g. **open court but error**) ?
query-API filters + `HighlightMetadata` + a **human correction/review loop** feeding the golden
set via the `annotation` registry + `candidate_segments` |
467     | **P5** | **Owned generative video-QA model** ($5A) | global flywheel **+ per-user
adapters**; P4 metadata is training signal ? fine-tune an open VLM (Qwen-VL) via LoRA ? serve as
`owned_vlm` **provider with Gemini paid fallback**; extend the eval harness to QA quality |
468     | **P6** ***(visionary)* | **Gait / biomechanics under a supervising coach** |
deliberately **back-burner**; technique feedback; highest Art. 9 bar ? gate behind explicit
biometric + parental consent ($7) |
469
470     P0?P1 deliver the headline ask (per-user upload + stored highlights + history). P2?P3
471     deliver the "plug-and-play storage/compute" ask. **P4 (rich semantic metadata) is the
472     immediate next value stage once rally detection + highlights are solid** ? e.g.
**rallies
473     that ended in a smash or backhand drive," **rallies where the court was open but the
474     player made an unforced error.** **P5** owns the model; **P6 (gait) is the visionary
475     horizon, intentionally deferred.**
476     **Sequencing principle (owner directive):** make the scaffolding (P0?P4) **rock-solid
477     first**, keep the **known paid stack as an always-ready fallback**, and only then shift
478     focus to **owned-model quality** (P5). The provider registry keeps the paid path one
479     config flip away.
480
481     ## 7. Privacy & legal (gait raises the bar ? design for it now)
482
483     - **The load-bearing nuance:** biometric data is special-category **only when used to
484     *uniquely identify* a person** (GDPR Art. 9). **Highlight/shot detection on players a
485     coach already knows is general analytics ? generally *not* Art. 9.** But
486     **gait/biomechanics that builds a per-athlete identifying template tips into Art. 9**
487     and needs **explicit, granular consent** referencing biometric data. Engineer gait

```

```

488     with that boundary (consent gate, purpose limitation, no cross-account
identification);
489     decide whether it runs **locally/on-tenant only** (strong posture) vs our cloud. (EDPB
490     Guidelines 3/2019 on video devices apply.)
491     - **Minors (e.g. academies).** Parental/guardian consent and safeguarding are first-
order,
492     not an afterthought. Model consent explicitly per relationship (user?subject,
coach?athlete
493     where the overlay applies, parent?minor).
494     - **BYO-storage is the central privacy/liability lever:** if raw video stays in the
495     tenant's own bucket/Drive and we process via signed URL or their compute, we minimize
496     what we hold ? shrinking breach surface, residency, and erasure obligations.
497     - **Paid-LLM no-train (C4):** use **Gemini via Vertex AI** (contractual no-training),
498     **not the consumer/free tier**; set short/zero retention, document it in the DPA, and
499     surface "your footage is never used to train AI" as a trust feature for academies. For
500     the gait phase, prefer keeping biometric inference off third-party APIs entirely.
501     - **Training consent is a distinct, higher bar (for the owned model, §5A).** Training on
502     user video needs a **separate, explicit, opt-in training licence** ? not bundled with
503     processing consent ? and it **compounds** the biometric (Art. 9) and minor-consent
504     issues above. Train **only on explicitly contributed/licensed video**; never train on
505     what merely passes through a tenant's bucket. **Exclude minors' video from the
training
506     pool by default**, keep the training corpus a **separate, firewalled, opt-in sub-
pool**,
507     and log **per-clip consent/provenance** for revocation + audit. Incentives must not
make
508     consent non-free (the product stays fully usable without contributing). And the
509     **teacher-distillation rule** (§5A): never train on paid-LLM outputs. **Two consent
tiers
510     (§5A):** (a)* "improve **my own** detection with **my** data" ? a per-user adapter
trained
511     only on the user's own video (lowest friction; no cross-user mixing; deletion = drop
the
512     adapter); (b)* "contribute to the **shared** model" ? pooled corpus, higher bar.
Default
513     for both is **off**.
514     - **Right to erasure / retention:** per-tenant, per-athlete delete must cascade across
DB +
515     storage artifacts (the `ON DELETE CASCADE` discipline already in `database.py` extends
here).
516
517     ## 8. Key decisions for the owner (the forks)
518
519     1. **Hosted-first vs BYO-first.** Recommended: **hosted control-plane first** (P0?P1)
with
520     **BYO-storage soon after** (P2); **BYO-compute later** (P3) as a power-
user/enterprise
521     option ? it's the highest-complexity, highest-security item.
522     2. **Database now or later.** Move to **Postgres at P1** for the hosted path, or stay
523     SQLite-per-tenant if self-hosted-single-academy is the first GTM. (Abstract DB access
524     regardless.)
525     3. **Auth: build vs buy** (Auth0 / Clerk / Supabase Auth / Cognito).
526     4. **Orchestration tech** (see research recommendation) ? bias to the simplest that
gives
527     retries + status.
528     5. **Gait processing location** ? local/on-tenant (privacy-max) vs cloud (capability-
max).
529     6. **How far to push BYO-compute** given its security/ops complexity.
530     7. **Owned-model base + licensing (§5A).** Which commercial-OK video VLM to build on
531     (Qwen-VL Apache-2.0 vs Gemma 3 custom license vs ?) and confirm a gated commercial
532     derivative is permitted ? see COMMERCIALIZATION C14.
533     8. **Training-data sourcing & consent.** Opt-in contribution terms, incentives, and the
534     teacher-distillation constraint (whether paid-LLM outputs may seed the corpus).
535     9. **When to start the owned model** ? strictly after the scaffolding + flywheel are
536     solid (the explicit directive), with the paid provider as the standing fallback.

```

537

538 ****Risks:**** scope explosion (sequence strictly P0?P5); multi-tenant data-leak bugs (RLS +

539 tests); BYO-compute security; ****biometric/minor liability**** (get this right before any

540 gait feature ships); ****training on user video without proper opt-in/licence****, and

541 ****base-model / teacher-distillation licence violations****; cost of hosted GPU (training

542 ***and* inference**); ****premature model-building**** before data/eval are ready (mitigated by

543 keeping the paid fallback and gating P5 behind the flywheel).

544

545 **## 9. Sources**

546

547 External best-practices research (2026-06-07) backing the recommendations above. Each

548 underlying claim in the research was tagged fact-vs-synthesis; figures are as-of and

549 should be re-validated before committing budget.

550

551 ****Storage abstraction:**** [fsspec](https://fsspec.com/) ·

[cloudpathlib](https://github.com/drivendataorg/cloudpathlib) ·

[smart_open](https://github.com/piskvorky/smart_open) · [Apache

Libcloud](https://libcloud.apache.org/) · [gdrive-fsspec](https://github.com/fsspec/gdrive-

fsspec) · [dropboxdrivefs](https://github.com/fsspec/dropboxdrivefs) · [AWS S3 presigned

URLs](https://docs.aws.amazon.com/AmazonS3/latest/userguide/using-presigned-url.html) · [token-

based S3 with STS](https://oneuptime.com/blog/post/2026-02-12-token-based-access-s3-sts/view) ·

[signed URLs across clouds](https://medium.com/@shaikhsharwan49/signed-urls-in-aws-gcp-and-azure-

a-beginner-friendly-guide-e2e549425865)

552

553 ****Multi-tenancy:**** [Bytebase: multi-tenant DB

patterns](https://www.bytebase.com/blog/multi-tenant-database-architecture-patterns-explained/)

· [Hunchbite: RLS vs schema-per-tenant](https://hunchbite.com/guides/multi-tenant-saas-

architecture) · [PlanetScale: tenancy in Postgres](https://planetscale.com/blog/approaches-to-

tenancy-in-postgres) · [DEV: DB-per-tenant vs shared schema](https://dev.to/young_gao/multi-

tenant-architecture-database-per-tenant-vs-shared-schema-1n2e)

554

555 ****BYO-compute / orchestration:**** [Pinecone BYOC](https://www.pinecone.io/blog/byoc/) ·

[Nuon: control/data-plane](https://nuon.co/blog/byoc-control-plane-data-plane-architectures) ·

[GitHub self-hosted runners (ARC)](https://docs.github.com/en/actions/hosting-your-own-

runners/managing-self-hosted-runners-with-actions-runner-controller/about-actions-runner-

controller) · [SkyPilot](https://github.com/skypilot-org/skypilot) · [SkyPilot AI control

plane](https://blog.skypilot.co/ai-job-orchestration-pt2-ai-control-plane/) · [Modal (AWS

Marketplace)](https://aws.amazon.com/marketplace/pp/prodview-j727623xqhh2k) · [Northflank:

Anyscale alternatives](https://northflank.com/blog/anyscale-alternatives-for-ai-ml-model-

deployment) · [DevPro: Python task queues

2025](https://devproportal.com/languages/python/python-background-tasks-celery-rq-dramatiq-

comparison-2025/) · [Judoscale: choosing a task queue](https://judoscale.com/blog/choose-python-

task-queue) · [Hatchet vs Temporal](https://hatchet.run/versus/hatchet-vs-temporal) ·

[Hatchet](https://github.com/hatchet-dev/hatchet)

556

557 ****Privacy / legal:**** [GDPR Art. 9](https://gdpr-info.eu/art-9-gdpr/) · [GDPR Local:

biometric compliance](https://gdprlocal.com/biometric-data-gdpr-compliance-made-simple/) ·

[Legiscope: Art. 9](https://www.legiscope.com/blog/gdpr-article-9-special-categories.html) ·

[EDPB Guidelines 3/2019 (video)](https://www.edpb.europa.eu/sites/default/files/files/file1/edpb

_guidelines_201903_video_devices_en_0.pdf) · [Fox Rothschild: EDPB video

guidance](https://www.foxrothschild.com/publications/video-surveillance-under-gdpr-edpb-issues-

final-guidance) · [ICO: children & UK GDPR](https://ico.org.uk/for-organisations/uk-gdpr-

guidance-and-resources/childrens-information/children-and-the-uk-gdpr-old/what-are-the-rules-

about-an-iss-and-consent/) · [EDPB Guidelines 05/2020 (consent)](https://www.edpb.europa.eu/site

s/default/files/files/file1/edpb_guidelines_202005_consent_en.pdf) · [AI data-retention

compared](https://ax-sentinel.com/blog/ai-data-retention-policies-compared) · [Xenoss:

enterprise LLM platforms](https://xenoss.io/blog/openai-vs-anthropic-vs-google-gemini-

enterprise-llm-platform-guide)

558

559 ****Comparable products:**** [Veo](https://www.veo.com/) · [Veo Help

Center](https://support.veo.com/hc/en-us/articles/4459159526929-Overview-of-cam-veo-co-Record-

manage-and-upload-footage-with-your-Veo-Cam-1) · [Latterly: Hudl

competitors](https://www.latterly.org/hudl-competitors/) · [Apple: Behind the Design ?

SwingVision](https://developer.apple.com/news/?id=0pg4dthn) · [SwingVision

FAQ](https://swing.vision/faq) · [Sportsvisio: coach video-analysis

comparison](https://www.sportsvisio.com/stories/video-analysis-software-for-coaches-complete-comparison)

560

561 **Training backends & harness (§5A):** [karpathy/nanochat (harness reference)](https://github.com/karpathy/nanochat) · [HF nanoVLM (minimal VLM)](https://github.com/huggingface/nanoVLM) · [nanoVLM blog](https://huggingface.co/blog/nanovlm) · [Tinker (GA + vision)](https://thinkingmachines.ai/blog/tinker-general-availability/) · [Tinker model lineup](https://tinker-docs.thinkingmachines.ai/model-lineup) · [Tinker ToS (weights/no-train)](https://thinkingmachines.ai/legal/tos/) · [Fireworks VLM fine-tuning](https://fireworks.ai/blog/vlm-tuning) · [Together AI fine-tuning](https://www.together.ai/fine-tuning) · [Predibase: fine-tune & serve VLMs](https://predibase.com/blog/how-to-fine-tune-and-serve-vlms-in-predibase) · [Predibase RFT/GRPO](https://docs.predibase.com/user-guide/fine-tuning/grpo) · [ms-swift (native video/audio)](https://github.com/modelscope/ms-swift) · [LLaMA-Factory (VLMs)](https://github.com/hiyouga/LlamaFactory) · [Unsloth (Qwen3-VL)](https://qwen.readthedocs.io/en/latest/training/unsloth.html) · [Fine-tuning tools 2026 (LoRA/QLoRA)](https://www.index.dev/blog/top-ai-fine-tuning-tools-lora-vs-qlora-vs-full)

562

563 **Owned model & licensing (§5A):** [Qwen2.5-VL](https://qwenlm.github.io/blog/qwen2.5-vl/) · [Qwen2.5 license split](https://qwenlm.github.io/blog/qwen2.5/) · [Qwen3-VL](https://github.com/QwenLM/Qwen3-VL) · [Gemma 3 (multimodal)](https://huggingface.co/blog/gemma3) · [Gemma Terms of Use](https://ai.google.dev/gemma/terms) · [Gemma license-risk analysis](https://wcr.legal/google-gemma-license-risks/) · [Gemma 4 / Apache-2.0](https://www.mindstudio.ai/blog/what-is-gemma-4-apache-2-license-commercial-ai-deployment) · [Llama 3.2 license (700M MAU + naming)](https://www.llama.com/llama3_2/license/) · [gpt-oss (text-only, Apache-2.0)](https://openai.com/index/introducing-gpt-oss/) · [Gemini API terms (compete clause)](https://ai.google.dev/gemini-api/terms) · [OpenAI terms](https://openai.com/policies/row-terms-of-use/) · [Anthropic: outputs & training](https://support.claude.com/en/articles/12326764-can-i-use-my-outputs-to-train-an-ai-model) · [QLoRA practical](https://apxml.com/courses/fine-tuning-adapting-large-language-models/chapter-4-parameter-efficient-fine-tuning/practice-qlora-fine-tuning) · [VLM LoRA cookbook](https://huggingface.co/learn/cookbook/en/fine_tuning_vlmtrl) · [GDPR Art. 9](https://gdpr-info.eu/art-9-gdpr/) · [athletes & biometric data](https://www.architectureandgovernance.com/applications-technology/biometric-data-and-athletes-privacy-law-and-compliance-implications/) · [federated learning + DP](https://research.google/blog/federated-learning-with-formal-differential-privacy-guarantees/) · [self-hosted LLM VRAM/cost](https://www.gpu-mart.com/guides/self-hosted-llm) · [Gemini API pricing](https://ai.google.dev/gemini-api/docs/pricing)

564

565 ## See also

566 - [`COMMERCIALIZATION.md`](COMMERCIALIZATION.md) ? C12 (SaaS foundations), C4 (paid-LLM), economics.

567 - [`DEFERRED\_HARDENING\_PLAN.md`](DEFERRED_HARDENING_PLAN.md) ? #16 async job model (the P0 infra slice), #7 packaging.

568 - [`PMF\_MARKET\_RESEARCH.md`](PMF_MARKET_RESEARCH.md) ? academy/coach beachhead (who this serves).

569 - [`CODE\_MAP.md`](CODE_MAP.md) ? current architecture + the registry extension points.

570 - [`ROADMAP.md`](ROADMAP.md) ? offline self-improving rally detection (the pipeline's own trajectory).

571

4. user (2026-06-10T05:59:44.280Z)

```

1      import os
2      import sys
3      import argparse
4      import json
5      import logging
6      import uvicorn
7      from dotenv import load_dotenv
8
9      load_dotenv()
10

```

```

11     # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).
12     logging.basicConfig(
13         level=logging.INFO,
14         format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',
15         datefmt='%H:%M:%S'
16     )
17     logger = logging.getLogger(__name__)
18     from fastapi import FastAPI, HTTPException
19     from fastapi.middleware.cors import CORSMiddleware
20     from pydantic import BaseModel
21     from typing import List, Dict, Any, Optional
22
23     from fastapi.staticfiles import StaticFiles
24     from fastapi.responses import HTMLResponse
25     from backend.pipeline.validators.base import registry
26     from backend.infrastructure.database import Database
27     from backend.pipeline.segmenters.base import segmenter_registry, BasePlaySegmenter
28     from backend.pipeline.stitcher.compiler import VideoCompiler
29     from backend.utils.hashing import compute_video_id # canonical file-identity hashing
30     from backend.config import load_config as load_app_config, AppConfig # new typed config
31     from backend.results import Result, Failure # standardized error/result contract
32     from backend.reporting import emit_indexer_report
33
34     app = FastAPI(title="Sports Highlight Indexer API")
35
36     # Enable CORS for local web interface
37     app.add_middleware(
38         CORSMiddleware,
39         allow_origins=["*"],
40         allow_credentials=True,
41         allow_methods=["*"],
42         allow_headers=["*"],
43     )
44
45     # Serves static frontend files directly (now under api/ per approved structure)
46     os.makedirs("backend/api/static", exist_ok=True)
47     app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")
48
49     @app.get("/", response_class=HTMLResponse)
50     def serve_index():
51         index_path = "backend/api/static/index.html"
52         if os.path.exists(index_path):
53             with open(index_path, "r", encoding="utf-8") as f:
54                 return f.read()
55         return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
backend/api/static/</h3>"
56
57     # Global variables initialized at startup
58     db_instance: Optional[Database] = None
59     global_config: Optional[AppConfig] = None # now typed (Pydantic model)
60
61     # Legacy thin wrapper for any remaining direct calls during transition.
62     # New code should import load_app_config / AppConfig from backend.config directly.
63     def load_config(config_path: str = "config.json") -> Dict[str, Any]:
64         cfg = load_app_config(config_path)
65         return cfg.model_dump(mode="json")
66
67     # --- API Models ---
68     class ProcessRequest(BaseModel):
69         video_path: str
70         player_pool: Optional[List[str]] = None
71         max_frames: Optional[int] = None
72         segmenter: Optional[str] = None
73

```



```

74     class QueryRequest(BaseModel):
75         video_id: str
76         server: Optional[str] = None
77         receiver: Optional[str] = None
78         duration_min: Optional[float] = None
79         event_type: Optional[str] = None
80
81     class CompileRequest(BaseModel):
82         video_id: str
83         segment_ids: List[int]
84         output_filename: str
85
86     # --- API Endpoints ---
87     @app.get("/api/config")
88     def get_config():
89         return global_config
90
91     @app.post("/api/process")
92     def process_video(req: ProcessRequest):
93         if not os.path.exists(req.video_path):
94             raise HTTPException(status_code=404, detail=f"Video file not found at
95 '{req.video_path}'")
96
97         video_id = compute_video_id(req.video_path)
98         was_reingest = db_instance.get_video(video_id) is not None
99
100        # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
101        logger.info(f"Running validations for {req.video_path}...")
102        val_results, val_context = registry.run_all_with_context(req.video_path,
global_config)
103        failures = [r for r in val_results if not r.passed]
104
105        # typed AppConfig: attribute access for the default segmenter (with safe fallback)
106        default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
107        seg_name = req.segmenter or default_seg
108        segmenter_actual = None
109        segmentation_ran = False
110        response: Optional[Dict[str, Any]] = None
111        try:
112            if failures:
113                db_instance.add_video(video_id, req.video_path, "failed", [r.dict() for r in
val_results])
114                response = {
115                    "video_id": video_id,
116                    "status": "failed",
117                    "message": "Video failed validation checks.",
118                    "results": [r.dict() for r in val_results],
119                }
120            return response
121
122        # 2. Run segmenter & indexer
123        print(f"[API] Video passed checks. Segmenting and indexing...")
124        db_instance.add_video(video_id, req.video_path, "processed", [r.dict() for r in
val_results])
125
126        segmenter_cls = segmenter_registry.get(seg_name)
127        if not segmenter_cls:
128            available = list(segmenter_registry.list_available().keys())
129            raise HTTPException(
130                status_code=400,
131                detail=f"Segmenter '{seg_name}' not found. Available segmenters:
{available}"

```

```

132
133     print(f"[API] Using segmenter: {seg_name}")
134     segmenter_actual = seg_name
135     segmenter = segmenter_cls(db_instance, global_config)
136     result = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)
137     segmentation_ran = True
138
139     if isinstance(result, Failure):
140         print(f"[API] Segmenter failed: {result.message}")
141         response = {
142             "video_id": video_id,
143             "status": "failed",
144             "message": f"Segmenter '{seg_name}' failed: {result.message}",
145             "results": [r.dict() for r in val_results],
146             "play_segments": [],
147         }
148         return response
149
150     segments = result.value if hasattr(result, "value") else result
151
152     response = {
153         "video_id": video_id,
154         "status": "success",
155         "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
156         "results": [r.dict() for r in val_results],
157         "play_segments": segments,
158     }
159     return response
160 finally:
161     # Always emit the per-video observability report (next to the video, or to
162     # report.output_dir). Never raises. Surface the paths in the response.
163     paths = emit_indexer_report(
164         db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
165         val_results=val_results, val_context=val_context,
166         segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
167         was_reingest=was_reingest, segmentation_ran=segmentation_ran,
168     )
169     if paths and isinstance(response, dict):
170         response["reports"] = paths
171
172 @app.post("/api/query")
173 def query_play_segments(req: QueryRequest):
174     filters = {
175         "server": req.server,
176         "receiver": req.receiver,
177         "duration_min": req.duration_min,
178         "event_type": req.event_type
179     }
180     segments = db_instance.query_play_segments(req.video_id, filters)
181     return {"play_segments": segments}
182
183 @app.post("/api/compile")
184 def compile_highlights(req: CompileRequest):
185     video = db_instance.get_video(req.video_id)
186     if not video:
187         raise HTTPException(status_code=404, detail="Indexed video record not found.")
188
189     # Retrieve matching play segments from db
190     all_segments = db_instance.query_play_segments(req.video_id, {})
191     matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]
192
193     if not matched_segments:

```

```

194         raise HTTPException(status_code=400, detail="No matching play segments found to
stitch.")
195
196     # Extract start/end time pairs
197     time_pairs = [(s["start_time"], s["end_time"]) for s in matched_segments]
198
199     # Run compiler
200     output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
201     compiler = VideoCompiler(output_dir)
202     try:
203         out_path = compiler.compile_highlights(video["filepath"], time_pairs,
req.output_filename)
204         return {"status": "success", "filepath": out_path}
205     except Exception as e:
206         raise HTTPException(status_code=500, detail=f"Highlight stitching failed:
{str(e)}")
207
208     # --- CLI Debug Utility & Orchestration ---
209     def run_cli_diagnose_clip(video_path: str, timestamp: float):
210         """CLI debugging utility to examine segment properties around a timestamp."""
211         print("-" * 60)
212         print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
213         print("-" * 60)
214
215         cfg = load_config() # legacy wrapper still works, returns dict for now
216
217         # 1. Validation check diagnostics
218         print("[DIAGNOSTIC] Running validation framework...")
219         results = registry.run_all(video_path, cfg)
220         for r in results:
221             status_symbol = "[PASS]" if r.passed else "[FAIL]"
222             print(f" {status_symbol} {r.validator_name}: {r.message}")
223             if r.details:
224                 print(f"      Details: {r.details}")
225
226         # 2. Local segment analysis around the timestamp (sampling +/- 5 seconds)
227         print("-" * 60)
228         print("[DIAGNOSTIC] Analyzing local motion around the timestamp...")
229         import cv2
230         cap = cv2.VideoCapture(video_path)
231         if not cap.isOpened():
232             print("[FAIL] OpenCV could not open video.")
233             return
234
235         fps = cap.get(cv2.CAP_PROP_FPS)
236         total_frames = cap.get(cv2.CAP_PROP_FRAME_COUNT)
237
238         start_frame = max(0, int((timestamp - 3.0) * fps))
239         end_frame = min(int(total_frames), int((timestamp + 3.0) * fps))
240
241         # Measure frame-by-frame changes in sample region
242         cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
243         prev_gray = None
244         motions = []
245
246         for f in range(start_frame, end_frame):
247             ret, frame = cap.read()
248             if not ret:
249                 break
250             gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
251             gray = cv2.GaussianBlur(gray, (21, 21), 0)
252
253             if prev_gray is not None:
254                 diff = cv2.absdiff(prev_gray, gray)

```

```

255         _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
256         motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
257         motions.append(motion_ratio)
258         prev_gray = gray
259
260     cap.release()
261
262     if motions:
263         avg_motion = sum(motions) / len(motions)
264         # Support both legacy dict (from wrapper) and future direct model
265         if hasattr(cfg, "indexing"):
266             threshold = getattr(cfg.indexing, "motion_threshold", 0.08)
267         else:
268             threshold = cfg.get("indexing", {}).get("motion_threshold", 0.08) # type:
ignore[attr-defined]
269         print(f" Sample Frame Count: {len(motions)}")
270         print(f" Average motion index around timestamp: {round(avg_motion, 4)}
(threshold: {threshold})")
271         if avg_motion > threshold:
272             print(" Result: [ACTIVE PLAY] High motion detected. Segment classifies as
active play.")
273         else:
274             print(" Result: [DEAD TIME] Idle movement. Segment classified as dead
time.")
275     else:
276         print(" [ERROR] No visual frames were extracted in the sample range.")
277
278     print("=" * 60)
279
280     def main():
281         parser = argparse.ArgumentParser(description="Sports Highlight Indexer
Orchestrator")
282         parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
283         parser.add_argument("--output-dir", default="output", help="Directory where
processed clips/highlights are saved")
284         parser.add_argument("--setup", action="store_true", help="Launch dependency checker
and setup utility")
285         parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a
video clip to run detailed diagnostics on")
286         parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
287         parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI
server on")
288         parser.add_argument("--max-frames", type=int, help="Limit processing to the first N
sub-sampled frames for debugging")
289
290         available_segmenters = list(segmenter_registry.list_available().keys())
291         parser.add_argument("--segmenter", choices=available_segmenters, help=f"Choose rally
detection segmenter. Available: {available_segmenters}")
292         parser.add_argument("--trim-start", type=float, help="Start time in seconds for the
debug trim segment feature")
293         parser.add_argument("--trim-end", type=float, help="End time in seconds for the
debug trim segment feature")
294         parser.add_argument("--trim-output", help="Output filename for the debug trimmed
segment (saves in output-dir unless absolute path)")
295
296         args = parser.parse_args()
297
298         if args.setup:
299             # Invoke setup diagnostics
300             import subprocess
301             print("[INFO] Invoking setup.py...")
302             subprocess.run([sys.executable, "setup.py"])
303             return

```

```

304
305     if args.trim_start is not None and args.trim_end is not None:
306         if not args.video_path:
307             print("[ERROR] Please provide --video-path to extract a segment.")
308             return
309
310         # Determine output path
311         out_filename = args.trim_output or "trimmed_segment.mp4"
312         if os.path.isabs(out_filename):
313             out_path = out_filename
314             out_dir = os.path.dirname(out_path)
315             out_name = os.path.basename(out_path)
316         else:
317             out_dir = args.output_dir
318             out_path = os.path.join(out_dir, out_filename)
319             out_name = out_filename
320
321         os.makedirs(out_dir, exist_ok=True)
322         print(f"[CLI] Extracting debug segment from {args.trim_start}s to
{args.trim_end}s from {args.video_path}...")
323
324         compiler = VideoCompiler(out_dir)
325         try:
326             res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
args.trim_end)], out_name)
327             print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
328         except Exception as e:
329             print(f"[ERROR] Failed to compile trimmed segment: {e}")
330         return
331
332     if args.debug_clip is not None:
333         if not args.video_path:
334             print("[ERROR] Please provide --video-path to debug a specific clip.")
335             return
336         run_cli_diagnose_clip(args.video_path, args.debug_clip)
337         return
338
339     # Initialize Global Config and DB
340     global global_config, db_instance
341     global_config = load_app_config() # returns typed AppConfig
342
343     # The CLI --output-dir overrides the configured output directory. It now lives
344     # on the typed model (OutputConfig.output_dir), so every consumer ? including
345     # /api/compile ? reads the same value instead of a write-only runtime hack.
346     global_config.output.output_dir = args.output_dir
347     os.makedirs(global_config.output.output_dir, exist_ok=True)
348
349     db_path = os.path.join(global_config.output.output_dir,
global_config.output.db_name)
350     db_instance = Database(db_path)
351
352     # CLI processing mode (no web UI needed)
353     if args.video_path and not args.server:
354         print(f"[CLI] Processing video file: {args.video_path}")
355         if not os.path.exists(args.video_path):
356             print(f"[FAIL] Video file not found: {args.video_path}")
357             return
358
359         video_id = compute_video_id(args.video_path)
360         was_reingest = db_instance.get_video(video_id) is not None
361
362         # Validate (with-context variant surfaces ffmpeg_meta for the report)
363         print(f"[CLI] Validating...")
364         val_results, val_context = registry.run_all_with_context(args.video_path,
global_config)

```

```

365         failures = [r for r in val_results if not r.passed]
366
367         # Save video status
368         status = "failed" if failures else "processed"
369         db_instance.add_video(video_id, args.video_path, status, [r.dict() for r in
val_results])
370
371         seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
"default_segmenter", "motion")
372         segmenter_actual = None
373         segmentation_ran = False
374         try:
375             print("\n" + "="*40)
376             print("VALIDATION SUMMARY")
377             print("="*40)
378             for r in val_results:
379                 status_symbol = "[PASS]" if r.passed else "[FAIL]"
380                 print(f"{status_symbol} {r.validator_name}: {r.message}")
381             print("="*40 + "\n")
382
383             if failures:
384                 print("[FAIL] Video failed checks. Indexing halted.")
385                 return
386
387             # Index
388             segmenter_cls = segmenter_registry.get(seg_name)
389             if not segmenter_cls:
390                 print(f"[FAIL] Segmenter '{seg_name}' not found.")
391                 return
392
393             print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
394             segmenter_actual = seg_name
395             segmenter = segmenter_cls(db_instance, global_config)
396             result = segmenter.process_video(video_id, args.video_path,
max_frames=args.max_frames)
397             segmentation_ran = True
398
399             if isinstance(result, Failure):
400                 print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
401                 print(f"[FAIL] Indexing halted for video.")
402                 segments = []
403             else:
404                 segments = result.value if hasattr(result, "value") else result
405                 print(f"[SUCCESS] Process complete. Indexed {len(segments)} play
segments inside database: {db_path}")
406             finally:
407                 # Always emit the per-video observability report. Never raises.
408                 paths = emit_indexer_report(
409                     db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,
410                     val_results=val_results, val_context=val_context,
411                     segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
412                     was_reingest=was_reingest, segmentation_ran=segmentation_ran,
413                 )
414             if paths:
415                 print(f"[REPORT] Technical report : {paths.get('technical_md')}")
416                 print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
417                 print(f"[REPORT] Machine JSON      : {paths.get('json')}")
418             return
419
420         # Start Local Server Mode
421         if args.server or not args.video_path:
422             print(f"[SERVER] Starting local FastAPI server on
http://localhost:{args.port}...")
423             uvicorn.run(app, host="127.0.0.1", port=args.port)

```

```
424
425     if __name__ == "__main__":
426         main()
427
```

5. user (2026-06-10T05:59:44.756Z)

```
1     import sqlite3
2     import json
3     import os
4     import logging
5     from typing import List, Dict, Any, Optional
6
7     logger = logging.getLogger(__name__)
8
9     class Database:
10         def __init__(self, db_path: str):
11             self.db_path = db_path
12             self._init_db()
13
14         def _get_connection(self):
15             conn = sqlite3.connect(self.db_path)
16             conn.row_factory = sqlite3.Row
17             # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
18             conn.execute("PRAGMA foreign_keys = ON")
19             return conn
20
21         def _init_db(self):
22             """Initializes tables for videos, play_segments, and events."""
23             with self._get_connection() as conn:
24                 cursor = conn.cursor()
25
26                 # Videos Table
27                 cursor.execute("""
28                     CREATE TABLE IF NOT EXISTS videos (
29                         id TEXT PRIMARY KEY,
30                         filepath TEXT NOT NULL,
31                         processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
32                         status TEXT NOT NULL,
33                         validation_results TEXT
34                     )
35                 """)
36
37                 # Play Segments Table (Extensible metadata JSON)
38                 cursor.execute("""
39                     CREATE TABLE IF NOT EXISTS play_segments (
40                         id INTEGER PRIMARY KEY AUTOINCREMENT,
41                         video_id TEXT NOT NULL,
42                         start_time REAL NOT NULL,
43                         end_time REAL NOT NULL,
44                         duration REAL NOT NULL,
45                         server TEXT,
46                         receiver TEXT,
47                         metadata TEXT,
48                         FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
49                     )
50                 """)
51
52                 # Events Table
53                 cursor.execute("""
54                     CREATE TABLE IF NOT EXISTS events (
55                         id INTEGER PRIMARY KEY AUTOINCREMENT,
56                         segment_id INTEGER NOT NULL,
57                         timestamp REAL NOT NULL,
58                         type TEXT NOT NULL,
```

```

59         player TEXT,
60         details TEXT,
61         FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE
CASCADE
62     )
63     """
64
65     # Versioned migrations live here. PRAGMA user_version is the schema
66     # contract ? bump it for every change and add an ordered `if version < N`
67     # block below. Tests assert the expected version, so accidental drift fails
68     CI.
69
70     version = cursor.execute("PRAGMA user_version").fetchone()[0]
71
72     if version < 1:
73         # v1: raw candidate detections (pre-confirmation), detector-agnostic.
74         # Common fields are columns; detector-specific metrics go in `stats`
75         JSON,
76         # so a new segmenter reuses this table by setting its own
77         `detector`/`stats`.
78         cursor.execute("""
79             CREATE TABLE IF NOT EXISTS candidate_segments (
80                 id INTEGER PRIMARY KEY AUTOINCREMENT,
81                 video_id TEXT NOT NULL,
82                 detector TEXT NOT NULL,
83                 chunk_index INTEGER,
84                 start_time REAL NOT NULL,
85                 end_time REAL NOT NULL,
86                 duration REAL NOT NULL,
87                 confidence REAL,
88                 stats TEXT,
89                 detection_params TEXT,
90                 confirmed_segment_id INTEGER,
91                 human_label TEXT,
92                 notes TEXT,
93                 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
94                 FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
95                 FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id)
96             ON DELETE SET NULL
97             )
98         """)
99         cursor.execute("""
100             CREATE INDEX IF NOT EXISTS idx_candidates_video_detector
101             ON candidate_segments(video_id, detector)
102         """)
103         cursor.execute("PRAGMA user_version = 1")
104         # future: if version < 2: cursor.execute("ALTER TABLE ..."); PRAGMA
105         user_version = 2
106
107         conn.commit()
108
109     def add_video(self, video_id: str, filepath: str, status: str, validation_results:
110     List[Dict[str, Any]]) -> bool:
111         try:
112             with self._get_connection() as conn:
113                 conn.cursor().execute(
114                     "INSERT OR REPLACE INTO videos (id, filepath, status,
115                     validation_results) VALUES (?, ?, ?, ?)",
116                     (video_id, filepath, status, json.dumps(validation_results))
117                 )
118                 conn.commit()
119             return True
120         except Exception as e:
121             logger.error(f"Failed to add video: {e}")
122             return False

```



```

116         def add_play_segment(self, video_id: str, start_time: float, end_time: float,
server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
= None) -> Optional[int]:
117             try:
118                 with self._get_connection() as conn:
119                     cursor = conn.cursor()
120                     cursor.execute(
121                         "INSERT INTO play_segments (video_id, start_time, end_time,
duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
122                         (video_id, start_time, end_time, end_time - start_time, server,
receiver, json.dumps(metadata or {})))
123                     )
124                     conn.commit()
125                     return cursor.lastrowid
126             except Exception as e:
127                 logger.error(f"Failed to add play segment: {e}")
128                 return None
129
130         def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
131             try:
132                 with self._get_connection() as conn:
133                     conn.cursor().execute(
134                         "INSERT INTO events (segment_id, timestamp, type, player, details)
VALUES (?, ?, ?, ?, ?)",
135                         (segment_id, timestamp, event_type, player, json.dumps(details or
{})))
136                     )
137                     conn.commit()
138                     return True
139             except Exception as e:
140                 logger.error(f"Failed to add event: {e}")
141                 return False
142
143         def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
144                                     chunk_index: Optional[int] = None, confidence:
Optional[float] = None,
145                                     stats: Optional[Dict[str, Any]] = None,
146                                     detection_params: Optional[Dict[str, Any]] = None) ->
Optional[int]:
147             """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
148             detector-specific dicts stored as JSON. Returns the new candidate id, or
None."""
149             try:
150                 with self._get_connection() as conn:
151                     cursor = conn.cursor()
152                     cursor.execute(
153                         """INSERT INTO candidate_segments
154                         (video_id, detector, chunk_index, start_time, end_time, duration,
155                         confidence, stats, detection_params)
156                         VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
157                         (video_id, detector, chunk_index, start_time, end_time, end_time -
start_time,
158                         confidence, json.dumps(stats or {}), json.dumps(detection_params or
{})))
159                     )
160                     conn.commit()
161                     return cursor.lastrowid
162             except Exception as e:
163                 logger.error(f"Failed to add candidate segment: {e}")
164                 return None
165
166         def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
167             """Record that a candidate detection was confirmed as a given play_segment."""

```

```

168         try:
169             with self._get_connection() as conn:
170                 conn.cursor().execute(
171                     "UPDATE candidate_segments SET confirmed_segment_id = ? WHERE id =
172                     ?",
173                     (segment_id, candidate_id)
174                 )
175                 conn.commit()
176             return True
177         except Exception as e:
178             logger.error(f"Failed to link candidate {candidate_id} to segment
179             {segment_id}: {e}")
180             return False
181
182     def query_candidate_segments(self, video_id: str, detector: Optional[str] = None,
183                                only_unlabeled: bool = False) -> List[Dict[str, Any]]:
184         """Fetch candidate detections for the annotation / fine-tuning workflow.
185         `stats` and `detection_params` are deserialized from JSON."""
186         query = "SELECT * FROM candidate_segments WHERE video_id = ?"
187         params: List[Any] = [video_id]
188
189         if detector:
190             query += " AND detector = ?"
191             params.append(detector)
192         if only_unlabeled:
193             query += " AND human_label IS NULL"
194
195         query += " ORDER BY start_time"
196
197         candidates = []
198         with self._get_connection() as conn:
199             rows = conn.cursor().execute(query, params).fetchall()
200             for row in rows:
201                 d = dict(row)
202                 d["stats"] = json.loads(d["stats"] or "{}")
203                 d["detection_params"] = json.loads(d["detection_params"] or "{}")
204                 candidates.append(d)
205             return candidates
206
207     def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:
208         with self._get_connection() as conn:
209             row = conn.cursor().execute("SELECT * FROM videos WHERE id = ?",
210             (video_id,)).fetchone()
211             if row:
212                 d = dict(row)
213                 d["validation_results"] = json.loads(d["validation_results"] or "[]")
214                 return d
215             return None
216
217     def query_play_segments(self, video_id: str, filters: Dict[str, Any]) ->
218     List[Dict[str, Any]]:
219         """
220         Dynamically queries play segments based on filters:
221         - duration_min: float
222         - server: string
223         - receiver: string
224         - event_type: string (e.g. smash, foul)
225         """
226         query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
227         params = [video_id]
228
229         if filters.get("duration_min"):
230             query += " AND r.duration >= ?"
231             params.append(filters["duration_min"])

```

```

229         if filters.get("server"):
230             query += " AND r.server = ?"
231             params.append(filters["server"])
232
233         if filters.get("receiver"):
234             query += " AND r.receiver = ?"
235             params.append(filters["receiver"])
236
237         if filters.get("event_type"):
238             # Check if there is an event of a specific type inside this segment
239             query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND
e.type = ?)"
240             params.append(filters["event_type"])
241
242         segments_list = []
243         with self._get_connection() as conn:
244             cursor = conn.cursor()
245             rows = cursor.execute(query, params).fetchall()
246
247             for row in rows:
248                 r_dict = dict(row)
249                 r_id = r_dict["id"]
250                 r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")
251
252                 # Fetch associated events
253                 event_rows = cursor.execute("SELECT * FROM events WHERE segment_id = ?",
(r_id,)).fetchall()
254                 r_dict["events"] = []
255                 for er in event_rows:
256                     e_dict = dict(er)
257                     e_dict["details"] = json.loads(e_dict["details"] or "{}")
258                     r_dict["events"].append(e_dict)
259
260                 segments_list.append(r_dict)
261
262         return segments_list
263
264     def clear_video_data(self, video_id: str):
265         with self._get_connection() as conn:
266             conn.cursor().execute("DELETE FROM videos WHERE id = ?", (video_id,))
267             conn.commit()
268

```

6. assistant (2026-06-10T06:00:12.251Z)

Now let me map the rest of the code surface.

7. user (2026-06-10T06:00:13.912Z)

```

/c/Users/avidu/Projects/badminton-highlight-indexer/backend/annotations/__init__.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/annotations/automated_provider.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/annotations/base.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/annotations/file_provider.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/config/__init__.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/config/loader.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/config/models.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/__init__.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/annotations.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/audio/__init__.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/audio/e1_probe.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/batch.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/calibrate_local.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/calibrate_wasb.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/calibration.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/classifier.py

```

/c/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/distill_local.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/eval_partial_labels.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/export_wasb_segments.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/gemini_refine.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/harness.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/metrics.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/rally_seq_proto.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/regression.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/training.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/infrastructure/database.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/main.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/audio/__init__.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/audio/hit_detector.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/detectors/__init__.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/detectors/base.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/detectors/rally_gate.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/detectors/tracknet_runner.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/detectors/wasb_infer.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/detectors/wasb_runner.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/segmenters/__init__.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/segmenters/base.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/segmenters/gemini.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/segmenters/motion.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/segmenters/tracknet_hybrid.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/segmenters/wasb_hybrid.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/segmenters/yolo_hybrid.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/stitcher/compiler.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/validators/__init__.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/validators/base.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/validators/blur.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/validators/format.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/validators/pts_continuity.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/validators/relevance.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/validators/resolution_fps.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/validators/scene_cut.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/providers/__init__.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/providers/base.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/providers/gemini.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/reporting/__init__.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/reporting/harvest.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/results.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/sports/__init__.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/sports/badminton.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/sports/base.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/tools/__init__.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/tools/export.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/tools/rally_slicer.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/utils/__init__.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/utils/geometry.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/utils/hashing.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/utils/video.py

8. user (2026-06-10T06:00:14.315Z)

run_all_tests.py
run_eval.py
run_phase3_eval.py
run_process_and_stitch.py
run_regression.py

9. user (2026-06-10T06:00:18.679Z)

```

1     import os
2     import time
3     import json
4     import logging
5     from typing import Tuple
6     from google import genai
7     from google.genai import types
8     from backend.providers.base import AIVideoProvider
9
10    logger = logging.getLogger(__name__)
11
12    # The Gemini File API caps uploads at 2GB; stay well under it. Callers must chunk so
13    # each uploaded clip is below this. Oversize uploads are refused (not silently
14    # attempted).
15    MAX_UPLOAD_MB = 500
16
17    class GeminiProvider(AIVideoProvider):
18        def __init__(self, api_key: str, model_name: str):
19            super().__init__(api_key, model_name)
20            # Initialize client lazily to avoid connection/auth issues during class
21            # instantiation
22            self._client = None
23
24            @property
25            def client(self):
26                if self._client is None:
27                    self._client = genai.Client(api_key=self.api_key)
28                return self._client
29
30            def analyze_video(self, video_path: str, prompt: str,
31                             chunk_duration: float = 0.0,
32                             label: str = "") -> Tuple[list, dict]:
33                log_prefix = f"Gemini {label}" if label else "Gemini"
34
35                metrics = {
36                    "upload_time": 0.0,
37                    "process_time": 0.0,
38                    "gen_time": 0.0
39                }
40
41                # 0. Refuse oversize uploads (Gemini File API limit is 2GB; we cap lower for
42                # safety/speed)
43                try:
44                    size_mb = os.path.getsize(video_path) / (1024 * 1024)
45                except OSError:
46                    size_mb = 0.0
47                if size_mb > MAX_UPLOAD_MB:
48                    logger.error(f"[{log_prefix}] {video_path} is {size_mb:.0f}MB >
49                    {MAX_UPLOAD_MB}MB cap ? "
50                                f"chunk smaller. Skipping upload.")
51                    return [], metrics
52
53                # 1. Upload (retry transient network failures, e.g. WinError 10053 / reset
54                # connections)
55                logger.info(f"[{log_prefix}] Uploading {video_path}...")
56                t_upload_start = time.time()
57                video_file = None
58                for attempt in range(3):
59                    try:
60                        video_file = self.client.files.upload(file=video_path)
61                        metrics["upload_time"] = time.time() - t_upload_start
62                        logger.info(f"[{log_prefix}] Upload complete in
63                        {metrics['upload_time']:.1f}s. URI: {video_file.uri}")
64                        break
65                    except Exception as e:

```

```

60         logger.warning(f"[{log_prefix}] Upload attempt {attempt + 1}/3 failed:
{e}")
61         if attempt < 2:
62             time.sleep(2 * (attempt + 1))
63         if video_file is None:
64             logger.error(f"[{log_prefix}] Upload failed after 3 attempts.")
65             return [], metrics
66
67         # 2. Wait for processing on Google servers
68         t_process_start = time.time()
69         try:
70             while video_file.state.name == "PROCESSING":
71                 logger.info(f"[{log_prefix}] Processing on Google servers. Elapsed:
{time.time()-t_process_start:.1f}s. Retrying in 10s...")
72                 time.sleep(10)
73                 video_file = self.client.files.get(name=video_file.name)
74                 if video_file.state.name == "FAILED":
75                     logger.error(f"[{log_prefix}] Video processing failed:
{video_file.error.message}")
76                     try:
77                         self.client.files.delete(name=video_file.name)
78                     except Exception:
79                         pass
80                     return [], metrics
81             except Exception as e:
82                 logger.error(f"[{log_prefix}] Status check failed: {e}")
83                 try:
84                     self.client.files.delete(name=video_file.name)
85                 except Exception:
86                     pass
87                 return [], metrics
88
89             metrics["process_time"] = time.time() - t_process_start
90             logger.info(f"[{log_prefix}] Processing finished in
{metrics['process_time']:.1f}s.")
91
92         # 3. Call Gemini (retry transient 503/UNAVAILABLE model-overload + network
blips)
93         logger.info(f"[{log_prefix}] Submitting multimodal request to
{self.model_name}...")
94         t_gen_start = time.time()
95         raw_text = None
96         for attempt in range(3):
97             try:
98                 response = self.client.models.generate_content(
99                     model=self.model_name,
100                     contents=[video_file, prompt],
101                     config=types.GenerateContentConfig(
102                         response_mime_type="application/json",
103                         temperature=0.0
104                     )
105                 )
106                 raw_text = response.text
107                 metrics["gen_time"] = time.time() - t_gen_start
108                 logger.info(f"[{log_prefix}] Response received in
{metrics['gen_time']:.1f}s:\n{raw_text}")
109                 break
110             except Exception as e:
111                 logger.warning(f"[{log_prefix}] generate attempt {attempt + 1}/3 failed:
{e}")
112                 if attempt < 2:
113                     time.sleep(3 * (attempt + 1))
114                 try:
115                     self.client.files.delete(name=video_file.name)
116                 except Exception:

```

```

117         pass
118
119     if not raw_text:
120         return [], metrics
121
122     # 4. Parse JSON
123     try:
124         segments = json.loads(raw_text)
125         if not isinstance(segments, list):
126             logger.error(f"[{log_prefix}] Response is not a JSON array.")
127             return [], metrics
128         return segments, metrics
129     except Exception as e:
130         logger.error(f"[{log_prefix}] JSON parse failed: {e}")
131         return [], metrics
132

```

10. user (2026-06-10T06:00:19.151Z)

```

1     import os
2     import subprocess
3     import tempfile
4     import logging
5     from typing import List, Tuple, Union
6
7     logger = logging.getLogger(__name__)
8
9     class VideoCompiler:
10         def __init__(self, output_dir: str, end_padding: float = 1.0, begin_padding: float =
11             0.5):
12             self.output_dir = output_dir
13             self.end_padding = end_padding
14             self.begin_padding = begin_padding
15             os.makedirs(output_dir, exist_ok=True)
16
17         @staticmethod
18         def _parse_time(val: Union[int, float, str]) -> float:
19             """Normalizes a timestamp value to float seconds.
20             Handles: float/int (pass-through), plain numeric strings ("12.4"),
21             and colon-separated strings ("MM:SS", "HH:MM:SS", "H:MM:SS.ms").
22             This mirrors the parse_time logic used in gemini.py to ensure
23             the stitcher can always consume whatever format the indexer produced."""
24             if isinstance(val, (int, float)):
25                 return float(val)
26             if isinstance(val, str):
27                 val = val.strip()
28                 if ":" in val:
29                     # Handle MM:SS, HH:MM:SS, etc.
30                     parts = val.split(":")
31                     parts.reverse()
32                     total = 0.0
33                     for i, p in enumerate(parts):
34                         try:
35                             total += float(p) * (60 ** i)
36                         except ValueError:
37                             pass
38                     return total
39                 try:
40                     return float(val)
41                 except ValueError:
42                     logger.warning(f"Could not parse timestamp value '{val}' as a number.
43                     Defaulting to 0.0.")
44                     return 0.0
45                 logger.warning(f"Unexpected timestamp type {type(val).__name__} for value
46                 '{val}'. Defaulting to 0.0.")

```

```

44         return 0.0
45
46     def _get_video_duration(self, video_path: str) -> float:
47         """Probes the video file to get its total duration in seconds.
48         Returns 0.0 if it cannot be determined."""
49         try:
50             result = subprocess.run(
51                 [
52                     "ffprobe", "-v", "error",
53                     "-show_entries", "format=duration",
54                     "-of", "default=noprint_wrappers=1:nokey=1",
55                     video_path
56                 ],
57                 capture_output=True, text=True, check=True
58             )
59             return float(result.stdout.strip())
60         except Exception as e:
61             logger.warning(f"Could not determine video duration via ffprobe: {e}")
62             return 0.0
63
64     def compile_highlights(self, raw_video_path: str, segments: List[Tuple[float,
65 float]], output_filename: str) -> str:
66         """
67         Extracts selected segments from a raw video and stitches them together
68         into a seamless highlights file, applying end_padding to prevent abrupt endings.
69         """
70         if not segments:
71             raise ValueError("No highlight segments provided to compile.")
72
73         if not os.path.exists(raw_video_path):
74             raise FileNotFoundError(f"[COMPILER ERROR] Source video file not found:
75 {raw_video_path}")
76
77         output_path = os.path.join(self.output_dir, output_filename)
78
79         # Probe video duration so we can detect out-of-range segments
80         video_duration = self._get_video_duration(raw_video_path)
81         if video_duration > 0:
82             logger.info(f"Source video duration: {video_duration:.2f}s")
83         else:
84             logger.warning("Video duration unknown. Cannot validate segment boundaries
85 against video length.")
86
87         # We will create temporary clips and stitch them using FFmpeg concat demuxer
88         temp_clips = []
89         skipped_segments = []
90
91         # Create a temp directory within output_dir
92         with tempfile.TemporaryDirectory(dir=self.output_dir) as tmp_dir:
93             logger.info(f"Trimming {len(segments)} segments
94 (begin_padding={self.begin_padding}s, end_padding={self.end_padding}s)...")
95             for idx, (raw_start, raw_end) in enumerate(segments):
96                 # Normalize timestamps to float seconds ? handles float, int,
97                 # plain numeric strings, and colon-separated formats like "MM:SS"
98                 start = self._parse_time(raw_start)
99                 end = self._parse_time(raw_end)
100                 segment_label = f"Segment #{idx+1}/{len(segments)} [{start:.2f}s -
101 {end:.2f}s]"
102
103                 # Validate the segment times
104                 if start < 0:
105                     logger.warning(f"{segment_label}: start_time is negative
106 ({start:.2f}s). Clamping to 0.")
107                     start = 0.0

```



```

103         if start >= end:
104             logger.error(f"{segment_label}: start_time ({start:.2f}s) >=
end_time ({end:.2f}s). Skipping invalid segment.")
105             skipped_segments.append((idx, start, end, "start >= end"))
106             continue
107
108         # Check if segment extends beyond video duration
109         if video_duration > 0:
110             if start >= video_duration:
111                 logger.error(f"{segment_label}: start_time ({start:.2f}s) is
beyond the video duration ({video_duration:.2f}s). "
112                             f"This segment cannot be extracted ? the video ran
out. Skipping.")
113                 skipped_segments.append((idx, start, end, "start beyond video
end"))
114                 continue
115
116             if end > video_duration:
117                 original_end = end
118                 end = video_duration
119                 logger.warning(f"{segment_label}: end_time ({original_end:.2f}s)
exceeds video duration ({video_duration:.2f}s). "
120                             f"Clamping end_time to {video_duration:.2f}s. The
segment may be truncated.")
121
122                 clip_path = os.path.join(tmp_dir, f"clip_{idx}.mp4")
123
124                 # Precise sub-second trim using fast re-encoding to preserve stream
headers
125                 padded_start = max(0.0, start - self.begin_padding)
126                 padded_end = end + self.end_padding
127
128                 # Clamp padded_end to video duration if known
129                 if video_duration > 0 and padded_end > video_duration:
130                     padded_end = video_duration
131                     logger.info(f"{segment_label}: Padded end ({end +
self.end_padding:.2f}s) exceeds video duration. "
132                             f"Clamping to {video_duration:.2f}s (end padding
partially applied).")
133
134                 trim_duration = padded_end - padded_start
135                 logger.info(f"Trimming {segment_label} -> [{padded_start:.3f}s -
{padded_end:.3f}s] (duration: {trim_duration:.2f}s)")
136
137                 cmd = [
138                     "ffmpeg", "-y",
139                     "-ss", str(round(padded_start, 3)),
140                     "-to", str(round(padded_end, 3)),
141                     "-i", raw_video_path,
142                     "-c:v", "libx264",
143                     "-preset", "superfast",
144                     "-crf", "22",
145                     "-c:a", "aac",
146                     "-b:a", "128k",
147                     clip_path
148                 ]
149
150                 try:
151                     result = subprocess.run(cmd, capture_output=True, check=True)
152                     # Verify the output clip was actually created and has content
153                     if os.path.exists(clip_path) and os.path.getsize(clip_path) > 0:
154                         temp_clips.append(clip_path)
155                     else:
156                         logger.error(f"{segment_label}: FFmpeg exited successfully but
output clip is missing or empty. Skipping.")

```

```

157         skipped_segments.append((idx, start, end, "empty output clip"))
158     except subprocess.CalledProcessError as e:
159         stderr_msg = e.stderr.decode(errors='replace').strip()
160         logger.error(f"{segment_label}: FFmpeg trim failed.")
161         print(f"  Command: {' '.join(cmd)}")
162         print(f"  FFmpeg stderr: {stderr_msg[-500:]}") # Last 500 chars to
avoid flooding
163         skipped_segments.append((idx, start, end, f"ffmpeg error:
{stderr_msg[-200:]}"))
164         continue
165
166     # Report summary of skipped segments
167     if skipped_segments:
168         logger.info(f"\n[SUMMARY] {len(skipped_segments)} out of {len(segments)}
segments were SKIPPED:")
169         for seg_idx, seg_start, seg_end, reason in skipped_segments:
170             logger.info(f"  - Segment #{seg_idx+1} [{seg_start:.2f}s -
{seg_end:.2f}s]: {reason}")
171             logger.info("")
172
173     if not temp_clips:
174         raise RuntimeError(
175             f"[COMPILER ERROR] Failed to generate ANY valid highlight clips. "
176             f"All {len(segments)} segments were skipped. "
177             f"Check the errors above for details. "
178             f"Common causes: video file is corrupted, segments are out of range,
or ffmpeg is not installed."
179         )
180
181     logger.info(f"Successfully trimmed {len(temp_clips)}/{len(segments)} clips.
Preparing to stitch...")
182
183     # Create a list file for FFmpeg concat demuxer
184     list_file_path = os.path.join(tmp_dir, "clips.txt")
185     with open(list_file_path, "w") as f:
186         for clip in temp_clips:
187             # Escape single quotes in path
188             safe_path = clip.replace("'", "'\\'")
189             f.write(f"file '{safe_path}'\n")
190
191     logger.info("Stitching clips together...")
192     # Stitch them together without re-encoding (since they were encoded with
identical settings)
193     concat_cmd = [
194         "ffmpeg", "-y",
195         "-f", "concat",
196         "-safe", "0",
197         "-i", list_file_path,
198         "-c", "copy",
199         output_path
200     ]
201
202     try:
203         result = subprocess.run(concat_cmd, capture_output=True, check=True)
204
205         # Final verification
206         if not os.path.exists(output_path) or os.path.getsize(output_path) == 0:
207             raise RuntimeError(
208                 f"[COMPILER ERROR] FFmpeg concatenation appeared to succeed, "
209                 f"but output file is missing or empty: {output_path}"
210             )
211
212         output_size_mb = os.path.getsize(output_path) / (1024 * 1024)
213         logger.info(f"Dynamic highlight reel saved: {output_path}
({output_size_mb:.1f} MB)")

```

```

214         return output_path
215     except subprocess.CalledProcessError as e:
216         stderr_msg = e.stderr.decode(errors='replace').strip()
217         raise RuntimeError(
218             f"[COMPILER ERROR] FFmpeg concatenation failed.\n"
219             f"    Command: {' '.join(concat_cmd)}\n"
220             f"    FFmpeg stderr: {stderr_msg[-500:]}")
221     )
222

```

11. user (2026-06-10T06:00:47.776Z)

```

1     """Abstract base for detector runners (TrackNet, WASB, future native, etc.).
2
3     This completes the Detector Runner Abstraction (low-regret item 6 continuation / item 1
4     of the post-observability re-prioritization) to de-risk the WSL/conda dependency for
5     the high-accuracy shuttle trajectory substrate used by tracknet_hybrid / wasb_hybrid.
6
7     The ABC lets us:
8     - Keep the current WSL adapters working unchanged for Windows users.
9     - Swap in (or A/B test) a Linux-native implementation later with zero changes to
10        the hybrid segmenters or the model-agnostic windowing brain.
11     - Provide a single place to evolve the healthcheck / error contract and thread
12        detector readiness into per-video reports.
13
14     Contract (deliberately matches what the concrete runners + hybrids + tests already did):
15     - run_predict(video_win_path: str, output_win_dir: str) -> Optional[str]
16       Run detector, return the Windows-side path to the produced trajectory CSV
17       (e.g. ".../clip_predict.csv" or ".../clip_wasb.csv"), or None on any failure.
18       The caller (hybrid) is responsible for output dir; impls may stage the video
19       into WSL fs for perf.
20
21     - healthcheck() -> tuple[bool, str]
22       Return (ok, message). Never raises. "ok" decides whether to proceed to run_predict.
23       Message is logged (and can be surfaced to reports or UX in future).
24       Intended to be cheap (env existence, import, GPU visibility, weights present).
25
26     Existing behavior is 100% preserved. The only additions are the inheritance and
27     the explicit common type for future implementers.
28
29     How to add a new runner (for docs / future contributors):
30     1. Subclass DetectorRunner in a new or existing module under pipeline/detectors/.
31     2. Implement run_predict and healthcheck (return (bool, str) tuple).
32     3. If it produces the same CSV schema (Frame,Visibility,X,Y), reuse or re-export
33        TrackNetRunner.parse_trajectory_csv and .trajectory_to_action_windows (they are
34        deliberately static and model-agnostic). If the new detector needs different
35        windowing, it can implement its own or we generalize later.
36     4. Lazy-import heavy native deps inside the methods (so importing the segmenter
37        registry never pulls torch/tf on a machine that doesn't have them).
38     5. Update the two hybrid segmenters (or introduce a detector= choice) to accept
39        DetectorRunner instances (they already do the right thing via duck-typing + the
40        type hints we add here).
41     6. Add unit tests that exercise the contract (isinstance, healthcheck shape, run
42        returns str|None) using mocks for the external bits ? see test_tracknet_runner.py.
43     7. No change to eval/calibration code paths (they consume the CSV + the static
44        windowing helpers directly).
45
46     Optional Linux-native path (the main de-risk goal):
47     A LinuxNativeRunner (or ONNXRunner) would:
48     - Skip all wsl / _stage / conda activate.
49     - Load weights locally (torch or onnx) or call a native binary.
50     - Write the exact same CSV format so trajectory_to_action_windows works verbatim.
51     - healthcheck can just check "weights file exists and torch importable + CUDA?".
52     This removes the WSL requirement for Linux users / future SaaS workers while
53     keeping the exact same candidate quality characteristics (or better, once native

```

```

54     weights are available).
55
56     Error surfacing into reports:
57     The healthcheck message is already logged by the hybrids on failure. The reporting
58     harvest sees the outcome as "0 candidates" + detector name. When we wire richer
59     segmenter Results (or pass detector_health into emit_indexer_report context), the
60     (ok, msg) can be recorded explicitly under segmentation.details or a new detector
61     stage. The ABC makes that future change local to one place.
62
63     See also:
64     - backend/pipeline/segmenters/{tracknet_hybrid,wasb_hybrid}.py (the two callers)
65     - tests/test_tracknet_runner.py (contract + pure logic tests; hybrids mock the runner)
66     - docs/TRACKNET_WSL_SETUP.md and CODE_MAP.md (2.2 Detectors section)
67     """
68
69     from abc import ABC, abstractmethod
70     from typing import Optional, Tuple
71
72
73     class DetectorRunner(ABC):
74         """Common interface for shuttle/ball trajectory detectors (WSL or otherwise)."""
75
76         @abstractmethod
77         def run_predict(self, video_win_path: str, output_win_dir: str) -> Optional[str]:
78             """Run inference and return path to trajectory CSV (or None on failure)."""
79             pass
80
81         @abstractmethod
82         def healthcheck(self) -> Tuple[bool, str]:
83             """Quick check if the runner (env, weights, etc.) is usable.
84
85             Returns (ok, message). Never raises. Cheap to call before a long run.
86             """
87             pass
88

```

12. user (2026-06-10T06:00:48.382Z)

```

1     """WASB shuttle-trajectory runner (WSL2 subprocess adapter).
2
3     Windows-side counterpart to backend/detectors/wasb_infer.py. It:
4     1. stages the video into the WSL filesystem (fast local reads, not /mnt),
5     2. copies wasb_infer.py into the WASB repo's src/ dir,
6     3. runs it in the WSL `wasb` conda env -> a Frame,Visibility,X,Y trajectory CSV,
7     4. reuses the (model-agnostic) trajectory->action-window logic from tracknet_runner.
8
9     So the only WASB-specific bits live here; the windowing + CSV parsing are shared
10    with the TrackNet path. Verified inference core: wasb_infer.py (see its docstring).
11    """
12    import os
13    import shlex
14    import logging
15    import subprocess
16    from dataclasses import dataclass
17    from typing import Dict, Any, Optional, Tuple
18
19    # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical.
20    # parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on
21    TrackNetRunner.
22    from backend.pipeline.detectors.tracknet_runner import to_wsl_mnt_path, TrackNetRunner
23    from backend.pipeline.detectors.base import DetectorRunner
24
25    logger = logging.getLogger(__name__)
26
27    # Windows path to the inference wrapper we stage into WSL.

```

```

27     _INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
28
29
30     @dataclass
31     class WasbConfig:
32         """Resolved settings for a WASB WSL run (built from config.json ->
indexing.wasb)."""
33         distro: str = "Ubuntu"
34         repo_dir: str = "~/models/WASB-SBDT"
35         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36         conda_env: str = "wasb"
37         weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
38         sport: str = "badminton"
39         wsl_stage_dir: str = "~/clips"
40         timeout_sec: int = 0 # 0 = no timeout
41
42         @classmethod
43         def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":
44             w = (idx_cfg or {}).get("wasb", {}) or {}
45             return cls(
46                 distro=w.get("wsl_distro", "Ubuntu"),
47                 repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
48                 conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
49                 conda_env=w.get("conda_env", "wasb"),
50                 weights_path=w.get("weights_path",
51                                     "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
52                 sport=w.get("sport", "badminton"),
53                 wsl_stage_dir=w.get("wsl_stage_dir", "~/clips"),
54                 timeout_sec=int(w.get("timeout_sec", 0)),
55             )
56
57
58     class WasbRunner(DetectorRunner): # DetectorRunner ABC (finish item 6 / new item 1)
59         def __init__(self, cfg: WasbConfig):
60             self.cfg = cfg
61
62         def _wsl_bash(self, inner_cmd: str) -> subprocess.CompletedProcess:
63             full = f"source {self.cfg.conda_sh} && {inner_cmd}"
64             argv = ["wsl", "-d", self.cfg.distro, "bash", "-c", full]
65             logger.debug("WSL exec: %s", full)
66             return subprocess.run(argv, capture_output=True, text=True,
67                                 timeout=(self.cfg.timeout_sec or None))
68
69         def healthcheck(self) -> Tuple[bool, str]:
70             """Verify the WSL `wasb` env imports torch and sees a GPU. Returns (ok, msg)."""
71             cmd = (f"conda activate {self.cfg.conda_env} && "
72                   f"python -c \"import torch; print('torch', torch.__version__, "
73                   f"'cuda', torch.cuda.is_available())\"")
74             try:
75                 res = self._wsl_bash(cmd)
76             except FileNotFoundError:
77                 return False, "`wsl` not found ? Windows + WSL2 required."
78             except subprocess.TimeoutExpired:
79                 return False, "WSL healthcheck timed out."
80             out = (res.stdout or "").strip()
81             if res.returncode != 0:
82                 return False, f"WSL `wasb` env check failed: {(res.stderr or out).strip()}"
83             return True, out
84
85         def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
86             src_mnt = to_wsl_mnt_path(video_win_path)
87             dest = f"{self.cfg.wsl_stage_dir}/{base}"
88             cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}

```

```

{shlex.quote(dest)} && echo OK"
89         try:
90             res = self._wsl_bash(cmd)
91         except subprocess.TimeoutExpired:
92             logger.error("Staging video into WSL timed out.")
93             return None
94         if res.returncode != 0 or "OK" not in (res.stdout or ""):
95             logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
96             return None
97         return dest
98
99     def _stage_infer_script(self) -> bool:
100         """Copy wasb_infer.py into the WASB repo src/ so it can import WASB modules."""
101         src_mnt = to_wsl_mnt_path(_INFER_WIN)
102         cmd = f"cp {shlex.quote(src_mnt)} {self.cfg.repo_dir}/src/wasb_infer.py && echo
OK"
103         res = self._wsl_bash(cmd)
104         return res.returncode == 0 and "OK" in (res.stdout or "")
105
106     def run_predict(self, video_win_path: str, output_win_dir: str) -> Optional[str]:
107         """Run WASB on a video. Returns the Windows path to the trajectory CSV, or
None."""
108         os.makedirs(output_win_dir, exist_ok=True)
109         # Normalize for cross-platform basename (tests use Windows paths on Linux).
110         video_win_path = str(video_win_path).replace("\\", "/")
111         base = os.path.basename(video_win_path)
112         stem = os.path.splitext(base)[0]
113         wsl_out_dir = to_wsl_mnt_path(output_win_dir)
114         expected_csv_win = os.path.join(output_win_dir, f"{stem}_wasb.csv")
115
116         if not self._stage_infer_script():
117             logger.error("Could not stage wasb_infer.py into WSL.")
118             return None
119         staged_video = self._stage_video(video_win_path, base)
120         if staged_video is None:
121             return None
122         frames_dir = f"{self.cfg.wsl_stage_dir}/{stem}_frames"
123
124         cmd = (
125             f"conda activate {self.cfg.conda_env} && cd {self.cfg.repo_dir}/src && "
126             f"python wasb_infer.py "
127             f"--video {shlex.quote(staged_video)} "
128             f"--frames_out_dir {shlex.quote(frames_dir)} "
129             f"--weights {shlex.quote(self.cfg.weights_path)} "
130             f"--sport {shlex.quote(self.cfg.sport)} "
131             f"--out {shlex.quote(wsl_out_dir + '/' + stem + '_wasb.csv')}"
132         )
133         logger.info("Running WASB inference on %s ...", base)
134         try:
135             res = self._wsl_bash(cmd)
136         except subprocess.TimeoutExpired:
137             logger.error("WASB inference timed out after %ss.", self.cfg.timeout_sec)
138             return None
139         if res.returncode != 0:
140             logger.error("WASB inference failed:\n%s", (res.stderr or
res.stdout)[-2000:])
141             return None
142         if not os.path.exists(expected_csv_win):
143             logger.error("WASB finished but expected CSV not found at %s",
expected_csv_win)
144             return None
145         logger.info("WASB trajectory CSV: %s", expected_csv_win)
146         return expected_csv_win
147

```

```

148         # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
149         parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
150         trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
151

```

13. user (2026-06-10T06:01:04.007Z)

```

1     """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).
2
3     This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
4     inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:
5
6         1. Translate the Windows video path to its WSL-visible form (and optionally
7           stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
8         2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
9         3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
10        4. Convert that trajectory into high-motion "action windows" ? the same
11           candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
12           pipeline (AI handoff, DB population) is unchanged.
13
14     Nothing here imports TensorFlow; all model code stays in WSL.
15     """
16
17     import os
18     import csv
19     import shlex
20     import logging
21     import subprocess
22     from dataclasses import dataclass
23     from typing import List, Dict, Any, Optional, Tuple
24
25     from backend.pipeline.detectors.base import DetectorRunner
26
27     logger = logging.getLogger(__name__)
28
29
30     @dataclass
31     class TrackNetConfig:
32         """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""
33         distro: str = "Ubuntu"
34         repo_dir: str = "~/models/TrackNetV4"          # WSL path to the cloned repo
35         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36         conda_env: str = "TrackNetV4"
37         weights_path: str = ""                        # WSL path to the .h5/.keras weights
38         (REQUIRED)
39         queue_length: int = 5
40         stage_in_wsl: bool = True                    # copy video into WSL fs first
41         (perf)
42         wsl_stage_dir: str = "~/clips"                # where staged videos/outputs live
43         in WSL
44         timeout_sec: int = 0                          # 0 = no timeout
45
46
47     @classmethod
48     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "TrackNetConfig":
49         tn = (idx_cfg or {}).get("tracknet", {}) or {}
50         return cls(
51             distro=tn.get("wsl_distro", "Ubuntu"),
52             repo_dir=tn.get("repo_dir", "~/models/TrackNetV4"),
53             conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
54             conda_env=tn.get("conda_env", "TrackNetV4"),
55             weights_path=tn.get("weights_path", ""),
56             queue_length=int(tn.get("queue_length", 5)),

```

```

53         stage_in_wsl=bool(tn.get("stage_in_wsl", True)),
54         wsl_stage_dir=tn.get("wsl_stage_dir", "~/clips"),
55         timeout_sec=int(tn.get("timeout_sec", 0)),
56     )
57
58
59 def to_wsl_mnt_path(win_path: str) -> str:
60     """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).
61
62     Already-POSIX paths (starting with / or ~) are returned unchanged so the
63     same helper is safe to call on either kind of path.
64     """
65     p = str(win_path)
66     if p.startswith("/") or p.startswith("~"):
67         return p
68     p = p.replace("\\", "/")
69     if len(p) >= 2 and p[1] == ":":
70         drive = p[0].lower()
71         return f"/mnt/{drive}{p[2:]}"
72     return p
73
74
75 @dataclass
76 class TrajectoryPoint:
77     frame: int
78     visible: bool
79     x: float
80     y: float
81
82
83 class TrackNetRunner(DetectorRunner): # DetectorRunner ABC (finish item 6 / new item 1)
84     """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.
85
86     Implements the common DetectorRunner contract so a future Linux-native or
87     alternative trajectory runner can be substituted without touching the hybrids.
88     """
89
90     def __init__(self, cfg: TrackNetConfig):
91         self.cfg = cfg
92
93     # ----- WSL exec
94
95     def _wsl_bash(self, inner_cmd: str) -> subprocess.CompletedProcess:
96         """Run a bash command string inside the configured WSL distro."""
97         full = f"source {self.cfg.conda_sh} && {inner_cmd}"
98         argv = ["wsl", "-d", self.cfg.distro, "bash", "-c", full]
99         logger.debug("WSL exec: %s", full)
100         return subprocess.run(
101             argv, capture_output=True, text=True,
102             timeout=(self.cfg.timeout_sec or None),
103         )
104
105     def healthcheck(self) -> Tuple[bool, str]:
106         """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.
107
108         Returns (ok, message). Cheap to call before a long run.
109         """
110         cmd = (
111             f"conda activate {self.cfg.conda_env} && "
112             f"python -c \"import tensorflow as tf; "
113             f"print('TF', tf.__version__, 'GPUs', "
114             f"len(tf.config.list_physical_devices('GPU'))))\" "
115         )
116         try:
117             res = self._wsl_bash(cmd)

```



```

117         except FileNotFoundError:
118             return False, "`wsl` not found ? is this running on Windows with WSL2
installed?"
119         except subprocess.TimeoutExpired:
120             return False, "WSL healthcheck timed out."
121         out = (res.stdout or "").strip()
122         if res.returncode != 0:
123             return False, f"WSL env check failed: {(res.stderr or out).strip()}"
124         return True, out
125
126 # ----- inference
127
128 def run_predict(self, video_win_path: str, output_win_dir: str) -> Optional[str]:
129     """Run predict.py on a video. Returns the Windows path to the produced CSV, or
None.
130
131     ``output_win_dir`` is a Windows directory (under the repo's output/) that
132     WSL writes into via its /mnt view, so the Windows process can read the CSV
133     back with no copy.
134     """
135     if not self.cfg.weights_path:
136         logger.error(
137             "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
138             "in config.json to the WSL path of your trained TrackNetV4 weights."
139         )
140         return None
141
142     os.makedirs(output_win_dir, exist_ok=True)
143     # Normalize separators so basename/splitext work when tests (or collector)
144     # pass Windows-style paths on a Linux host.
145     video_win_path = str(video_win_path).replace("\\", "/")
146     base = os.path.basename(video_win_path)
147     stem, ext = os.path.splitext(base)
148
149     # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
150     if ext.lower() not in (".mp4", ".avi"):
151         logger.error("TrackNet predict.py only supports .mp4/.avi (got %s)", ext)
152         return None
153
154     wsl_out = to_wsl_mnt_path(output_win_dir)
155     expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")
156
157     # Resolve the video path WSL will read.
158     if self.cfg.stage_in_wsl:
159         staged = self._stage_video(video_win_path, base)
160         if staged is None:
161             return None
162         wsl_video = staged
163     else:
164         wsl_video = to_wsl_mnt_path(video_win_path)
165
166     cmd = (
167         f"conda activate {self.cfg.conda_env} && "
168         f"cd {self.cfg.repo_dir}/src && "
169         f"python predict.py "
170         f"--video_path {shlex.quote(wsl_video)} "
171         f"--model_weights {shlex.quote(self.cfg.weights_path)} "
172         f"--output_dir {shlex.quote(wsl_out)} "
173         f"--queue_length {self.cfg.queue_length}"
174     )
175     logger.info("Running TrackNet inference on %s ...", base)
176     try:
177         res = self._wsl_bash(cmd)
178     except subprocess.TimeoutExpired:
179         logger.error("TrackNet inference timed out after %ss.",

```

```

self.cfg.timeout_sec)
180         return None
181
182         if res.returncode != 0:
183             logger.error("TrackNet predict.py failed:\n%s", (res.stderr or
res.stdout)[-2000:])
184             return None
185
186         if not os.path.exists(expected_csv_win):
187             logger.error("TrackNet finished but expected CSV not found at %s",
expected_csv_win)
188             return None
189             logger.info("TrackNet trajectory CSV: %s", expected_csv_win)
190             return expected_csv_win
191
192     def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
193         """Copy the input video into the WSL filesystem (avoids slow /mnt/c reads).
194
195         Returns the WSL path to the staged copy, or None on failure.
196         """
197         src_mnt = to_wsl_mnt_path(video_win_path)
198         dest = f"{self.cfg.wsl_stage_dir}/{base}"
199         cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{shlex.quote(dest)} && echo OK"
200         try:
201             res = self._wsl_bash(cmd)
202         except subprocess.TimeoutExpired:
203             logger.error("Staging video into WSL timed out.")
204             return None
205         if res.returncode != 0 or "OK" not in (res.stdout or ""):
206             logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
207             return None
208             return dest
209
210     # ----- CSV -> windows
211
212     @staticmethod
213     def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
214         """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
215         points: List[TrajectoryPoint] = []
216         with open(csv_win_path, newline="") as f:
217             reader = csv.DictReader(f)
218             for row in reader:
219                 try:
220                     points.append(TrajectoryPoint(
221                         frame=int(row["Frame"]),
222                         visible=int(row["Visibility"]) == 1,
223                         x=float(row["X"]),
224                         y=float(row["Y"]),
225                     ))
226                 except (KeyError, ValueError):
227                     continue
228             points.sort(key=lambda p: p.frame)
229             return points
230
231     @staticmethod
232     def trajectory_to_action_windows(
233         points: List[TrajectoryPoint],
234         fps: float,
235         frame_width: float,
236         chunk_start: float = 0.0,
237         velocity_thresh: float = 0.02,
238         merge_gap: float = 2.0,
239         min_window_duration: float = 2.0,

```

```

240     chunk_index: Optional[int] = None,
241 ) -> List[Dict[str, Any]]:
242     """Convert a shuttle trajectory into candidate rally windows.
243
244     A frame is "active" when the shuttle is visible AND its inter-frame
245     displacement (normalised by frame width, per second) exceeds
246     ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
247     ``merge_gap`` seconds of each other are grouped into one window; short
248     windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
249     shape feeding the AI-handoff phase is identical.
250     """
251     if fps <= 0:
252         fps = 30.0
253     if frame_width <= 0:
254         frame_width = 1280.0
255
256     # Compute per-frame normalised velocity from the last *visible* point.
257     active = [] # (frame_idx, velocity)
258     prev: Optional[TrajectoryPoint] = None
259     for p in points:
260         if not p.visible:
261             prev = None # break the trajectory; absent shuttle = not in flight
262             continue
263         if prev is not None:
264             dframes = p.frame - prev.frame
265             if dframes > 0:
266                 dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
267                 velocity = (dist / frame_width) * (fps / dframes)
268                 if velocity > velocity_thresh:
269                     active.append((p.frame, velocity))
270             prev = p
271
272     if not active:
273         return []
274
275     max_gap_frames = int(merge_gap * fps)
276
277     def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
278         vels = [v for _, v in group]
279         return {
280             "start": group[0][0] / fps + chunk_start,
281             "end": group[-1][0] / fps + chunk_start,
282             "active_frame_count": len(group),
283             "peak_velocity": max(vels),
284             "mean_velocity": sum(vels) / len(vels),
285             "track_id_count": 1, # single shuttle; kept for window-shape parity
286             "chunk_index": chunk_index,
287         }
288
289     windows = []
290     group = [active[0]]
291     for af in active[1:]:
292         if af[0] - group[-1][0] <= max_gap_frames:
293             group.append(af)
294         else:
295             windows.append(_finalize(group))
296             group = [af]
297     windows.append(_finalize(group))
298
299     return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
300

```

14. user (2026-06-10T06:01:16.358Z)

```

1     from abc import ABC, abstractmethod

```

```

2     from typing import List, Dict, Any, Type, Optional
3     from backend.infrastructure.database import Database
4     from backend.results import Result # new standardized contract (Result[Value] ==
Success[Value] | Failure)
5
6     class BasePlaySegmenter(ABC):
7         def __init__(self, db: Database, config: Dict[str, Any]):
8             self.db = db
9             self.config = config
10
11         @property
12         @abstractmethod
13         def name(self) -> str:
14             """Unique identifier for this segmenter."""
15             pass
16
17         @property
18         @abstractmethod
19         def description(self) -> str:
20             """Human-readable explanation of this segmenter."""
21             pass
22
23         @abstractmethod
24         def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
25             """
26             Processes a video to detect play segments, populates SQLite database,
27             and returns a Result.
28
29             Use isinstance(result, Failure) (or result.is_success()) to distinguish
30             "no segments found" from "something went wrong" (missing key, healthcheck
31             failure, etc.). This is the start of the standardized error/result contract.
32             """
33             pass
34
35     class SegmenterRegistry:
36         def __init__(self):
37             self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}
38
39         def register(self, segmenter_cls: Type[BasePlaySegmenter]):
40             """Registers a new segmenter class under its name property."""
41             try:
42                 temp_inst = segmenter_cls(None, {})
43                 name = temp_inst.name
44             except Exception as e:
45                 raise ValueError(f"Failed to resolve name property from segmenter class
{segmenter_cls.__name__}: {e}")
46             self._segmenters[name] = segmenter_cls
47
48         def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
49             return self._segmenters.get(name)
50
51         def list_available(self) -> Dict[str, str]:
52             """Returns a dict of registered segmenter names and their descriptions."""
53             res = {}
54             for name, cls in self._segmenters.items():
55                 try:
56                     temp_inst = cls(None, {})
57                     desc = temp_inst.description
58                 except Exception:
59                     desc = "No description available."
60                 res[name] = desc
61             return res
62
63     segmenter_registry = SegmenterRegistry()

```

15. user (2026-06-10T06:01:16.774Z)

```

1      import os
2      import time
3      import json
4      import logging
5      import subprocess
6      import concurrent.futures
7      from typing import List, Dict, Any, Tuple
8      from backend.infrastructure.database import Database
9      from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
10     from backend.utils.video import get_video_duration, extract_video_chunk
11     from backend.sports import sport_registry
12     from backend.providers import provider_registry
13
14     logger = logging.getLogger(__name__)
15
16     class GeminiPlaySegmenter(BasePlaySegmenter):
17         @property
18         def name(self) -> str:
19             return "gemini"
20
21         @property
22         def description(self) -> str:
23             return "High-precision AI-powered semantic play segment detection using Google
Gemini Multimodal API."
24
25         def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> List[Dict[str, Any]]:
26             """
27             Processes a video to detect play segments using Google Gemini Multimodal Video
API,
28             populates SQLite, and indexes detailed events.
29             """
30             # Get sport profile config
31             sport_name = self.config.get("sport", "badminton")
32             try:
33                 sport_profile = sport_registry.get(sport_name)
34             except KeyError as e:
35                 logger.error(str(e))
36                 return []
37
38             # Get AI provider and model config
39             idx_cfg = self.config.get("indexing", {})
40             provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
41             model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
42
43             # Get appropriate API key based on the provider
44             api_key_env_var = f"{provider_name.upper()}_API_KEY"
45             api_key = os.environ.get(api_key_env_var)
46             if not api_key:
47                 logger.error(
48                     f"{api_key_env_var} environment variable is not set! "
49                     f"To run the {provider_name} segmenter, set your API key. "
50                     f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
51                 )
52             return []
53
54         try:
55             provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)

```

```

56         except KeyError as e:
57             logger.error(str(e))
58             return []
59
60         # A: Check for debug_duration to trim the video first
61         debug_duration = self.config.get("indexing", {}).get("debug_duration")
62         video_to_upload = video_path
63         is_trimmed = False
64         temp_trimmed_path = None
65         is_compressed = False
66         temp_compressed_path = None
67
68         if debug_duration:
69             logger.info(f"DEBUG FEATURE: Trimming video to first {debug_duration}
seconds for fast processing...")
70             temp_trimmed_path = os.path.join("output", f"temp_trimmed_{video_id}.mp4")
71             os.makedirs("output", exist_ok=True)
72             if os.path.exists(temp_trimmed_path):
73                 try:
74                     os.remove(temp_trimmed_path)
75                 except Exception:
76                     pass
77
78             cmd = [
79                 "ffmpeg", "-y",
80                 "-ss", "0",
81                 "-t", str(debug_duration),
82                 "-i", video_path,
83                 "-c", "copy",
84                 temp_trimmed_path
85             ]
86             logger.debug(f"Running command: {' '.join(cmd)}")
87             try:
88                 subprocess.run(cmd, check=True, stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL)
89                 logger.info(f"Trimmed video successfully created at:
{temp_trimmed_path}")
90                 video_to_upload = temp_trimmed_path
91                 is_trimmed = True
92             except Exception as e:
93                 logger.warning(f"Failed to trim video using ffmpeg: {e}. Falling back to
full video.")
94
95
96
97         # Helper to clean up local temp files
98         def cleanup_temp_files():
99             if is_trimmed and temp_trimmed_path and os.path.exists(temp_trimmed_path):
100                 try:
101                     os.remove(temp_trimmed_path)
102                 except Exception:
103                     pass
104
105         # 1. Determine actual video duration
106         video_duration = get_video_duration(video_to_upload)
107         if video_duration > 0:
108             logger.info(f"Detected video duration: {video_duration:.1f}s")
109         else:
110             logger.warning("Could not detect video duration. Prompt will not include
duration context.")
111
112         # 2. Decide whether to use chunked processing
113         idx_cfg = self.config.get("indexing", {})
114         chunk_duration = idx_cfg.get("chunk_duration", 300) # 5 minutes default
115         chunk_overlap = idx_cfg.get("chunk_overlap", 30) # 30 seconds overlap

```

```

116         use_chunking = video_duration > 0 and video_duration > chunk_duration
117
118         ai_segments = []
119         chunks_dir = os.path.join("output", "chunks")
120
121         if use_chunking:
122             # --- CHUNKED PROCESSING ---
123             step = chunk_duration - chunk_overlap
124             chunk_starts = []
125             t = 0.0
126             while t < video_duration:
127                 chunk_starts.append(t)
128                 t += step
129             num_chunks = len(chunk_starts)
130             logger.info(
131                 f"Video is {video_duration:.1f}s ? splitting into {num_chunks}
overlapping chunks "
132                 f"({chunk_duration}s each, {chunk_overlap}s overlap).")
133             )
134
135             os.makedirs(chunks_dir, exist_ok=True)
136
137             def process_single_chunk(ci: int, chunk_start: float) -> Tuple[str, list,
dict]:
138                 chunk_end = min(chunk_start + chunk_duration, video_duration)
139                 actual_chunk_dur = chunk_end - chunk_start
140                 chunk_label = f"Chunk {ci+1}/{num_chunks}"
141                 chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
142
143                 # Extract chunk with ffmpeg
144                 logger.info(f"Extracting {chunk_label}: {chunk_start:.1f}s -
{chunk_end:.1f}s ({actual_chunk_dur:.1f}s)")
145                 success = extract_video_chunk(video_to_upload, chunk_start,
actual_chunk_dur, chunk_path)
146                 if not success:
147                     logger.error(f"Failed to extract {chunk_label}. Skipping.")
148                     return chunk_label, [], {}
149
150                 # Measure chunk size
151                 try:
152                     chunk_size_mb = os.path.getsize(chunk_path) / (1024 * 1024)
153                 except Exception:
154                     chunk_size_mb = 0.0
155
156                 # Get prompt from sport profile
157                 prompt = sport_profile.get_prompt(chunk_duration=actual_chunk_dur)
158
159                 # Process this chunk through the provider
160                 chunk_segments, metrics = provider.analyze_video(
161                     chunk_path, prompt, chunk_duration=actual_chunk_dur,
label=chunk_label
162                 )
163                 metrics["file_size_mb"] = chunk_size_mb
164                 metrics["segments_found"] = len(chunk_segments)
165
166                 # Offset timestamps back to full-video coordinates
167                 for segment in chunk_segments:
168                     if "start_time" in segment:
169                         try:
170                             segment["start_time"] = float(segment["start_time"]) +
chunk_start
171                         except (ValueError, TypeError):
172                             pass
173                     if "end_time" in segment:
174                         try:

```

```

175             segment["end_time"] = float(segment["end_time"]) +
chunk_start
176             except (ValueError, TypeError):
177                 pass
178
179         logger.info(f"{chunk_label} returned {len(chunk_segments)} play
segments.")
180
181         # Clean up chunk file
182         try:
183             os.remove(chunk_path)
184         except Exception:
185             pass
186
187         return chunk_label, chunk_segments, metrics
188
189     # Run chunks in parallel
190     all_metrics = {}
191     logger.info("Starting parallel processing with up to 5 concurrent
workers...")
192     t_chunking_start = time.time()
193     with concurrent.futures.ThreadPoolExecutor(max_workers=5) as executor:
194         futures = [executor.submit(process_single_chunk, ci, start) for ci,
start in enumerate(chunk_starts)]
195         for future in concurrent.futures.as_completed(futures):
196             label, segments, metrics = future.result()
197             ai_segments.extend(segments)
198             if metrics:
199                 all_metrics[label] = metrics
200
201     # Print Summary Log
202     summary_lines = [
203         "",
204         "=" * 60,
205         "                GEMINI CHUNKING SUMMARY",
206         "=" * 60,
207         f"Total time: {time.time() - t_chunking_start:.1f}s",
208         f"Total chunks: {num_chunks}",
209         f"Total play segments found: {len(ai_segments)}",
210         "-" * 60,
211         f"{'Chunk':<12} | {'Size':<8} | {'Upload':<8} | {'Process':<8} |
{'Gen.':<8} | {'Segments'}",
212         "-" * 60,
213     ]
214     for i in range(num_chunks):
215         lbl = f"Chunk {i+1}/{num_chunks}"
216         m = all_metrics.get(lbl, {})
217         sz = f"{m.get('file_size_mb', 0):.1f}MB" if m else "N/A"
218         upl = f"{m.get('upload_time', 0):.1f}s" if m else "N/A"
219         prc = f"{m.get('process_time', 0):.1f}s" if m else "N/A"
220         gen = f"{m.get('gen_time', 0):.1f}s" if m else "N/A"
221         seg = m.get('segments_found', 0) if m else 0
222         summary_lines.append(f" {lbl:<12} | {sz:<8} | {upl:<8} | {prc:<8} |
{gen:<8} | {seg}")
223     summary_lines.append("=" * 60)
224     logger.info("\n".join(summary_lines))
225
226     # Clean up local temp files
227     cleanup_temp_files()
228     try:
229         os.rmdir(chunks_dir)
230     except Exception:
231         pass
232
233     logger.info(f"All chunks processed. {len(ai_segments)} total play segments

```



```

before deduplication.")
234
235         else:
236             # --- SINGLE-FILE PROCESSING (short videos) ---
237             try:
238                 file_sz = os.path.getsize(video_to_upload) / (1024 * 1024)
239             except Exception:
240                 file_sz = 0.0
241
242             # Get prompt from sport profile
243             prompt = sport_profile.get_prompt(chunk_duration=video_duration)
244             ai_segments, metrics = provider.analyze_video(
245                 video_to_upload, prompt, chunk_duration=video_duration,
label="SingleFile"
246             )
247             summary_lines = [
248                 "",
249                 "=" * 50,
250                 "          GEMINI SINGLE FILE SUMMARY",
251                 "=" * 50,
252                 f" File Size: {file_sz:.1f} MB",
253                 f" Upload time: {metrics.get('upload_time', 0):.1f}s",
254                 f" API Processing time: {metrics.get('process_time', 0):.1f}s",
255                 f" Prompt Generation time: {metrics.get('gen_time', 0):.1f}s",
256                 f" Play segments found: {len(ai_segments)}",
257                 "=" * 50,
258             ]
259             logger.info("\n".join(summary_lines))
260             cleanup_temp_files()
261
262         if not ai_segments:
263             return []
264
265         # Populate SQLite DB with parsed play segments
266         segments_data = []
267         logger.info(f"Processing {len(ai_segments)} play segments through
validation...")
268
269         def parse_time(val) -> float:
270             """Converts a timestamp value to total float seconds.
271             Handles float/int pass-through, plain numeric strings, and
272             colon-separated formats (MM:SS, HH:MM:SS). Logs warnings
273             when unexpected formats are encountered."""
274             if isinstance(val, (int, float)):
275                 return float(val)
276             if isinstance(val, str):
277                 val = val.strip()
278                 if ":" in val:
279                     # Gemini returned MM:SS or HH:MM:SS despite being asked for decimal
seconds.
280
281                     # Convert it, but warn so the issue is visible in logs.
282                     parts = val.split(":")
283                     parts.reverse()
284                     total = 0.0
285                     for i, p in enumerate(parts):
286                         try:
287                             total += float(p) * (60 ** i)
288                         except ValueError:
289                             pass
290                     logger.warning(f"Timestamp '{val}' was in colon-separated format
(MM:SS/HH:MM:SS) instead of decimal seconds. Converted to {total:.2f}s.")
291                     return total
292                 try:
293                     return float(val)
294                 except ValueError:

```

```

294         logger.warning(f"Could not parse timestamp string '{val}' as a
number. Defaulting to 0.0.")
295         return 0.0
296         logger.warning(f"Unexpected timestamp type {type(val).__name__} for value
'{val}'. Defaulting to 0.0.")
297         return 0.0
298
299     # 7. Parse timestamps and build candidate segment list
300     idx_cfg = self.config.get("indexing", {})
301     min_segment_duration = idx_cfg.get("min_segment_duration", 3.0)
302     max_segment_duration = 90.0 # Singles play segments almost never exceed this
303
304     candidates = []
305     for idx, segment in enumerate(ai_segments):
306         start = parse_time(segment.get("start_time", 0.0))
307         end = parse_time(segment.get("end_time", 0.0))
308         if start >= end:
309             logger.info(f"Skipping segment #{idx+1}: start_time ({start:.2f}s) >=
end_time ({end:.2f}s)")
310             continue
311             candidates.append({
312                 "idx": idx,
313                 "start": start,
314                 "end": end,
315                 "raw": segment
316             })
317
318     # 8. Post-hoc validation pass
319     logger.info(f"Running validation on {len(candidates)} candidate play
segments...")
320     validated = []
321     rejected = []
322
323     for c in candidates:
324         label = f"Segment #{c['idx']+1} [{c['start']:.2f}s - {c['end']:.2f}s]"
325
326         # A: Reject segments that start beyond the video
327         if video_duration > 0 and c["start"] >= video_duration:
328             rejected.append((label, f"start_time ({c['start']:.2f}s) is beyond video
duration ({video_duration:.1f}s)"))
329             continue
330
331         # B: Clamp end_time to video duration
332         if video_duration > 0 and c["end"] > video_duration:
333             logger.info(f"{label}: end_time ({c['end']:.2f}s) exceeds video duration
({video_duration:.1f}s). Clamping.")
334             c["end"] = video_duration
335
336         dur = c["end"] - c["start"]
337
338         # C: Reject segments shorter than min_segment_duration
339         if dur < min_segment_duration:
340             rejected.append((label, f"duration ({dur:.2f}s) is below minimum
({min_segment_duration}s)"))
341             continue
342
343         # D: Flag suspiciously long segments
344         if dur > max_segment_duration:
345             logger.warning(
346                 f"{label}: duration ({dur:.1f}s) is unusually long
(>{max_segment_duration}s). "
347                 f"This may be a detection error. Keeping but flagging."
348             )
349
350         validated.append(c)

```

```

351
352     # E: Resolve overlapping segments (keep the longer one)
353     if len(validated) > 1:
354         # Sort by start time
355         validated.sort(key=lambda r: r["start"])
356         non_overlapping = [validated[0]]
357         for curr in validated[1:]:
358             prev = non_overlapping[-1]
359             # Check if current segment overlaps with the previous one
360             if curr["start"] < prev["end"]:
361                 prev_dur = prev["end"] - prev["start"]
362                 curr_dur = curr["end"] - curr["start"]
363                 kept, dropped = (prev, curr) if prev_dur >= curr_dur else (curr,
prev)
364                 kept_label = f"Segment #{kept['idx']+1} [{kept['start']:.2f}s -
{kept['end']:.2f}s]"
365                 dropped_label = f"Segment #{dropped['idx']+1}
[{dropped['start']:.2f}s - {dropped['end']:.2f}s]"
366                 rejected.append((dropped_label, f"overlaps with {kept_label} ? kept
the longer segment"))
367                 if kept == curr:
368                     non_overlapping[-1] = curr # Replace previous with current
369                 else:
370                     non_overlapping.append(curr)
371                 validated = non_overlapping
372
373     # Log validation summary
374     if rejected:
375         reject_lines = [f" ? {label}: {reason}" for label, reason in rejected]
376         logger.info(f"{len(rejected)} play segments were REJECTED:\n" +
"\n".join(reject_lines))
377         logger.info(f"{len(validated)} play segments passed validation (out of
{len(ai_segments)} from Gemini).")
378
379     stitch_cfg = self.config.get("stitching", {})
380     begin_pad = stitch_cfg.get("begin_padding", 0.5)
381     end_pad = stitch_cfg.get("end_padding", 0.5)
382
383     total_highlight_dur = 0.0
384     for c in validated:
385         total_highlight_dur += (c["end"] - c["start"]) + begin_pad + end_pad
386
387     mins = int(total_highlight_dur // 60)
388     secs = int(total_highlight_dur % 60)
389     logger.info(
390         f"Estimated final highlight video duration: {mins:02d}:{secs:02d} "
391         f"({total_highlight_dur:.1f}s total; includes {begin_pad}s begin +
{end_pad}s end padding per segment)"
392     )
393
394     # 9. Populate SQLite DB with validated play segments
395     for c in validated:
396         start = c["start"]
397         end = c["end"]
398         segment = c["raw"]
399         dur = end - start
400
401         # Extract shot_count from Gemini or compute estimate if not present
402         shot_count = segment.get("shuttle_exchange_count")
403         if shot_count is None:
404             shot_count = max(3, int(dur / 0.8)) # 1 shot every 0.8 seconds as
fallback estimate
405         else:
406             try:
407                 shot_count = int(shot_count)

```

```

408             except Exception:
409                 shot_count = max(3, int(dur / 0.8))
410
411             # Only store metrics we actually know ? no fabricated data
412             metadata = {
413                 "shot_count": shot_count,
414                 "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
415                 "detector": model_name
416             }
417
418             segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
419
420             if segment_id:
421                 segments_data.append({
422                     "id": segment_id,
423                     "start_time": start,
424                     "end_time": end,
425                     "duration": dur,
426                     "metadata": metadata
427                 })
428
429             return segments_data
430
431     # Register the segmenter
432     segmenter_registry.register(GeminiPlaySegmenter)
433

```

16. user (2026-06-10T06:01:34.979Z)

```

1     """WASB-based hybrid play segmenter.
2
3     Identical three-phase shape to TrackNetHybridSegmenter, but Phase 2 uses **WASB**
4     (nttcom/WASB-SBDT, MIT) shuttle-trajectory detection via WSL2
5     (backend/detectors/wasb_runner.py) instead of TrackNetV4. WASB ships pretrained
6     badminton weights and runs today, so this is the live shuttle-motion substrate.
7     Phase 3 (AI semantic boundaries) and Phase 4 (DB) are shared with the other hybrids.
8
9     For the per-video *calibration* goal we mainly use Phase 2 (trajectory -> candidate
10    windows); the windowing thresholds (velocity_thresh / merge_gap / min_window_duration)
11    are what get tuned against golden rally labels.
12    """
13    import os
14    import time
15    import logging
16    import concurrent.futures
17    from typing import List, Dict, Any
18
19    import cv2
20
21    from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
22    from backend.utils.video import get_video_duration, extract_video_chunk
23    from backend.sports import sport_registry
24    from backend.providers import provider_registry
25    from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
26    from backend.pipeline.detectors.base import DetectorRunner # ABC for the de-risked
detector seam
27
28    logger = logging.getLogger(__name__)
29
30
31    def _fmt_ts(seconds: float) -> str:
32        seconds = max(0, int(seconds))
33        h, rem = divmod(seconds, 3600)

```

```

34         m, s = divmod(rem, 60)
35         return f"{h:02d}:{m:02d}:{s:02d}"
36
37
38     class WasbHybridSegmenter(BasePlaySegmenter):
39         @property
40         def name(self) -> str:
41             return "wasb_hybrid"
42
43         @property
44         def description(self) -> str:
45             return ("Two-stage pipeline: WASB shuttle-trajectory detection (WSL2, pretrained
badminton "
46                     "weights) for precise candidate rallies + AI provider for semantic
boundaries.")
47
48         @staticmethod
49         def _probe_fps_width(video_path: str) -> tuple:
50             cap = cv2.VideoCapture(video_path)
51             fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
52             width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
53             cap.release()
54             return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
55
56         def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None,
57                         max_frames: int = None) -> List[Dict[str, Any]]:
58             sport_name = self.config.get("sport", "badminton")
59             try:
60                 sport_profile = sport_registry.get(sport_name)
61             except KeyError as e:
62                 logger.error(str(e))
63                 return []
64
65             idx_cfg = self.config.get("indexing", {})
66             provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
67             model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
68
69             api_key_env_var = f"{provider_name.upper()}_API_KEY"
70             api_key = os.environ.get(api_key_env_var)
71             if not api_key:
72                 logger.error(f"{api_key_env_var} not set; cannot run the {provider_name}
handoff.")
73                 return []
74             try:
75                 provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
76             except KeyError as e:
77                 logger.error(str(e))
78                 return []
79
80             video_duration = get_video_duration(video_path)
81             if video_duration <= 0:
82                 logger.error("Invalid video duration.")
83                 return []
84             fps, frame_width = self._probe_fps_width(video_path)
85
86             wasb_cfg = WasbConfig.from_indexing_cfg(idx_cfg)
87             runner: DetectorRunner = WasbRunner(wasb_cfg)
88
89             w = idx_cfg.get("wasb", {})
90             velocity_thresh = w.get("velocity_threshold", 0.02)
91             merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)

```

```

92         min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
93         handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
94         ai_max_workers = idx_cfg.get("ai_max_workers", 5)
95         log_candidates = idx_cfg.get("log_yolo_candidates", True)
96         store_candidates = idx_cfg.get("store_yolo_candidates", True)
97
98         detection_params = {
99             "detector": "wasb_hybrid",
100             "velocity_thresh": velocity_thresh,
101             "merge_gap": merge_gap,
102             "min_action_window_duration": min_window_duration,
103             "weights_path": wasb_cfg.weights_path,
104         }
105
106         ok, msg = runner.healthcheck()
107         if not ok:
108             logger.error("WASB WSL environment not ready: %s", msg)
109             return []
110         logger.info("WASB WSL env OK: %s", msg)
111
112         # --- Phase 2: shuttle trajectory -> candidate rally windows ---
113         t_start = time.time()
114         out_dir = os.path.join("output", "wasb")
115         csv_path = runner.run_predict(video_path, out_dir)
116         if not csv_path:
117             logger.error("WASB produced no trajectory; aborting.")
118             return []
119
120         points = runner.parse_trajectory_csv(csv_path)
121         logger.info("Parsed %d trajectory points (%d visible).",
122                     len(points), sum(1 for p in points if p.visible))
123
124         all_candidate_windows = runner.trajectory_to_action_windows(
125             points, fps=fps, frame_width=frame_width, chunk_start=0.0,
126             velocity_thresh=velocity_thresh, merge_gap=merge_gap,
127             min_window_duration=min_window_duration, chunk_index=0,
128         )
129         logger.info("Phase 2 (WASB) completed in %.1fs. Candidate windows: %d",
130                     time.time() - t_start, len(all_candidate_windows))
131
132         if log_candidates:
133             for cw in all_candidate_windows:
134                 logger.info(f"[WASB rally] {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
135                             f"({cw['end'] - cw['start']:.1f}s) | "
136                             f"peak_v={cw['peak_velocity']:.3f} "
137                             f"mean_v={cw['mean_velocity']:.3f}")
138             candidate_ids = [None] * len(all_candidate_windows)
139             if store_candidates:
140                 for i, cw in enumerate(all_candidate_windows):
141                     stats = {
142                         "peak_velocity": cw["peak_velocity"], "mean_velocity":
143                         cw["mean_velocity"],
144                         "active_frame_count": cw["active_frame_count"], "track_id_count":
145                         cw["track_id_count"],
146                     }
147                     candidate_ids[i] = self.db.add_candidate_segment(
148                         video_id, detector="wasb_hybrid", start_time=cw["start"],
149                         end_time=cw["end"],
150                         chunk_index=cw["chunk_index"], confidence=min(1.0,

```

```

151         if not all_candidate_windows:
152             return []
153
154         # --- Phase 3: AI provider handoff (shared pattern) ---
155         handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
156         os.makedirs(handoff_dir, exist_ok=True)
157         logger.info("Phase 3: %s validation on %d candidate windows...",
158                     provider_name, len(all_candidate_windows))
159
160         def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
161             w_start = max(0, window["start"] - handoff_padding)
162             w_end = min(video_duration, window["end"] + handoff_padding)
163             w_dur = w_end - w_start
164             w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
165             if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
166                 return []
167             prompt = sport_profile.get_prompt(chunk_duration=w_dur)
168             segments, _ = provider.analyze_video(w_path, prompt, chunk_duration=w_dur,
169                                                 label=f"Handoff {wi+1}")
170             valid = []
171             for seg in segments:
172                 try:
173                     seg["start_time"] = float(seg["start_time"]) + w_start
174                     seg["end_time"] = float(seg["end_time"]) + w_start
175                     seg["_candidate_id"] = candidate_ids[wi]
176                     valid.append(seg)
177                 except (ValueError, TypeError, KeyError) as e:
178                     logger.warning("Dropping segment with unparseable timestamps: %s ?
%s", seg, e)
179             try:
180                 os.remove(w_path)
181             except OSError:
182                 pass
183             return valid
184
185         ai_segments: List[dict] = []
186         with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
187             futures = [executor.submit(process_candidate_window, wi, cw)
188                         for wi, cw in enumerate(all_candidate_windows)]
189             for future in futures:
190                 ai_segments.extend(future.result())
191         try:
192             os.rmdir(handoff_dir)
193         except OSError:
194             pass
195         logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
196
197         # --- Phase 4: DB population (shared pattern) ---
198         segments_data = []
199         for segment in ai_segments:
200             try:
201                 start = float(segment["start_time"])
202                 end = float(segment["end_time"])
203                 dur = end - start
204                 shot_count = segment.get("shuttle_exchange_count")
205                 try:
206                     shot_count = int(shot_count)
207                 except (ValueError, TypeError):
208                     shot_count = max(3, int(dur / 0.8))
209                 metadata = {"shot_count": shot_count,
210                             "shot_count_confidence":
segment.get("shot_count_confidence", "Unknown"),

```

```

211         "detector": f"wasb-hybrid-{model_name}")
212         segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
213         if segment_id:
214             candidate_id = segment.get("_candidate_id")
215             if candidate_id is not None:
216                 self.db.link_candidate_to_segment(candidate_id, segment_id)
217                 segments_data.append({"id": segment_id, "start_time": start,
"end_time": end,
218                                     "duration": dur, "metadata": metadata})
219         except Exception as e:
220             logger.warning("Skipping invalid segment struct: %s", e)
221
222         return segments_data
223
224
225     segmenter_registry.register(WasbHybridSegmenter)
226

```

17. user (2026-06-10T06:01:35.617Z)

```

1     import cv2
2     import time
3     import numpy as np
4     import random
5     from typing import List, Dict, Any, Tuple
6     import logging
7     from backend.infrastructure.database import Database
8     from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
9     from backend.results import Result, Success, Failure # standardized contract
10
11     logger = logging.getLogger(__name__)
12
13     class MotionPlaySegmenter(BasePlaySegmenter):
14         @property
15         def name(self) -> str:
16             return "motion"
17
18         @property
19         def description(self) -> str:
20             return "Traditional CV pixel-motion heuristics tracking frame differences."
21
22         def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> List[Dict[str, Any]]:
23             """
24             Processes a video to detect play segments using motion heuristics,
25             populates SQLite with detected segments, and indexes detailed events.
26             """
27             if not player_pool:
28                 player_pool = ["Avi", "Suresh", "Ramesh", "Deepak"]
29
30             cap = cv2.VideoCapture(video_path)
31             if not cap.isOpened():
32                 logger.error(f"Segmenter could not open video: {video_path}")
33                 return Failure(message=f"Could not open video: {video_path}",
is_recoverable=False)
34
35             fps = cap.get(cv2.CAP_PROP_FPS)
36             total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
37             duration = total_frames / fps if fps > 0 else 0
38
39             # Sub-sample settings (5 FPS for analysis)
40             analysis_fps = 5.0
41             frame_interval = max(1, int(fps / analysis_fps))
42

```



```

43     idx_cfg = self.config.get("indexing", {})
44     motion_threshold = idx_cfg.get("motion_threshold", 0.08)
45     min_segment_duration = idx_cfg.get("min_segment_duration", 3.0)
46     dead_time_merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
47
48     prev_gray = None
49     motion_signals = []
50     timestamps = []
51
52     frame_idx = 0
53     processed_count = 0
54     t_start = time.time()
55     # Emit a progress line roughly every 5% of the video so long runs are
56     # observable instead of silent (a full 1080p match is ~tens of minutes).
57     progress_step = max(1, total_frames // 20) if total_frames > 0 else 0
58     next_progress = progress_step
59     logger.info(f"[motion] start: {total_frames} frames @ {fps:.1f}fps "
60               f"({duration/60:.1f} min), analyzing every {frame_interval}th
frame")
61     while True:
62         # Skip decoding for intermediate frames using cap.grab() for high
performance
63         for _ in range(frame_interval - 1):
64             if not cap.grab():
65                 break
66             frame_idx += 1
67
68         ret, frame = cap.read()
69         if not ret:
70             break
71
72         # Process frame
73         gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
74         # Apply Gaussian Blur to smooth noise
75         gray = cv2.GaussianBlur(gray, (21, 21), 0)
76
77         t = frame_idx / fps
78         timestamps.append(t)
79
80         if prev_gray is not None:
81             # Frame absolute difference (highly optimized motion tracking)
82             diff = cv2.absdiff(prev_gray, gray)
83             _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
84
85             # Dilate to fill gaps
86             thresh = cv2.dilate(thresh, None, iterations=2)
87
88             # Motion ratio
89             motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
90             motion_signals.append(motion_ratio)
91         else:
92             motion_signals.append(0.0)
93
94         prev_gray = gray
95         frame_idx += 1
96         processed_count += 1
97
98         if progress_step and frame_idx >= next_progress:
99             pct = 100.0 * frame_idx / total_frames
100             elapsed = time.time() - t_start
101             eta = elapsed / (frame_idx / total_frames) - elapsed if frame_idx else 0
102             logger.info(f"[motion] {pct:5.1f}% ? {frame_idx}/{total_frames} frames,
"
103                       f"{processed_count} analyzed, {elapsed:.0f}s elapsed,

```

```

~{eta:.0f}s left")
104         next_progress += progress_step
105
106         if max_frames is not None and processed_count >= max_frames:
107             logger.debug(f"Reached max_frames limit of {max_frames}. Halting indexer
frame parsing.")
108             break
109
110         if frame_idx >= total_frames:
111             break
112
113         cap.release()
114
115         # Smooth signal using a moving average
116         window_size = 5
117         smoothed = []
118         for i in range(len(motion_signals)):
119             start = max(0, i - window_size // 2)
120             end = min(len(motion_signals), i + window_size // 2 + 1)
121             smoothed.append(sum(motion_signals[start:end]) / (end - start))
122
123         # Detect active play boundaries
124         in_segment = False
125         segment_start = None
126         raw_segments: List[Tuple[float, float]] = []
127
128         for i, t in enumerate(timestamps):
129             signal = smoothed[i]
130             if signal > motion_threshold:
131                 if not in_segment:
132                     in_segment = True
133                     segment_start = t
134             else:
135                 if in_segment:
136                     in_segment = False
137                     raw_segments.append((segment_start, t))
138
139         # Handle video ending while in active segment
140         if in_segment:
141             raw_segments.append((segment_start, timestamps[-1]))
142
143         # Post-Processing: 1. Merge segments separated by less than dead_time_merge_gap
144         merged_segments: List[Tuple[float, float]] = []
145         if raw_segments:
146             curr_start, curr_end = raw_segments[0]
147             for next_start, next_end in raw_segments[1:]:
148                 if next_start - curr_end < dead_time_merge_gap:
149                     # Merge
150                     curr_end = next_end
151                 else:
152                     merged_segments.append((curr_start, curr_end))
153                     curr_start, curr_end = next_start, next_end
154             merged_segments.append((curr_start, curr_end))
155
156         # Post-Processing: 2. Filter segments shorter than min_segment_duration
157         final_segments: List[Tuple[float, float]] = [
158             r for r in merged_segments if (r[1] - r[0]) >= min_segment_duration
159         ]
160
161         # Populate SQLite DB with segments and metadata events
162         segments_data = []
163         for start, end in final_segments:
164             # Assign Server and Receiver randomly from pool
165             server, receiver = random.sample(player_pool, 2)
166

```

```

167         # Formulate dynamic rich tags
168         dur = end - start
169         metadata = {
170             "shot_count": random.randint(4, 25),
171             "intensity": "high" if dur > 12 else "medium",
172             "avg_speed_kmh": round(random.uniform(40, 85), 1)
173         }
174
175         segment_id = self.db.add_play_segment(video_id, start, end, server,
receiver, metadata)
176
177         if segment_id:
178             # Add sub-segment events
179             # A: Serve Event (always at start)
180             serve_foul = random.random() < 0.15 # 15% chance of serve foul
181             self.db.add_event(
182                 segment_id=segment_id,
183                 timestamp=start + 0.5,
184                 event_type="serve",
185                 player=server,
186                 details={"status": "foul" if serve_foul else "valid", "type":
"low_serve" if random.random() > 0.5 else "high_serve"}
187             )
188
189             # B: Smash Events (1 to 3 random smashes per segment)
190             smashes_count = random.randint(0, 3)
191             for _ in range(smashes_count):
192                 smash_time = random.uniform(start + 1.5, end - 1.0)
193                 smasher = random.choice([server, receiver])
194                 self.db.add_event(
195                     segment_id=segment_id,
196                     timestamp=round(smash_time, 2),
197                     event_type="smash",
198                     player=smasher,
199                     details={"speed_kmh": round(random.uniform(120, 250), 1)}
200                 )
201
202             # C: Segment Termination Event (foul/winner)
203             foul_types = ["out_of_bounds", "net_touch", "double_hit", "fault"]
204             is_foul = random.random() > 0.4
205             error_player = random.choice([server, receiver])
206
207             self.db.add_event(
208                 segment_id=segment_id,
209                 timestamp=end - 0.2,
210                 event_type="foul" if is_foul else "winner",
211                 player=error_player,
212                 details={"type": random.choice(foul_types)} if is_foul else
{"shot_type": "drop" if random.random() > 0.5 else "smash"}
213             )
214
215             # Store segment representation to return to caller
216             segments_data.append({
217                 "id": segment_id,
218                 "start_time": start,
219                 "end_time": end,
220                 "duration": dur,
221                 "server": server,
222                 "receiver": receiver,
223                 "metadata": metadata
224             })
225
226         return Success(value=segments_data)
227
228     # Register the segmenter

```

```
229     segmenter_registry.register(MotionPlaySegmenter)
230
```

18. user (2026-06-10T06:01:56.779Z)

```
1     """TrackNet-based hybrid play segmenter.
2
3     Same three-phase shape as HybridYoloSegmenter, but Phase 2 (candidate
4     generation) uses a TrackNetV4 *shuttle* trajectory instead of YOLO *person*
5     motion. TrackNet runs in WSL2 (see backend/detectors/tracknet_runner.py and
6     docs/DECISIONS.md ADR-001); the shuttle trajectory is converted into candidate
7     rally windows, which are then handed to the AI provider for semantic boundary
8     confirmation (Phase 3) and written to the DB (Phase 4).
9
10    Rationale: a tiny, fast, motion-blurred shuttle is exactly what general
11    detectors miss; a purpose-built trajectory model localises it far more
12    precisely, giving cleaner candidate rallies than person-velocity heuristics.
13    """
14
15    import os
16    import time
17    import logging
18    import concurrent.futures
19    from typing import List, Dict, Any
20
21    import cv2
22
23    from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
24    from backend.utils.video import get_video_duration, extract_video_chunk
25    from backend.sports import sport_registry
26    from backend.providers import provider_registry
27    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
28    from backend.pipeline.detectors.base import DetectorRunner # ABC for the de-risked
detector seam
29
30    logger = logging.getLogger(__name__)
31
32
33    def _fmt_ts(seconds: float) -> str:
34        seconds = max(0, int(seconds))
35        h, rem = divmod(seconds, 3600)
36        m, s = divmod(rem, 60)
37        return f"{h:02d}:{m:02d}:{s:02d}"
38
39
40    class TrackNetHybridSegmenter(BasePlaySegmenter):
41        @property
42        def name(self) -> str:
43            return "tracknet_hybrid"
44
45        @property
46        def description(self) -> str:
47            return ("Two-stage pipeline: TrackNetV4 shuttle-trajectory detection (WSL2) for
precise "
48                    "candidate rallies + AI provider for high-precision semantic
boundaries.")
49
50        @staticmethod
51        def _probe_fps_width(video_path: str) -> tuple:
52            """Return (fps, frame_width) for a video, with sensible fallbacks."""
53            cap = cv2.VideoCapture(video_path)
54            fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
55            width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
56            cap.release()
57            return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
```

```

58
59     def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None,
60                         max_frames: int = None) -> List[Dict[str, Any]]:
61         # --- Sport profile + AI provider wiring (identical to the other segmenters) ---
62         sport_name = self.config.get("sport", "badminton")
63         try:
64             sport_profile = sport_registry.get(sport_name)
65         except KeyError as e:
66             logger.error(str(e))
67             return []
68
69         idx_cfg = self.config.get("indexing", {})
70         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
71         model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
72
73         api_key_env_var = f"{provider_name.upper()}_API_KEY"
74         api_key = os.environ.get(api_key_env_var)
75         if not api_key:
76             logger.error(
77                 f"{api_key_env_var} environment variable is not set! "
78                 f"To run the {provider_name} handoff, set your API key. "
79                 f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
80             )
81             return []
82         try:
83             provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
84         except KeyError as e:
85             logger.error(str(e))
86             return []
87
88         video_duration = get_video_duration(video_path)
89         if video_duration <= 0:
90             logger.error("Invalid video duration.")
91             return []
92         fps, frame_width = self._probe_fps_width(video_path)
93
94         # --- TrackNet runner config + thresholds ---
95         tn_cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
96         runner: DetectorRunner = TrackNetRunner(tn_cfg)
97
98         tn = idx_cfg.get("tracknet", {})
99         velocity_thresh = tn.get("velocity_threshold", 0.02) # normalised shuttle
displacement/sec
100         merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
101         min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
102         handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
103         ai_max_workers = idx_cfg.get("ai_max_workers", 5)
104         log_candidates = idx_cfg.get("log_yolo_candidates", True)
105         store_candidates = idx_cfg.get("store_yolo_candidates", True)
106
107         detection_params = {
108             "detector": "tracknet_hybrid",
109             "velocity_thresh": velocity_thresh,
110             "merge_gap": merge_gap,
111             "min_action_window_duration": min_window_duration,
112             "weights_path": tn_cfg.weights_path,
113             "queue_length": tn_cfg.queue_length,
114         }
115
116         # --- Phase 1: env healthcheck (fail fast with a clear message) ---
117         ok, msg = runner.healthcheck()

```

```

118         if not ok:
119             logger.error("TrackNet WSL environment not ready: %s", msg)
120             return []
121         logger.info("TrackNet WSL env OK: %s", msg)
122
123         # --- Phase 2: shuttle trajectory -> candidate rally windows ---
124         # TrackNet processes the whole video in one pass (it carries its own
125         # frame buffering), so unlike the YOLO segmenter we don't pre-chunk here.
126         t_start = time.time()
127         tn_out_dir = os.path.join("output", "tracknet")
128         csv_path = runner.run_predict(video_path, tn_out_dir)
129         if not csv_path:
130             logger.error("TrackNet produced no trajectory; aborting.")
131             return []
132
133         points = runner.parse_trajectory_csv(csv_path)
134         logger.info("Parsed %d trajectory points (%d visible).",
135                     len(points), sum(1 for p in points if p.visible))
136
137         all_candidate_windows = runner.trajectory_to_action_windows(
138             points, fps=fps, frame_width=frame_width, chunk_start=0.0,
139             velocity_thresh=velocity_thresh, merge_gap=merge_gap,
140             min_window_duration=min_window_duration, chunk_index=0,
141         )
142         logger.info("Phase 2 (TrackNet) completed in %.1fs. Candidate windows: %d",
143                     time.time() - t_start, len(all_candidate_windows))
144
145         if log_candidates:
146             for w in all_candidate_windows:
147                 logger.info(
148                     f"[TrackNet rally] {_fmt_ts(w['start'])}-{_fmt_ts(w['end'])} "
149                     f"({w['end'] - w['start']:.1f}s) | frames={w['active_frame_count']}"
150                     f"peak_v={w['peak_velocity']:.3f} mean_v={w['mean_velocity']:.3f}"
151                 )
152
153         # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
154         candidate_ids = [None] * len(all_candidate_windows)
155         if store_candidates:
156             for i, w in enumerate(all_candidate_windows):
157                 stats = {
158                     "peak_velocity": w["peak_velocity"],
159                     "mean_velocity": w["mean_velocity"],
160                     "active_frame_count": w["active_frame_count"],
161                     "track_id_count": w["track_id_count"],
162                 }
163                 confidence = min(1.0, w["peak_velocity"])
164                 candidate_ids[i] = self.db.add_candidate_segment(
165                     video_id, detector="tracknet_hybrid",
166                     start_time=w["start"], end_time=w["end"],
167                     chunk_index=w["chunk_index"], confidence=confidence,
168                     stats=stats, detection_params=detection_params,
169                 )
170
171         if not all_candidate_windows:
172             return []
173
174         # --- Phase 3: AI provider handoff (identical pattern to HybridYoloSegmenter)
175         ---
176         handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
177         os.makedirs(handoff_dir, exist_ok=True)
178         logger.info("Phase 3: %s validation on %d candidate windows...",
179                     provider_name, len(all_candidate_windows))
180
181         def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:

```

```

181         w_start = max(0, window["start"] - handoff_padding)
182         w_end = min(video_duration, window["end"] + handoff_padding)
183         w_dur = w_end - w_start
184         w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
185         if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
186             return []
187         prompt = sport_profile.get_prompt(chunk_duration=w_dur)
188         segments, _ = provider.analyze_video(w_path, prompt, chunk_duration=w_dur,
189                                           label=f"Handoff {wi+1}")
190         valid = []
191         for seg in segments:
192             try:
193                 seg["start_time"] = float(seg["start_time"]) + w_start
194                 seg["end_time"] = float(seg["end_time"]) + w_start
195                 seg["_candidate_id"] = candidate_ids[wi]
196                 valid.append(seg)
197             except (ValueError, TypeError, KeyError) as e:
198                 logger.warning("Dropping segment with unparseable timestamps: %s ?
%s", seg, e)
199         try:
200             os.remove(w_path)
201         except OSError:
202             pass
203         return valid
204
205     ai_segments: List[dict] = []
206     with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
207         futures = [executor.submit(process_candidate_window, wi, w)
208                     for wi, w in enumerate(all_candidate_windows)]
209         for future in futures:
210             ai_segments.extend(future.result())
211
212     try:
213         os.rmdir(handoff_dir)
214     except OSError:
215         pass
216     logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
217
218     # --- Phase 4: DB population (identical pattern to HybridYoloSegmenter) ---
219     segments_data = []
220     for segment in ai_segments:
221         try:
222             start = float(segment["start_time"])
223             end = float(segment["end_time"])
224             dur = end - start
225             shot_count = segment.get("shuttle_exchange_count")
226             if shot_count is None:
227                 shot_count = max(3, int(dur / 0.8))
228             else:
229                 try:
230                     shot_count = int(shot_count)
231                 except (ValueError, TypeError):
232                     shot_count = max(3, int(dur / 0.8))
233             metadata = {
234                 "shot_count": shot_count,
235                 "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
236                 "detector": f"tracknet-hybrid-{model_name}",
237             }
238             segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
239             if segment_id:

```

```

240         candidate_id = segment.get("_candidate_id")
241         if candidate_id is not None:
242             self.db.link_candidate_to_segment(candidate_id, segment_id)
243         segments_data.append({
244             "id": segment_id, "start_time": start, "end_time": end,
245             "duration": dur, "metadata": metadata,
246         })
247     except Exception as e:
248         logger.warning("Skipping invalid segment struct: %s", e)
249
250     return segments_data
251
252
253 # Register the segmenter
254 segmenter_registry.register(TrackNetHybridSegmenter)
255

```

19. user (2026-06-10T06:01:57.742Z)

```

1     """Two-stage rally segmenter: torchvision person detection + ByteTrack tracking
2     (Stage 1, local) feeding an AI provider for semantic boundaries (Stage 2).
3
4     Stage 1 originally used the AGPL-licensed ultralytics YOLO. To keep the hosted
5     service free of copyleft obligations (see docs/MONETIZATION_AUDIT.md and ADR-005),
6     detection now uses torchvision's permissively-licensed (BSD) detectors and the
7     association/tracking that YOLO.track() used to bundle in is provided by
8     supervision's MIT-licensed ByteTrack.
9
10    The registry key stays "yolo_hybrid" for back-compat with existing configs and the
11    DB `detector` tags, even though no YOLO is involved anymore.
12    """
13    import os
14    import time
15    import logging
16    import cv2
17    import torch
18    import concurrent.futures
19    from typing import List, Dict, Any, Tuple
20
21    from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
22    from backend.utils.video import get_video_duration, extract_video_chunk
23    from backend.utils.geometry import get_velocity, get_velocity_normalised
24    from backend.sports import sport_registry
25    from backend.providers import provider_registry
26
27    logger = logging.getLogger(__name__)
28
29    # torchvision detection models are trained on COCO where the "person" category is
30    # class 1 (index 0 is the implicit background). NB: ultralytics YOLO used class 0 ?
31    # this off-by-one is the classic gotcha when swapping detectors. Keep it named.
32    _PERSON_CLASS_ID = 1
33
34    # Permissively-licensed torchvision detectors selectable via config
35    # `indexing.detector_model`. All are BSD-licensed (code + COCO weights).
36    _DETECTOR_FACTORIES = {
37        "fasterrcnn_resnet50_fpn": "fasterrcnn_resnet50_fpn",
38        "fasterrcnn_mobilenet_v3_large_fpn": "fasterrcnn_mobilenet_v3_large_fpn",
39        "retinanet_resnet50_fpn": "retinanet_resnet50_fpn",
40    }
41    _DEFAULT_DETECTOR = "fasterrcnn_resnet50_fpn"
42
43
44    def _fmt_ts(seconds: float) -> str:
45        """Format seconds as HH:MM:SS for cross-verifying a timestamp against the video."""
46        seconds = max(0, int(seconds))

```



```

47         h, rem = divmod(seconds, 3600)
48         m, s = divmod(rem, 60)
49         return f"{h:02d}:{m:02d}:{s:02d}"
50
51
52     class HybridYoloSegmenter(BasePlaySegmenter):
53         def __init__(self, db, config):
54             super().__init__(db, config)
55             self.model = None
56             self.device = None
57
58             @property
59             def name(self) -> str:
60                 return "yolo_hybrid"
61
62             @property
63             def description(self) -> str:
64                 return ("Two-stage pipeline: torchvision person detection + ByteTrack for fast "
65                         "candidate filtering, then an AI provider for high-precision semantic
66 boundaries.")
67
68             def _lazy_load_model(self):
69                 if self.model is not None:
70                     return
71
72                 # Imported lazily so the module imports without torchvision present and so
73                 # the test suite (which pre-sets self.model) never pulls in model weights.
74                 from torchvision.models import detection as tv_detection
75
76                 idx_cfg = self.config.get("indexing", {})
77                 use_gpu = idx_cfg.get("use_gpu", True)
78                 self.device = "cuda" if use_gpu and torch.cuda.is_available() else "cpu"
79                 if use_gpu and not torch.cuda.is_available():
80                     logger.warning("use_gpu is True but CUDA is not available. Falling back to
81 CPU.")
82
83                 model_name = idx_cfg.get("detector_model", _DEFAULT_DETECTOR)
84                 if model_name not in _DETECTOR_FACTORIES:
85                     logger.warning(f"Unknown detector_model '{model_name}'. Falling back to
86 { _DEFAULT_DETECTOR}.")
87                 model_name = _DEFAULT_DETECTOR
88
89                 builder = getattr(tv_detection, _DETECTOR_FACTORIES[model_name])
90                 logger.info(f"Loading torchvision detector '{model_name}' on {self.device}...")
91                 # weights="DEFAULT" pulls the best available COCO-pretrained checkpoint.
92                 self.model = builder(weights="DEFAULT")
93                 self.model.eval()
94                 self.model.to(self.device)
95
96             def _make_tracker(self, fps: float):
97                 """Create a fresh ByteTrack tracker for one chunk (track IDs are chunk-local).
98
99                 Imported lazily so tests can mock this method without supervision installed.
100                 """
101                 import supervision as sv
102                 frame_rate = max(1, int(round(fps)))
103                 return sv.ByteTrack(frame_rate=frame_rate)
104
105             def _detect_and_track(self, frame, tracker, score_thresh: float) -> List[Tuple[int,
106 Tuple[float, float, float, float]]]:
107                 """Detect persons in a frame and assign persistent track IDs.
108
109                 Returns a list of (track_id, bbox) where bbox is (x1, y1, x2, y2) in pixels.
110                 This replaces the old ultralytics ``model.track(...)`` call, which bundled
111                 detection AND association: torchvision does the detection, supervision's

```

```

108         ByteTrack does the association.
109         """
110         import numpy as np
111         import supervision as sv
112
113         # cv2 frames are BGR uint8 HxWxC; torchvision wants RGB float [0,1] CxHxW.
114         rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
115         tensor = torch.from_numpy(rgb).permute(2, 0,
116         1).float().div(255.0).to(self.device)
117         with torch.no_grad():
118             outputs = self.model([tensor])
119
120         out = outputs[0]
121         boxes = out["boxes"].detach().cpu().numpy()
122         labels = out["labels"].detach().cpu().numpy()
123         scores = out["scores"].detach().cpu().numpy()
124
125         # Keep only confident person detections.
126         mask = (labels == _PERSON_CLASS_ID) & (scores >= score_thresh)
127         if not mask.any():
128             return []
129
130         # ByteTrack requires confidence to be populated (it filters on it internally).
131         dets = sv.Detections(xyxy=boxes[mask], confidence=scores[mask],
132         class_id=labels[mask])
133         tracked = tracker.update_with_detections(dets)
134
135         result: List[Tuple[int, Tuple[float, float, float, float]]] = []
136         for i in range(len(tracked)):
137             tid = tracked.tracker_id[i]
138             if tid is None:
139                 continue
140             x1, y1, x2, y2 = tracked.xyxy[i]
141             result.append((int(tid), (float(x1), float(y1), float(x2), float(y2))))
142         return result
143
144     def _extract_action_windows_from_chunk(self, chunk_path: str, chunk_start: float,
145     velocity_thresh: float, merge_gap: float,
146     frame_skip: int = 3,
147     tracker_config: str = "bytetrack.yaml",
148     chunk_index: int = None) -> List[Dict[str,
149     Any]]:
150         """
151         Runs person detection + tracking on a video chunk and extracts high-motion
152         action windows. Returns a list of dicts in the full video's timeframe, each
153         with:
154
155             start, end (seconds), active_frame_count, peak_velocity, mean_velocity,
156             track_id_count, chunk_index.
157
158         Args:
159             frame_skip: Process every Nth frame (1 = every frame). Higher values
160             dramatically reduce inference cost with minimal quality loss.
161             tracker_config: Retained for back-compat (was the YOLO tracker yaml); the
162             tracker is now supervision ByteTrack and this is unused.
163             chunk_index: Index of the source chunk (recorded on each window for
164             traceability).
165         """
166         cap = cv2.VideoCapture(chunk_path)
167         if not cap.isOpened():
168             logger.error(f"Could not open {chunk_path}")
169             return []
170
171         fps = cap.get(cv2.CAP_PROP_FPS)
172         if fps <= 0:
173             fps = 30.0

```

```

168
169     frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
170     if frame_width <= 0:
171         frame_width = 1280.0 # Sensible fallback
172
173     # Effective sampling rate after frame-skipping
174     effective_fps = fps / max(frame_skip, 1)
175
176     # A fresh tracker per chunk ? track IDs only need to be consistent within a
177     # chunk (windows are computed per chunk, then merged across chunks by time).
178     tracker = self._make_tracker(effective_fps)
179     score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
0.5)
180
181     # Per active frame we keep (frame_idx, peak_velocity, set_of_active_track_ids)
182     # so windows can be enriched with motion stats for logging + fine-tuning.
183     active_frames = []
184     frame_idx = 0
185     track_history = {}
186
187     while True:
188         ret, frame = cap.read()
189         if not ret:
190             break
191
192         # Skip frames for performance ? at 30fps, every 3rd frame still
193         # gives ~10 samples/second, plenty for human-scale velocity.
194         if frame_idx % max(frame_skip, 1) != 0:
195             frame_idx += 1
196             continue
197
198         # Detect persons + assign persistent track IDs.
199         detections = self._detect_and_track(frame, tracker, score_thresh)
200
201         frame_peak_velocity = 0.0
202         frame_active_ids = set()
203         current_ids = set()
204
205         for track_id, bbox in detections:
206             current_ids.add(track_id)
207
208             if track_id in track_history:
209                 prev_bbox = track_history[track_id]
210                 velocity = get_velocity_normalised(prev_bbox, bbox, effective_fps,
frame_width)
211                 if velocity > velocity_thresh:
212                     frame_peak_velocity = max(frame_peak_velocity, velocity)
213                     frame_active_ids.add(track_id)
214
215                 track_history[track_id] = bbox
216
217         # Prune stale IDs that are no longer visible ? prevents false
218         # velocity spikes when the tracker reassigns an old ID.
219         track_history = {k: v for k, v in track_history.items() if k in current_ids}
220
221         if frame_active_ids:
222             active_frames.append((frame_idx, frame_peak_velocity, frame_active_ids))
223
224         frame_idx += 1
225
226     cap.release()
227
228     # Group active frames into continuous windows
229     if not active_frames:
230         return []

```

```

231
232     max_gap_frames = int(merge_gap * fps)
233
234     def _finalize(group: List[Tuple[int, float, set]]) -> Dict[str, Any]:
235         velocities = [v for _, v, _ in group]
236         track_ids = set().union(*[ids for _, _, ids in group])
237         return {
238             "start": group[0][0] / fps + chunk_start,
239             "end": group[-1][0] / fps + chunk_start,
240             "active_frame_count": len(group),
241             "peak_velocity": max(velocities),
242             "mean_velocity": sum(velocities) / len(velocities),
243             "track_id_count": len(track_ids),
244             "chunk_index": chunk_index,
245         }
246
247     windows = []
248     group = [active_frames[0]]
249     for af in active_frames[1:]:
250         if af[0] - group[-1][0] <= max_gap_frames:
251             group.append(af)
252         else:
253             windows.append(_finalize(group))
254             group = [af]
255     windows.append(_finalize(group))
256
257     # Filter out extremely short action windows
258     min_window_duration = self.config.get("indexing",
259     {}).get("min_action_window_duration", 2.0)
260     windows = [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
261
262     return windows
263
264     def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
265     None, max_frames: int = None) -> List[Dict[str, Any]]:
266         self._lazy_load_model()
267
268         # Get sport profile config
269         sport_name = self.config.get("sport", "badminton")
270         try:
271             sport_profile = sport_registry.get(sport_name)
272         except KeyError as e:
273             logger.error(str(e))
274             return []
275
276         # Get AI provider and model config
277         idx_cfg = self.config.get("indexing", {})
278         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
279         "gemini"))
280         model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
281         "gemini-2.5-flash"))
282
283         # Get appropriate API key based on the provider
284         api_key_env_var = f"{provider_name.upper()}_API_KEY"
285         api_key = os.environ.get(api_key_env_var)
286         if not api_key:
287             logger.error(
288                 f"{api_key_env_var} environment variable is not set! "
289                 f"To run the {provider_name} segmenter, set your API key. "
290                 f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
291             )
292             return []
293
294         try:
295             provider = provider_registry.get(provider_name, api_key=api_key,

```

```

model_name=model_name)
292         except KeyError as e:
293             logger.error(str(e))
294             return []
295
296         logger.info(f"Initializing Hybrid Segmenter (torchvision + ByteTrack) for video:
{video_path}")
297
298         video_duration = get_video_duration(video_path)
299         if video_duration <= 0:
300             logger.error("Invalid video duration.")
301             return []
302
303         chunk_duration = idx_cfg.get("chunk_duration", 300)
304         chunk_overlap = idx_cfg.get("chunk_overlap", 30)
305         velocity_thresh = idx_cfg.get("yolo_velocity_threshold", 0.15) # Fraction of
frame width per second
306         merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
307         frame_skip = idx_cfg.get("yolo_frame_skip", 3)
308         tracker_config = idx_cfg.get("yolo_tracker", "bytetrack.yaml")
309         handoff_padding = idx_cfg.get("ai_handoff_padding",
idx_cfg.get("gemini_handoff_padding", 2.0))
310         ai_max_workers = idx_cfg.get("ai_max_workers", idx_cfg.get("gemini_max_workers",
5))
311         log_candidates = idx_cfg.get("log_yolo_candidates", True)
312         store_candidates = idx_cfg.get("store_yolo_candidates", True)
313
314         # Snapshot of the params that produced these detections ? stored alongside each
315         # candidate so a fine-tuning run can reproduce / correlate against them.
316         detection_params = {
317             "velocity_thresh": velocity_thresh,
318             "merge_gap": merge_gap,
319             "frame_skip": frame_skip,
320             "tracker": tracker_config,
321             "detector_model": idx_cfg.get("detector_model", _DEFAULT_DETECTOR),
322             "detector_score_threshold": idx_cfg.get("detector_score_threshold", 0.5),
323             "min_action_window_duration": idx_cfg.get("min_action_window_duration",
2.0),
324         }
325
326         chunks_dir = os.path.join("output", "chunks_yolo")
327         os.makedirs(chunks_dir, exist_ok=True)
328
329         step = chunk_duration - chunk_overlap
330         chunk_starts = []
331         t = 0.0
332         while t < video_duration:
333             chunk_starts.append(t)
334             t += step
335
336         num_chunks = len(chunk_starts)
337         logger.info(f"Video duration: {video_duration:.1f}s. Splitting into {num_chunks}
chunks.")
338
339         all_candidate_windows = []
340
341         t_start = time.time()
342         prefetch_workers = idx_cfg.get("yolo_prefetch_workers", 2)
343
344         if num_chunks > 1 and prefetch_workers > 0:
345             # --- PIPELINED PROCESSING ---
346             # GPU inference must stay sequential (single GPU), but ffmpeg chunk
347             # extraction is pure I/O. We pre-extract upcoming chunks in background
348             # threads so extraction of chunk N+1 overlaps with inference on chunk N.
349             logger.info(f"Running detection Phase 2 with pipelined extraction

```

```

({prefetch_workers} prefetch workers)...")
350
351         extract_executor =
concurrent.futures.ThreadPoolExecutor(max_workers=prefetch_workers)
352
353         def _extract_chunk(ci: int, cs: float) -> Tuple[int, float, str, bool]:
354             chunk_end = min(cs + chunk_duration, video_duration)
355             actual_dur = chunk_end - cs
356             chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
357             success = extract_video_chunk(video_path, cs, actual_dur, chunk_path)
358             return (ci, cs, chunk_path, success)
359
360         # Submit all extractions upfront ? they'll queue and run as workers free up
361         extract_futures = [
362             extract_executor.submit(_extract_chunk, ci, cs)
363             for ci, cs in enumerate(chunk_starts)
364         ]
365
366         # Process results in order: wait for extraction, then run inference on GPU
367         for future in extract_futures:
368             ci, chunk_start, chunk_path, success = future.result()
369             chunk_end = min(chunk_start + chunk_duration, video_duration)
370             logger.info(f"Processing Chunk {ci+1}/{num_chunks} ({chunk_start:.1f}s -
{chunk_end:.1f}s)...")
371
372             if not success:
373                 logger.warning(f"Chunk {ci+1} extraction failed. Skipping.")
374                 continue
375
376             windows = self._extract_action_windows_from_chunk(
377                 chunk_path, chunk_start, velocity_thresh, merge_gap,
378                 frame_skip=frame_skip, tracker_config=tracker_config, chunk_index=ci
379             )
380             logger.info(f"Found {len(windows)} candidate action windows in chunk
{ci+1}.")
381             all_candidate_windows.extend(windows)
382
383             try:
384                 os.remove(chunk_path)
385             except Exception:
386                 pass
387
388         extract_executor.shutdown(wait=False)
389     else:
390         # --- SEQUENTIAL PROCESSING (single chunk or prefetch disabled) ---
391         for ci, chunk_start in enumerate(chunk_starts):
392             chunk_end = min(chunk_start + chunk_duration, video_duration)
393             actual_dur = chunk_end - chunk_start
394             chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
395
396             logger.info(f"Processing Chunk {ci+1}/{num_chunks} ({chunk_start:.1f}s -
{chunk_end:.1f}s)...")
397             success = extract_video_chunk(video_path, chunk_start, actual_dur,
chunk_path)
398             if not success:
399                 continue
400
401             windows = self._extract_action_windows_from_chunk(
402                 chunk_path, chunk_start, velocity_thresh, merge_gap,
403                 frame_skip=frame_skip, tracker_config=tracker_config, chunk_index=ci
404             )
405             logger.info(f"Found {len(windows)} candidate action windows in chunk
{ci+1}.")
406             all_candidate_windows.extend(windows)
407

```

```

408         try:
409             os.remove(chunk_path)
410         except Exception:
411             pass
412
413     # Deduplicate overlapping candidate windows across chunks (chunks overlap by
414     # chunk_overlap, so the same rally can surface in two adjacent chunks).
415     def _merge_stats(a: Dict[str, Any], b: Dict[str, Any]) -> Dict[str, Any]:
416         fa, fb = a["active_frame_count"], b["active_frame_count"]
417         total = fa + fb or 1
418         return {
419             "start": a["start"],
420             "end": max(a["end"], b["end"]),
421             "active_frame_count": fa + fb,
422             "peak_velocity": max(a["peak_velocity"], b["peak_velocity"]),
423             # Frame-weighted mean so longer sub-windows dominate proportionally.
424             "mean_velocity": (a["mean_velocity"] * fa + b["mean_velocity"] * fb) /
total,
425             "track_id_count": max(a["track_id_count"], b["track_id_count"]),
426             "chunk_index": a["chunk_index"],
427         }
428
429     if all_candidate_windows:
430         all_candidate_windows.sort(key=lambda w: w["start"])
431         merged_windows = [all_candidate_windows[0]]
432         for current in all_candidate_windows[1:]:
433             last = merged_windows[-1]
434             if current["start"] <= last["end"]: # Overlap
435                 merged_windows[-1] = _merge_stats(last, current)
436             else:
437                 merged_windows.append(current)
438         all_candidate_windows = merged_windows
439
440     logger.info(f"Phase 2 tracking completed in {time.time() - t_start:.1f}s.")
441     logger.info(f"Total candidate windows: {len(all_candidate_windows)}")
442
443     # Per-rally console log (final, full-video timestamps) for video cross-
444     verification.
445     if log_candidates:
446         for w in all_candidate_windows:
447             logger.info(
448                 f"[rally] {_fmt_ts(w['start'])}-{_fmt_ts(w['end'])} "
449                 f"({w['end'] - w['start']:.1f}s) | frames={w['active_frame_count']}"
450                 f"peak_v={w['peak_velocity']:.3f} mean_v={w['mean_velocity']:.3f} "
451                 f"tracks={w['track_id_count']}"
452             )
453
454     # Persist raw candidates for the fine-tuning dataset. Map window index ->
455     candidate_id
456     # so confirmed AI segments can be linked back in Phase 4.
457     candidate_ids = [None] * len(all_candidate_windows)
458     if store_candidates:
459         for i, w in enumerate(all_candidate_windows):
460             stats = {
461                 "peak_velocity": w["peak_velocity"],
462                 "mean_velocity": w["mean_velocity"],
463                 "active_frame_count": w["active_frame_count"],
464                 "track_id_count": w["track_id_count"],
465             }
466             # Use peak velocity (capped at 1.0) as a coarse generic confidence
467             score.
468             confidence = min(1.0, w["peak_velocity"])
469             candidate_ids[i] = self.db.add_candidate_segment(
470                 video_id, detector="yolo_hybrid",

```

```

468             start_time=w["start"], end_time=w["end"],
469             chunk_index=w["chunk_index"], confidence=confidence,
470             stats=stats, detection_params=detection_params,
471         )
472     )
473     try:
474         os.rmdir(chunks_dir)
475     except Exception:
476         pass
477
478     # Phase 3: AI Provider Handoff
479     gemini_segments = []
480     gemini_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
481     os.makedirs(gemini_dir, exist_ok=True)
482
483     logger.info(f"Starting Phase 3: AI Provider ({provider_name}) Validation on
484 {len(all_candidate_windows)} candidate windows...")
485
486     def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
487         w_start = max(0, window["start"] - handoff_padding)
488         w_end = min(video_duration, window["end"] + handoff_padding)
489         w_dur = w_end - w_start
490
491         w_path = os.path.join(gemini_dir, f"handoff_{video_id}_{wi}.mp4")
492         # Re-encode short handoff clips to avoid keyframe-boundary drift
493         success = extract_video_chunk(video_path, w_start, w_dur, w_path,
494 reencode=True)
495         if not success:
496             return []
497
498         prompt = sport_profile.get_prompt(chunk_duration=w_dur)
499         segments, metrics = provider.analyze_video(w_path, prompt,
500 chunk_duration=w_dur, label=f"Handoff {wi+1}")
501
502         # Coordinate mapping ? shift local timestamps back to full-video time
503         valid_segments = []
504         for segment in segments:
505             try:
506                 segment["start_time"] = float(segment["start_time"]) + w_start
507                 segment["end_time"] = float(segment["end_time"]) + w_start
508                 # Stamp the source candidate so Phase 4 can link
509                 prediction->confirmation.
510                 segment["candidate_id"] = candidate_ids[wi]
511                 valid_segments.append(segment)
512             except (ValueError, TypeError, KeyError) as e:
513                 logger.warning(f"Dropping segment with unparseable timestamps:
514 {segment} ? {e}")
515
516         try:
517             os.remove(w_path)
518         except OSError:
519             pass
520
521         return valid_segments
522
523     with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
524 executor:
525         futures = [executor.submit(process_candidate_window, wi, w) for wi, w in
526 enumerate(all_candidate_windows)]
527         # Iterate in submission order to preserve temporal ordering
528         for future in futures:
529             gemini_segments.extend(future.result())
530
531     try:
532         os.rmdir(gemini_dir)

```



```

526         except OSError:
527             pass
528
529         logger.info(f"Phase 3 completed. AI confirmed {len(gemini_segments)} semantic
play segments.")
530
531         # Phase 4: DB Population
532         segments_data = []
533         for segment in gemini_segments:
534             try:
535                 start = float(segment["start_time"])
536                 end = float(segment["end_time"])
537                 dur = end - start
538
539                 shot_count = segment.get("shuttle_exchange_count")
540                 if shot_count is None: shot_count = max(3, int(dur / 0.8))
541                 else:
542                     try: shot_count = int(shot_count)
543                     except (ValueError, TypeError): shot_count = max(3, int(dur / 0.8))
544
545                 metadata = {
546                     "shot_count": shot_count,
547                     "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
548                     "detector": f"yolo-hybrid-{model_name}"
549                 }
550
551                 segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
552                 if segment_id:
553                     # Link back to the candidate that produced this segment. A candidate
554                     # may yield multiple AI segments (one-to-many); confirmed_segment_id
555                     # the last one ? the link table records "this candidate was
confirmed".
556                     candidate_id = segment.get("_candidate_id")
557                     if candidate_id is not None:
558                         self.db.link_candidate_to_segment(candidate_id, segment_id)
559                     segments_data.append({
560                         "id": segment_id,
561                         "start_time": start,
562                         "end_time": end,
563                         "duration": dur,
564                         "metadata": metadata
565                     })
566             except Exception as e:
567                 logger.warning(f"Skipping invalid segment struct: {e}")
568
569         return segments_data
570
571     # Register the segmenter
572     segmenter_registry.register(HybridYoloSegmenter)
573

```

20. user (2026-06-10T06:02:05.533Z)

```

1     """Per-video observability reports for the indexer.
2
3     `obsreport` is a SOFT dependency: if it is not installed, this module degrades to a
4     no-op and the pipeline runs exactly as before. The single entrypoint
5     `emit_indexer_report(...)` is designed to be called from a `finally` block and
6     NEVER raises ? emitting a report must not be able to break a processing run.
7     """
8     from __future__ import annotations
9

```

```

10 import os
11 from typing import Any, Dict, List, Optional
12
13 try: # soft dependency ? never hard-fail the pipeline on a reporting import
14     import obsreport # noqa: F401
15     from . import harvest
16
17     OBSREPORT_AVAILABLE = True
18 except Exception: # pragma: no cover - exercised only when obsreport is absent
19     OBSREPORT_AVAILABLE = False
20
21
22 def report_enabled(config: Optional[Dict[str, Any]]) -> bool:
23     return bool((config or {}).get("report", {}).get("enabled", True))
24
25
26 def emit_indexer_report(
27     *,
28     db,
29     video_id: str,
30     video_path: Optional[str],
31     config: Optional[Dict[str, Any]],
32     val_results: List[Any],
33     val_context: Optional[Dict[str, Any]] = None,
34     segmenter_requested: Optional[str] = None,
35     segmenter_actual: Optional[str] = None,
36     was_reingest: bool = False,
37     segmentation_ran: bool = True,
38 ) -> Optional[Dict[str, str]]:
39     """Harvest + write the report. Returns the written file paths, or None if
40     reporting is unavailable/disabled/failed. Never raises."""
41     config = config or {}
42     # Accept either a plain dict or a typed config object (e.g.
43     backend.config.AppConfig).
44     # Normalize to a plain nested dict so harvest's dict access works uniformly.
45     if hasattr(config, "model_dump"):
46         try:
47             config = config.model_dump(mode="json")
48         except Exception:
49             config = {}
50     if not OBSREPORT_AVAILABLE or not report_enabled(config):
51         return None
52     try:
53         play_segments = db.query_play_segments(video_id, {}) if db is not None else []
54         candidates = db.query_candidate_segments(video_id) if db is not None else []
55         rec = harvest.record_run(
56             video_id=video_id,
57             video_path=video_path,
58             config=config,
59             val_results=val_results,
60             val_context=val_context,
61             play_segments=play_segments,
62             candidates=candidates,
63             segmenter_requested=segmenter_requested,
64             segmenter_actual=segmenter_actual,
65             was_reingest=was_reingest,
66             segmentation_ran=segmentation_ran,
67         )
68         output_dir = (config.get("report") or {}).get("output_dir")
69         res = rec.write(output_dir=output_dir)
70         return res.paths or None
71     except Exception: # pragma: no cover - defensive; reporting must never break a run
72         return None

```

21. user (2026-06-10T06:02:05.726Z)

```
1      """Build an observability report for one indexer run.
2
3      Pure-ish harvest: given the validation results/context, the persisted segments,
4      and run metadata, populate an `obsreport.RunRecorder` with the raw signals. The
5      *footgun rules* (which turn signals into flags) live declaratively in spec.yaml,
6      so this module only records facts faithfully ? it does not decide warnings.
7
8      Imported only after `backend.reporting` confirms obsreport is installed, so it may
9      import obsreport freely.
10     """
11     from __future__ import annotations
12
13     import os
14     from pathlib import Path
15     from typing import Any, Dict, List, Optional
16
17     try:
18         import obsreport # soft dep; only needed for RunRecorder / load_spec at runtime
19     except ImportError:
20         obsreport = None # type: ignore[assignment]
21
22     _AI_SEGMENTERS = {"yolo_hybrid", "tracknet_hybrid", "wasb_hybrid", "gemini"}
23     _SPEC_PATH = Path(__file__).resolve().parent / "spec.yaml"
24
25
26     def load_spec() -> "obsreport.ReportSpec":
27         return obsreport.load_spec(_SPEC_PATH)
28
29
30     # --- media metadata -----
31     def video_meta_from_context(val_context: Optional[Dict[str, Any]], video_path:
Optional[str]) -> Dict[str, Any]:
32         """Derive VideoMeta fields from the validators' shared ffprobe_meta. fps_source
33         distinguishes a real probe from the absence of one (the fallback footgun)."""
34         meta: Dict[str, Any] = {"fps_source": "unknown"}
35         if video_path and os.path.exists(video_path):
36             try:
37                 meta["size_mb"] = round(os.path.getsize(video_path) / (1024 * 1024), 1)
38             except OSError:
39                 pass
40
41         ffmata = (val_context or {}).get("ffprobe_meta") or {}
42         streams = ffmata.get("streams") or []
43         fmt = ffmata.get("format") or {}
44         if streams:
45             s = streams[0]
46             w, h = s.get("width"), s.get("height")
47             meta["width"], meta["height"] = w, h
48             if w and h:
49                 meta["resolution"] = f"{w}x{h}"
50             fps_str = s.get("r_frame_rate", "0/1")
51             try:
52                 num, den = (int(x) for x in fps_str.split("/"))
53                 if den:
54                     meta["fps"] = round(num / den, 2)
55                     meta["fps_source"] = "probed"
56             except (ValueError, ZeroDivisionError):
57                 pass
58             if s.get("codec_name"):
59                 meta["container"] = fmt.get("format_name") or s.get("codec_name")
60         if fmt.get("duration"):
61             try:
62                 meta["duration_s"] = round(float(fmt["duration"]), 1)
```

```

63         except (ValueError, TypeError):
64             pass
65     if fmt.get("format_name"):
66         meta["container"] = fmt.get("format_name")
67
68     # Audio readiness signal (for Input Quality UX extension per review item 2).
69     # Guidelines make "record with audio on, mic unobstructed" a first-class requirement
70     # (ADR-007) because shuttle thwacks are a strong rally cue for recall.
71     # We only record presence here (cheap, from ffprobe already in context); the actual
72     # thwack onset probe (pipeline/audio/hit_detector) remains a diagnostic /
calibration
73     # tool for now.
74     audio_streams = [s for s in streams if (s or {}).get("codec_type") == "audio"]
75     meta["audio_present"] = len(audio_streams) > 0
76     return meta
77
78
79     # --- stage builders -----
80     def _validation_stage(rec: "obsreport.RunRecorder", val_results: List[Any]) -> None:
81         with rec.stage("validation") as st:
82             total = len(val_results)
83             passed = sum(1 for r in val_results if getattr(r, "passed", False))
84             st.add_metric("checks_passed", passed)
85             st.add_metric("checks_total", total)
86             failures = [r for r in val_results if not getattr(r, "passed", True)]
87             for r in failures:
88                 st.messages.append(f"FAILED {getattr(r, 'validator_name', '?')}: {getattr(r,
'message', '')}")
89             st.status = "error" if failures else ("ok" if total else "not_run")
90
91
92     def _segmentation_stage(
93         rec: "obsreport.RunRecorder",
94         play_segments: List[Dict[str, Any]],
95         candidates: List[Dict[str, Any]],
96         segmenter_requested: Optional[str],
97         segmenter_actual: Optional[str],
98         config_default: Optional[str],
99         was_reingest: bool,
100         ran: bool,
101     ) -> None:
102         with rec.stage("segmentation") as st:
103             if not ran:
104                 st.status = "not_run"
105             seg_count = len(play_segments)
106             total_play = round(sum(float(s.get("duration", 0.0)) for s in play_segments), 1)
107             detector_tag = None
108             if play_segments:
109                 detector_tag = (play_segments[0].get("metadata") or {}).get("detector")
110             st.add_metric("candidate_windows", len(candidates))
111             st.add_metric("segment_count", seg_count, audience="both")
112             st.add_metric("total_play_s", total_play, unit="s")
113             # "diverges" only when a segmenter actually RAN, the user did NOT explicitly
114             # pick it, and it differs from config's default (the genuine surprise case).
115             # An explicit --segmenter override or a halted run is not a divergence.
116             matches_default = True
117             if ran and segmenter_actual and config_default:
118                 matches_default = (segmenter_actual == config_default)
119             st.details.update({
120                 "segmenter_requested": segmenter_requested,
121                 "segmenter_actual": segmenter_actual,
122                 "config_default": config_default,
123                 "segmenter_explicitly_requested": segmenter_requested is not None,
124                 "matches_config_default": matches_default,
125                 "was_reingest": was_reingest,

```

```

126         "detector": detector_tag,
127     })
128     # Surface metadata trustworthiness at the data level (not just in a warning):
129     # the motion baseline fabricates per-segment metadata; everything else measures
130     it.
131     if ran and segmenter_actual:
132         st.details["metadata_source"] = (
133             "synthetic/heuristic ? NOT measured" if segmenter_actual == "motion"
134             else "measured (detector/AI)"
135         )
136
137     def _ai_stage(rec: "obsreport.RunRecorder", config: Dict[str, Any], segmenter_actual:
Optional[str],
138                 play_segments: List[Dict[str, Any]], ran: bool) -> None:
139         """Only meaningful for AI-based segmenters. Records whether the provider key was
140         present ? the signal the silent_provider_noop rule keys on."""
141         ai_based = segmenter_actual in _AI_SEGMENTERS
142         provider = (config.get("ai_provider") or "gemini")
143         key_env = f"{provider.upper()}_API_KEY"
144         with rec.stage("ai_handoff") as st:
145             if not ran or not ai_based:
146                 st.status = "not_run" if not ran else "skipped"
147             st.details.update({
148                 "ai_based": ai_based,
149                 "provider": provider if ai_based else None,
150                 "model": config.get("ai_model") if ai_based else None,
151                 "api_key_present": bool(os.environ.get(key_env)) if ai_based else None,
152             })
153             if ai_based:
154                 st.add_metric("confirmed_segments", len(play_segments))
155
156
157     def _highlights(rec: "obsreport.RunRecorder", play_segments: List[Dict[str, Any]],
158                   video_meta: Dict[str, Any]) -> None:
159         total = round(sum(float(s.get("duration", 0.0)) for s in play_segments), 1)
160         coverage = None
161         dur = video_meta.get("duration_s")
162         if dur:
163             coverage = round(100.0 * total / dur, 1)
164         notes = []
165         if not play_segments:
166             notes.append("No highlights were produced for this video.")
167         rec.set_highlights(segment_count=len(play_segments), total_highlight_s=total,
168                           coverage_pct=coverage, notes=notes)
169
170
171     # --- top-level -----
172     def record_run(
173         *,
174         video_id: str,
175         video_path: Optional[str],
176         config: Dict[str, Any],
177         val_results: List[Any],
178         val_context: Optional[Dict[str, Any]],
179         play_segments: List[Dict[str, Any]],
180         candidates: List[Dict[str, Any]],
181         segmenter_requested: Optional[str] = None,
182         segmenter_actual: Optional[str] = None,
183         was_reingest: bool = False,
184         segmentation_ran: bool = True,
185         tool_version: Optional[str] = None,
186         spec: Optional["obsreport.ReportSpec"] = None,
187         **recorder_kwargs: Any,
188     ) -> "obsreport.RunRecorder":

```

```

189     """Assemble a finalized recorder for an indexer run (does NOT write)."""
190     rec = obsreport.RunRecorder(
191         "badminton-highlight-indexer",
192         tool_version=tool_version,
193         video_id=video_id,
194         video_id_kind="file_md5",
195         video_path=video_path,
196         spec=spec if spec is not None else load_spec(),
197         **recorder_kwargs,
198     )
199     vm = video_meta_from_context(val_context, video_path)
200     # If we never probed an fps but segmentation still ran, the segmenter must have
201     # used its internal fallback (30fps / 1280px) ? surface that as the footgun it is,
202     # AND show the assumed numeric value so the report reads "30 (fallback)" not "?"
203     (fallback)".
204     if vm.get("fps_source") == "unknown" and segmentation_ran:
205         vm["fps_source"] = "fallback"
206         if vm.get("fps") is None:
207             vm["fps"] = 30.0 # matches the runners' fallback (_probe_fps_width -> 30.0
208             / 1280)
209     rec.set_video_meta(**vm)
210     _validation_stage(rec, val_results)
211     config_default = (config.get("indexing") or {}).get("default_segmenter")
212     _segmentation_stage(rec, play_segments, candidates, segmenter_requested,
213                         segmenter_actual, config_default, was_reingest,
214                         segmentation_ran)
215     _ai_stage(rec, config, segmenter_actual, play_segments, segmentation_ran)
216     _highlights(rec, play_segments, vm)
217     rec.finalize()
218     return rec

```

22. user (2026-06-10T06:02:17.808Z)

```

1     """Pydantic models for the application configuration.
2
3     This module defines the canonical, validated, typed configuration for the
4     Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6     The models support graceful migration from old config.json files via extra="allow".
7     """
8
9     from __future__ import annotations
10
11     from typing import Any, Literal
12
13     from pydantic import BaseModel, Field, field_validator
14
15     Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18     class ValidationRelevanceConfig(BaseModel):
19         court_green_lower: list[int] = Field(default=[35, 40, 40])
20         court_green_upper: list[int] = Field(default=[85, 255, 255])
21         court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24     class ValidationConfig(BaseModel):
25         min_resolution_width: int = 1280
26         min_resolution_height: int = 720
27         min_fps: float = 29.9
28         min_blur_threshold: float = 20.0
29         max_scene_cuts_per_minute: float = 0.5

```

```

30         relevance: ValidationRelevanceConfig =
Field(default_factory=ValidationRelevanceConfig)
31
32
33     class TrackNetConfig(BaseModel):
34         wsl_distro: str = "Ubuntu"
35         repo_dir: str = "~/models/TrackNetV4"
36         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37         conda_env: str = "TrackNetV4"
38         weights_path: str = ""
39         queue_length: int = 5
40         stage_in_wsl: bool = True
41         wsl_stage_dir: str = "~/clips"
42         timeout_sec: int = 0
43         velocity_threshold: float = 0.02
44
45
46     class IndexingConfig(BaseModel):
47         default_segmenter: str = "yolo_hybrid"
48         use_gpu: bool = True
49         motion_threshold: float = 0.08
50         min_segment_duration: float = 3.0
51         dead_time_merge_gap: float = 2.0
52         chunk_duration: float = 90.0
53         chunk_overlap: float = 10.0
54         detector_model: str = "fasterrcnn_resnet50_fpn"
55         detector_score_threshold: float = 0.5
56         yolo_velocity_threshold: float = 0.15
57         yolo_frame_skip: int = 3
58         yolo_tracker: str = "bytetrack.yaml"
59         yolo_parallel_chunks: bool = False
60         yolo_chunk_workers: int = 2
61         min_action_window_duration: float = 2.0
62         log_yolo_candidates: bool = True
63         store_yolo_candidates: bool = True
64         ai_handoff_padding: float = 2.0
65         ai_max_workers: int = 5
66
67         tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
68
69         @field_validator("default_segmenter")
70         @classmethod
71         def segmenter_must_be_registered(cls, v: str) -> str:
72             # Registry check happens at runtime in main/segmenter selection.
73             # Here we just allow any string for forward compatibility.
74             return v
75
76
77     class OutputConfig(BaseModel):
78         default_highlights_name: str = "highlights.mp4"
79         db_name: str = "sports_indexer.db"
80         # Directory where the DB, highlight reels, and processed clips are written.
81         # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
82         output_dir: str = "output"
83
84
85     class StitchingConfig(BaseModel):
86         begin_padding: float = 0.5
87         end_padding: float = 0.5
88         use_cache: bool = False
89         min_shot_count: int = 1
90
91
92     class AppConfig(BaseModel):
93         """Root configuration object.

```

```

94
95     All runtime configuration should be accessed via an instance of this class
96     (or the loader) rather than raw dict lookups.
97     """
98
99     sport: Sport = "badminton"
100     ai_provider: str = "gemini"
101     ai_model: str = "gemini-2.5-flash"
102
103     validation: ValidationConfig = Field(default_factory=ValidationConfig)
104     indexing: IndexingConfig = Field(default_factory=IndexingConfig)
105     output: OutputConfig = Field(default_factory=OutputConfig)
106     stitching: StitchingConfig = Field(default_factory=StitchingConfig)
107
108     # Allow extra keys during transition so existing config.json files do not break.
109     model_config = {
110         "extra": "allow",
111         "validate_assignment": True,
112     }
113
114     # Temporary migration shim for legacy dict-style access (config.get("section",
dict
115     {}).get(...)).
116     # Will be removed once all call sites (validators, segmenters, etc.) move to typed
117     # attribute access / the typed config. A nested *section* is returned as a plain
118     # via model_dump, so legacy chained access keeps working ? returning the Pydantic
119     # section model instead breaks it (a model has no .get), which bit every validator
120     # real runs. Scalar leaves are returned as-is.
121     def get(self, key: str, default: Any = None) -> Any:
122         _MISSING = object()
123         val: Any = _MISSING
124         if key in type(self).model_fields or hasattr(self, key):
125             val = getattr(self, key, _MISSING)
126         if val is _MISSING:
127             # Legacy convenience: find a leaf key inside any nested section. Sections
128             # discovered dynamically from the model fields, so a future top-level
129             # (e.g. `audio`, `jobs`) is covered automatically without editing a
130             # hardcoded list.
131             for fname in type(self).model_fields:
132                 section = getattr(self, fname, None)
133                 if isinstance(section, BaseModel) and hasattr(section, key):
134                     val = getattr(section, key)
135                     break
136             if val is _MISSING:
137                 return default
138             if isinstance(val, BaseModel):
139                 return val.model_dump(mode="json")
140             return val
141
142     def __getitem__(self, key: str) -> Any:
143         val = self.get(key)
144         if val is None and key not in type(self).model_fields:
145             # Re-raise like a real dict for unknown top-level
146             raise KeyError(key)
147         return val
148
149     # Convenience type alias for callers that want to annotate
150     Config = AppConfig

```



```

1      """Configuration loader and saver.
2
3      Provides a single place to load validated AppConfig with built-in defaults
4      merged with optional file overrides (config.json).
5
6      This replaces scattered json.load + .get() patterns throughout the codebase.
7      """
8
9      from __future__ import annotations
10
11     import json
12     import logging
13     from pathlib import Path
14     from typing import Any
15
16     from .models import AppConfig
17
18     logger = logging.getLogger(__name__)
19
20     DEFAULT_CONFIG_PATH = Path("config.json")
21
22
23     def _get_built_in_defaults() -> dict[str, Any]:
24         """Return a plain dict of the current built-in defaults.
25
26         The AppConfig model is the single source of truth for defaults.
27         """
28         return AppConfig().model_dump(mode="json")
29
30
31     def load_config(path: str | Path | None = None) -> AppConfig:
32         """Load configuration with the following precedence:
33
34         1. Built-in defaults defined in the Pydantic models.
35         2. Values from the JSON file (if present and readable).
36         3. (Future) environment / CLI overrides.
37
38         Missing file or unreadable file ? returns a fully defaulted AppConfig.
39         Unknown keys in the file are tolerated (extra="allow" on the model).
40         """
41         cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
42
43         file_data: dict[str, Any] = {}
44         if cfg_path.exists():
45             try:
46                 with open(cfg_path, "r", encoding="utf-8") as f:
47                     file_data = json.load(f)
48             except Exception as exc:
49                 # Non-fatal: continue with defaults + whatever we could parse.
50                 # In production we might log here.
51                 logger.warning(f"Failed to read {cfg_path}: {exc}. Using defaults + partial
data.")
52
53         # Merge: start from defaults, overlay file data (Pydantic will validate/coerce).
54         merged = {**_get_built_in_defaults(), **file_data}
55
56         return AppConfig.model_validate(merged)
57
58
59     def save_default_config(path: str | Path | None = None) -> Path:
60         """Write a fresh default config.json.
61
62         Used by setup.py diagnostics and the --setup CLI path.
63         Overwrites if the file exists.
64         """

```

```

65     cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
66     cfg = AppConfig()
67     cfg_path.parent.mkdir(parents=True, exist_ok=True)
68     cfg_path.write_text(cfg.model_dump_json(indent=4), encoding="utf-8")
69     return cfg_path
70
71
72     # Backwards-compatible alias used by some older call sites during transition.
73     def get_default_config() -> dict[str, Any]:
74         """Return defaults as a plain dict (for transitional use only)."""
75         return _get_builtin_defaults()
76

```

24. user (2026-06-10T06:02:27.851Z)

```

1     from abc import ABC, abstractmethod
2     from pydantic import BaseModel
3     from typing import List, Dict, Any, Optional
4
5     class ValidationResult(BaseModel):
6         validator_name: str
7         passed: bool
8         message: str
9         details: Dict[str, Any] = {}
10
11     class VideoValidator(ABC):
12         @property
13         @abstractmethod
14         def name(self) -> str:
15             """Unique identifier for this validator."""
16             pass
17
18         @property
19         @abstractmethod
20         def description(self) -> str:
21             """Human-readable explanation of what this validator checks."""
22             pass
23
24         @abstractmethod
25         def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
26             """
27             Executes validation.
28             - video_path: Local path to the MP4 file.
29             - config: Dict parsed from config.json.
30             - context: Dynamic context dict shared across validators (stores shared values
like ffprobe metadata).
31             """
32             pass
33
34     class ValidationRegistry:
35         def __init__(self):
36             self._validators: List[VideoValidator] = []
37
38         def register(self, validator: VideoValidator):
39             """Registers a new validator."""
40             self._validators.append(validator)
41
42         def run_all_with_context(self, video_path: str, config: Dict[str, Any]):
43             """Runs all validators and returns ``(results, context)``.
44
45             The per-run ``context`` accumulates shared values (notably
46             ``context['ffprobe_meta']`` produced by FormatValidator). ``run_all``
47             discards it; callers that want media metadata for an observability report
48             use this variant instead of re-probing.

```

```

49         """
50         results = []
51         context = {} # Shared context dictionary (e.g. for caching ffprobe outputs)
52         for val in self._validators:
53             try:
54                 res = val.validate(video_path, config, context)
55                 results.append(res)
56             except Exception as e:
57                 results.append(ValidationResult(
58                     validator_name=val.name,
59                     passed=False,
60                     message=f"Validator threw an unexpected error: {str(e)}"
61                 ))
62         return results, context
63
64     def run_all(self, video_path: str, config: Dict[str, Any]) ->
List[ValidationResult]:
65         """Runs all registered validators in sequence, accumulating results."""
66         results, _ = self.run_all_with_context(video_path, config)
67         return results
68
69     # Shared global registry
70     registry = ValidationRegistry()
71

```

25. user (2026-06-10T06:02:28.390Z)

```

1     import subprocess
2     import os
3     import shutil
4     import logging
5
6     logger = logging.getLogger(__name__)
7
8     def get_video_duration(video_path: str) -> float:
9         """Returns the exact duration of a video file in seconds using ffprobe.
10         Returns 0.0 on failure."""
11         try:
12             duration_output = subprocess.check_output([
13                 "ffprobe", "-v", "error", "-show_entries", "format=duration",
14                 "-of", "default=noprint_wrappers=1:nokey=1", video_path
15             ]).decode().strip()
16             return float(duration_output)
17         except Exception as e:
18             logger.error(f"Could not detect video duration for {video_path}: {e}")
19             return 0.0
20
21     def extract_video_chunk(video_path: str, start_time: float, duration: float,
22                             output_path: str, reencode: bool = False,
23                             scale_width: int = None) -> bool:
24         """Extracts a chunk from a video file.
25
26         Args:
27             reencode: If True, re-encodes with libx264 (slower but frame-accurate).
28                       If False, uses stream-copy (fast but cuts at nearest keyframe).
29                       Use True for short clips where keyframe drift matters.
30             scale_width: If set (requires reencode), downscale to this width (height auto,
31                          even). Useful for high-res sources headed to an API that analyses at
32                          low res anyway ? shrinks uploads with no analysis loss.
33
34         """
35         cmd = [
36             "ffmpeg", "-y",
37             "-ss", str(start_time),
38             "-t", str(duration),
39             "-i", video_path,

```

```

39     ]
40     if reencode:
41         if scale_width:
42             cmd += ["-vf", f"scale={int(scale_width)}:-2"]
43             cmd += ["-c:v", "libx264", "-preset", "ultrafast", "-crf", "23", "-c:a", "aac"]
44         else:
45             cmd += ["-c", "copy"]
46         cmd.append(output_path)
47         try:
48             subprocess.run(cmd, check=True, stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL)
49             return True
50         except Exception as e:
51             logger.error(f"Failed to extract chunk {start_time}s - {start_time+duration}s:
{e}")
52             return False
53

```

26. user (2026-06-10T06:02:36.106Z)

```

1     from abc import ABC, abstractmethod
2     from typing import Tuple
3
4     class AIVideoProvider(ABC):
5         def __init__(self, api_key: str, model_name: str):
6             self.api_key = api_key
7             self.model_name = model_name
8
9         @abstractmethod
10        def analyze_video(self, video_path: str, prompt: str,
11                        chunk_duration: float = 0.0,
12                        label: str = "") -> Tuple[list, dict]:
13            """Sends a video file to the AI model along with a prompt, and returns a tuple
of
14            (segments_list, metrics_dict).
15
16            Args:
17                video_path: Path to the video file to analyze.
18                prompt: Prompt instructions.
19                chunk_duration: Duration of this chunk in seconds, or 0.0 if not chunked.
20                label: Diagnostic label for logging.
21
22            Returns:
23                Tuple containing:
24                - A list of detected segment dicts (e.g. [{"start_time": ..., "end_time":
...}])
25                - A dictionary of performance metrics (e.g. {"upload_time": ...,
"process_time": ..., "gen_time": ...})
26            """
27            pass
28

```

27. user (2026-06-10T06:02:36.585Z)

```

1     """Canonical file-identity hashing for the indexer.
2
3     `video_id` is the MD5 of the *whole* video file. Historically each endpoint
4     (`backend/main.py`, `run_eval.py`, `run_phase3_eval.py`, `run_process_and_stitch.py`)
5     defined its own `get_file_md5` with a different read-chunk size (64KB / 8KB / 1MB).
6     The hash values agreed (same bytes hashed), but the duplicated code was a footgun:
7     any future change to one copy would silently diverge the DB key.
8
9     This module is the single source of truth. Import `compute_video_id` everywhere a
10    video needs to be keyed.
11    """

```

```

12
13     from __future__ import annotations
14
15     import hashlib
16
17     # 64 KB matches the value used by the live pipeline (backend/main.py) before this
18     # was centralized. The chunk size does not affect the resulting digest ? it only
19     # bounds memory use while streaming large files.
20     _CHUNK_SIZE = 65536
21
22
23     def compute_video_id(filepath: str) -> str:
24         """Return the MD5 hex digest of the whole file ? the canonical video_id.
25
26         Streams the file in fixed-size chunks so multi-gigabyte recordings never load
27         into memory at once.
28         """
29         hasher = hashlib.md5()
30         with open(filepath, "rb") as f:
31             for chunk in iter(lambda: f.read(_CHUNK_SIZE), b''):
32                 hasher.update(chunk)
33         return hasher.hexdigest()
34

```

28. user (2026-06-10T06:02:37.013Z)

```

1     """Lightweight Result type for standardizing success vs. failure across the pipeline.
2
3     This is the start of the Error/Result Contract standardization (low-regret item #3).
4
5     The goal is to make it impossible to confuse "no data / no rallies" with "something
actually failed"
6     (e.g. missing API key, WSL healthcheck failure, provider error, etc.).
7
8     We use simple dataclasses so callers can use isinstance checks or structural pattern
matching.
9     This is intentionally small and incremental ? we start by applying it to the segmenter
10    and provider layers (the biggest sources of silent [] returns today).
11
12    The types are generic where it makes sense. Callers can still access .value on Success
13    or .message on Failure.
14
15    This module has no dependencies on the rest of the app.
16    """
17
18    from __future__ import annotations
19
20    from dataclasses import dataclass, field
21    from typing import Generic, TypeVar, List, Optional, Any, Union
22
23    T = TypeVar("T")
24
25
26    @dataclass
27    class Success(Generic[T]):
28        """Represents a successful outcome with an optional list of non-fatal warnings."""
29
30        value: T
31        warnings: List[str] = field(default_factory=list)
32
33        def is_success(self) -> bool:
34            return True
35
36        def unwrap(self) -> T:
37            """Return the value. Only safe to call if you already know it's a Success."""

```

```

38         return self.value
39
40
41 @dataclass
42 class Failure:
43     """Represents a failure with a human-readable message and optional details."""
44
45     message: str
46     details: Optional[dict[str, Any]] = None
47     is_recoverable: bool = False # e.g. missing API key (recoverable) vs internal crash
48
49     def is_success(self) -> bool:
50         return False
51
52     def unwrap(self) -> Any:
53         """Raise to make misuse obvious in code that assumes success."""
54         raise RuntimeError(f"Attempted to unwrap a Failure: {self.message}")
55
56
57 # Convenient alias for use in type hints: Result[list[dict], Failure]
58 # Result[ValueType] means Success[ValueType] | Failure
59 # (the Failure "error" type is fixed for this contract)
60 Result = Union[Success[T], Failure]
61

```

29. user (2026-06-10T06:02:46.667Z)

```

1     import os
2     import sys
3     import json
4     import logging
5     from dotenv import load_dotenv
6
7     load_dotenv()
8
9     # Configure root logger ? all backend modules use logging.getLogger(__name__)
10    logging.basicConfig(
11        level=logging.INFO,
12        format="%(asctime)s [%(name)s] %(levelname)s: %(message)s",
13        datefmt="%H:%M:%S",
14    )
15
16    from backend.infrastructure.database import Database
17    from backend.config import load_config
18    from backend.utils.hashing import compute_video_id
19    from backend.pipeline.validators.base import registry
20    from backend.pipeline.segmenters import segmenter_registry
21    from backend.pipeline.stitcher.compiler import VideoCompiler
22
23    def main():
24        video_path = r"C:\Users\avidu\Videos\Testing\KushagraYashAviFirstVideo.mp4"
25        output_dir = r"C:\Users\avidu\Videos\Testing"
26        output_filename = "Final_2_OutputOfKushagraYashAviFirstVideo.mp4"
27
28        print(f"Loading config...")
29        cfg = load_config() # typed AppConfig ? single source of defaults
30
31        print(f"Verifying files exist...")
32        if not os.path.exists(video_path):
33            print(f"Error: Video file not found at {video_path}")
34            return
35
36        os.makedirs(cfg.output.output_dir, exist_ok=True)
37        db_path = os.path.join(cfg.output.output_dir, cfg.output.db_name)
38        db = Database(db_path)

```

```

39
40     # Calculate MD5 of the video to uniquely identify it
41     import time
42     print("Calculating video MD5 (this might take a few seconds on large files)...",
flush=True)
43     t0 = time.time()
44     video_id = compute_video_id(video_path)
45     print(f"Calculated MD5: {video_id} (took {time.time()-t0:.1f}s)", flush=True)
46
47     segments = []
48     cached_video = db.get_video(video_id)
49     cache_hit = False
50
51     if cached_video and cached_video.get("status") == "processed":
52         segments = db.query_play_segments(video_id, {})
53         if len(segments) > 0:
54             print(f"\n[CACHE] Found cached indexed video in database (MD5: {video_id})
with {len(segments)} parsed play segments.", flush=True)
55             choice = input("Would you like to (1) Run stitching on the cached segments,
or (2) Reprocess the video with Gemini from scratch? [1/2]: ").strip()
56             if choice == '1':
57                 print("[CACHE] Using cached play segments. Skipping segmenter API
call.", flush=True)
58                 cache_hit = True
59             else:
60                 print("[CACHE] Ignoring cache...", flush=True)
61         else:
62             print(f"[CACHE] Video exists in DB but contains 0 play segments. Treating as
cache miss.", flush=True)
63
64     if not cache_hit:
65         print("[CACHE] Initializing clean indexing...", flush=True)
66         db.clear_video_data(video_id)
67
68     print("Bypassing OpenCV validations for performance on massive files...",
flush=True)
69     db.add_video(video_id, video_path, "processed", [])
70
71     print("Running segmenter & indexing...", flush=True)
72     seg_name = cfg.indexing.default_segmenter
73     segmenter_cls = segmenter_registry.get(seg_name)
74     if not segmenter_cls:
75         print(f"[FAIL] Segmenter '{seg_name}' not found.")
76         return
77     print(f"Using segmenter: {seg_name}", flush=True)
78     segmenter = segmenter_cls(db, cfg)
79     segments = segmenter.process_video(video_id, video_path)
80     print(f"Successfully indexed {len(segments)} play segments.", flush=True)
81
82     if not segments:
83         print("No play segments detected. Cannot compile highlights.")
84         return
85
86
87     end_padding = cfg.stitching.end_padding
88     begin_padding = cfg.stitching.begin_padding
89     min_shot_count = cfg.stitching.min_shot_count
90
91     if cache_hit:
92         print("\n--- Stitching Customizations ---")
93         try:
94             bpad_input = input(f"Enter begin_padding in seconds (default
{begin_padding}): ").strip()
95             if bpad_input:
96                 begin_padding = float(bpad_input)

```

```

97
98         pad_input = input(f"Enter end_padding in seconds (default {end_padding}):
99     ").strip()
100         if pad_input:
101             end_padding = float(pad_input)
102
103         shot_input = input(f"Enter min_shot_count to filter by (default
104 {min_shot_count}): ").strip()
105         if shot_input:
106             min_shot_count = int(shot_input)
107         except ValueError:
108             print("[WARNING] Invalid input detected, falling back to defaults from
109 config.json.")
110
111         print(f"\nCompiling highlights (with begin_padding={begin_padding}s,
112 end_padding={end_padding}s, min_shot_count={min_shot_count})...", flush=True)
113
114         # Filter segments where shot_count >= min_shot_count
115         time_pairs = []
116         skipped_count = 0
117         for s in segments:
118             meta = s.get("metadata", {})
119             if isinstance(meta, str):
120                 try:
121                     meta = json.loads(meta)
122                 except Exception:
123                     meta = {}
124             shot_count = meta.get("shot_count", 0)
125             if shot_count >= min_shot_count:
126                 time_pairs.append((s["start_time"], s["end_time"]))
127             else:
128                 skipped_count += 1
129
130         if skipped_count > 0:
131             print(f"[FILTER] Excluded {skipped_count} play segments with fewer than
132 {min_shot_count} shots.", flush=True)
133         print(f"[FILTER] Retaining {len(time_pairs)} play segments for stitching.",
134 flush=True)
135
136         if not time_pairs:
137             print("No play segments passed the filtering criteria. Cannot compile
138 highlights.")
139         return
140
141         compiler = VideoCompiler(output_dir, end_padding=end_padding,
142 begin_padding=begin_padding)
143         try:
144             out_path = compiler.compile_highlights(video_path, time_pairs, output_filename)
145             print(f"Stitched highlight video saved to: {out_path}", flush=True)
146         except Exception as e:
147             print(f"Error during stitching: {e}", flush=True)
148
149     if __name__ == "__main__":
150         main()
151

```

30. user (2026-06-10T06:02:47.019Z)

```

1     # Observability report spec for the badminton-highlight-indexer.
2     # The report ENGINE (config language, renderers) lives in the shared `obsreport`
3     # library; this file is the indexer-specific layout + footgun rules. Every
4     # actionable rule cites a README/FAQ section so a reader can self-debug.
5     name: indexer
6     title: Badminton Highlight Indexer ? Video Report
7

```



```

8      # Customer here = whoever receives the highlight reel.
9      customer_verdict:
10         pass: "? Your highlights are ready."
11         pass_with_warnings: "? Your highlights are ready. (A couple of minor notes above.)"
12         fail: "?? We couldn't finish generating highlights for this video. Please contact
support."
13
14     sections:
15         - { id: run,          title: "Run",          kind: identity,  audience: both }
16         - { id: video,       title: "Video",        kind: video_meta, audience: both }
17         - { id: highlights,  title: "Highlights",    kind: highlights, audience: customer
}
18         - { id: cust_notes, title: "Things to know", kind: flags,      audience: customer
}
19         - { id: stages,     title: "Processing stages", kind: stages,    audience:
technical }
20         - { id: findings,   title: "Findings & warnings", kind: flags,      audience:
technical }
21         - { id: verdict,    title: "Verdict",        kind: verdict,   audience: both }
22
23     rules:
24         # The classic silent failure: 0 segments because the AI provider no-op'd on a
25         # missing API key ? NOT because the video has no rallies.
26         - code: silent_provider_noop
27           severity: error
28           when:
29             - { path: "stages.ai_handoff.details.api_key_present", op: eq, value: false }
30             - { path: "stages.segmentation.metrics.segment_count", op: eq, value: 0 }
31         message: >-
32             0 segments AND no AI provider API key was set. The segmenter (see Processing
33             stages > segmentation > segmenter_actual) hands its candidate windows to the AI
34             provider, which silently did nothing without a key ? so this is a MISSING
35             CREDENTIAL, not 'the video has no rallies'. Fix: put GEMINI_API_KEY=<key> in a
36             .env file in the project root and re-run (see README#Q7).
37         faq_ref: "README#Q7"
38         customer_message: >-
39             We couldn't generate highlights for this video due to a configuration issue
40             on our side. Please contact support.
41
42         # API key WAS present but an AI segmenter still produced nothing ? likely a
43         # provider error / quota / WSL healthcheck issue rather than a truly empty video.
44         - code: ai_empty_vs_no_rallies
45           severity: warn
46           when:
47             - { path: "stages.ai_handoff.details.ai_based", op: eq, value: true }
48             - { path: "stages.ai_handoff.details.api_key_present", op: eq, value: true }
49             - { path: "stages.segmentation.metrics.segment_count", op: eq, value: 0 }
50         message: >-
51             An AI-based segmenter returned 0 segments even though the API key is set.
52             Empty can mean a provider error, exhausted quota, or (for WASB/TrackNet) a
53             failed WSL healthcheck ? these all return [] silently. Re-run with -u for
54             unbuffered logs and check the provider/WSL output before concluding 'no rallies'.
55         faq_ref: "README#Q7"
56
57         # fps was assumed rather than probed -> velocity thresholds + timing may be wrong.
58         - code: fps_fallback_used
59           severity: warn
60           when:
61             - { path: "video_meta.fps_source", op: eq, value: fallback }
62         message: >-
63             fps was NOT probed from the file ? a fallback default (shown as the fps value)
64             was used. Because rally timing and motion thresholds scale with fps, boundaries
65             and motion sensitivity may be miscalibrated. This usually means validation didn't
66             run (it probes the file) or ffprobe couldn't read r_frame_rate. Confirm the real
67             fps with ffprobe and re-run.

```

```

68     faq_ref: "README#Q14"
69
70 # Audio stream missing or disabled. Per VIDEO_RECORDING_GUIDELINES (and ADR-007)
71 # the shuttle "thwack" is a first-class rally cue for recall; recording with audio
72 # on (mic unobstructed, no aggressive noise suppression) is now a hard requirement
73 # for good results with the shuttle-tracking substrates.
74 - code: audio_disabled_or_missing
75   severity: warn
76   when:
77     - { path: "video_meta.audio_present", op: eq, value: false }
78   message: >-
79     No audio stream was detected in the input (or it could not be read). The shuttle
80     "thwack" impact sound is a strong cue that improves rally recall (especially for
81     the WASB/TrackNet detector paths). Record with audio enabled, keep the camera
82     mic unobstructed, and avoid heavy wind/noise-reduction filters that flatten the
83     transient. See VIDEO_RECORDING_GUIDELINES.md (the "Audio (REQUIRED)" row) and
84     AUDIO_RALLY_DETECTION_PLAN.md.
85   faq_ref: "VIDEO_RECORDING_GUIDELINES"
86   customer_message: >-
87     For best highlight results, record with audio on and the mic unobstructed.
88     The shuttle impact sound helps the detector find rallies. Re-record or
89     re-process after enabling audio (see VIDEO_RECORDING_GUIDELINES.md).
90
91 # The motion baseline fabricates its per-segment metadata/events ? they are not
92 # measured. Warn so nobody trusts them as ground truth.
93 - code: synthetic_motion_metadata
94   severity: warn
95   when:
96     - { path: "stages.segmentation.details.segmenter_actual", op: eq, value: motion }
97   message: >-
98     The 'motion' segmenter was used: its per-segment metadata (intensity, speed,
99     events, server/receiver) are PLACEHOLDER values (unmeasured) ? fabricated by the
100    heuristic, NOT derived from the video ? so a PASS with segments here does not mean
101    the metadata is trustworthy. Use yolo_hybrid/gemini for real shot metadata.
102    faq_ref: "README#Q10"
103
104 # The segmenter that ran differs from config's default -> you may be looking at a
105 # different detector than expected (config/setup/CLI defaults are known to diverge).
106 - code: segmenter_divergence
107   severity: info
108   when:
109     - { path: "stages.segmentation.details.matches_config_default", op: eq, value:
110 false }
111     - { path: "stages.segmentation.details.segmenter_explicitly_requested", op: eq,
112 value: false }
113   message: >-
114     The segmenter that ran differs from config.json's default_segmenter, and no
115     explicit --segmenter was given ? so a fallback default kicked in. The indexer's
116     defaults are known to diverge across config.json / setup.py / the CLI; confirm
117     you ran the detector you meant.
118   faq_ref: "README#Q7"
119
120 # Re-processing an already-indexed video replaced its prior segments (INSERT OR
121 # REPLACE cascades). Note it so a changed result isn't mistaken for instability.
122 - code: reingest_replaced_prior
123   severity: info
124   when:
125     - { path: "stages.segmentation.details.was_reingest", op: eq, value: true }
126   message: >-
127     This video was already in the database; re-processing REPLACED its previous
128     segments (videos are keyed by file MD5; re-ingest cascades a delete). Expected
129     when you intend to re-run; just know the earlier results are gone.
130   faq_ref: "README#Q11"
131
132 # A validation check failed -> segmentation was halted. Point the reader at the

```

```

131     # report's Validation section + the per-check FAQ.
132     - code: validation_failed
133       severity: error
134       when:
135         - { path: "stages.validation.status", op: eq, value: error }
136       message: >-
137         One or more validation checks failed, so indexing was halted. See the
138         Processing stages > validation block above: each failed check states the
139         reason (relevance -> README#Q2, fps/keyframes -> README#Q5).
140       faq_ref: "README#Q13"
141       customer_message: >-
142         This video couldn't be processed because it didn't pass our quality checks
143         (for example: too blurry, too low resolution, or not a badminton court).
144

```

31. user (2026-06-10T06:03:18.209Z)

```

1     from typing import Dict, Type
2     from backend.providers.base import AIVideoProvider
3     from backend.providers.gemini import GeminiProvider
4
5     class ProviderRegistry:
6         def __init__(self):
7             self._registry: Dict[str, Type[AIVideoProvider]] = {}
8             # Register Gemini provider
9             self.register("gemini", GeminiProvider)
10
11         def register(self, name: str, provider_cls: Type[AIVideoProvider]):
12             self._registry[name] = provider_cls
13
14         def get(self, name: str, api_key: str, model_name: str) -> AIVideoProvider:
15             if name not in self._registry:
16                 raise KeyError(f"AI Provider '{name}' is not registered in the
ProviderRegistry. Available: {list(self._registry.keys())}")
17             return self._registry[name](api_key=api_key, model_name=model_name)
18
19         def list_available(self) -> Dict[str, str]:
20             return {name: cls.__doc__ or f"{name.capitalize()} provider" for name, cls in
self._registry.items()}
21
22     provider_registry = ProviderRegistry()
23

```

32. user (2026-06-10T06:03:18.998Z)

```

fastapi>=0.111.0
uvicorn>=0.30.1
opencv-python>=4.9.0.80
numpy>=1.26.4
pydantic>=2.7.1
jinja2>=3.1.4
python-dotenv>=1.0.0
google-generativeai>=2.4.0
torch>=2.12.0
torchvision>=0.27.0
supervision>=0.21.0,<0.30.0

```

Per-video observability reports (SOFT dependency: the pipeline degrades to a no-op if absent). Shared, permissively-licensed report toolkit. Pin a tag once the private repo is published, e.g.:

obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3

For local development install it editable from a sibling checkout:

pip install -e ../sports-obsreport --no-build-isolation

```
total 227
drwxr-xr-x 1 avidu 197609    0 Jun 10 11:31 .
drwxr-xr-x 1 avidu 197609    0 Jun  7 12:37 ..
drwxr-xr-x 1 avidu 197609    0 Jun  7 14:02 .claude
-rw-r--r-- 1 avidu 197609 69632 Jun 10 11:31 .coverage
-rw-r--r-- 1 avidu 197609   88 May 30 14:08 .coveragerc
-rw-r--r-- 1 avidu 197609   55 May 19 22:43 .env
drwxr-xr-x 1 avidu 197609    0 Jun 10 11:31 .git
drwxr-xr-x 1 avidu 197609    0 Jun 10 11:31 .github
-rw-r--r-- 1 avidu 197609   739 May 30 14:08 .gitignore
drwxr-xr-x 1 avidu 197609    0 May 19 15:58 .pytest_cache
drwxr-xr-x 1 avidu 197609    0 Jun  8 07:11 .sixth
-rw-r--r-- 1 avidu 197609 2995 Jun  8 16:37 CLAUDE.md
-rw-r--r-- 1 avidu 197609 27457 Jun  8 16:37 README.md
-rw-r--r-- 1 avidu 197609 5173 Jun  8 07:11 TEST_RUN_SUMMARY.md
drwxr-xr-x 1 avidu 197609    0 Jun 10 11:07 __pycache__
drwxr-xr-x 1 avidu 197609    0 Jun  8 07:11 backend
-rw-r--r-- 1 avidu 197609 2057 May 30 19:26 config.json
drwxr-xr-x 1 avidu 197609    0 Jun 10 11:19 docs
drwxr-xr-x 1 avidu 197609    0 Jun 10 11:31 output
-rw-r--r-- 1 avidu 197609   240 May 30 14:08 pytest.ini
-rw-r--r-- 1 avidu 197609   629 Jun  7 14:14 requirements.txt
-rwxr-xr-x 1 avidu 197609 2055 Jun  7 14:52 run_all_tests.py
-rw-r--r-- 1 avidu 197609 5620 Jun  8 07:11 run_eval.py
-rw-r--r-- 1 avidu 197609 7919 Jun  8 07:11 run_phase3_eval.py
-rw-r--r-- 1 avidu 197609 5966 Jun  8 07:11 run_process_and_stitch.py
-rw-r--r-- 1 avidu 197609 3603 Jun  8 07:11 run_regression.py
drwxr-xr-x 1 avidu 197609    0 Jun  7 14:14 scratch
drwxr-xr-x 1 avidu 197609    0 Jun  7 06:52 scripts
-rw-r--r-- 1 avidu 197609 4579 Jun  7 14:14 setup.py
drwxr-xr-x 1 avidu 197609    0 Jun 10 11:31 tests
```

Byte-compiled / optimized / DLL files

```
__pycache__/
*.py[cod]
*$py.class
*.pt
```

Local environment secrets

```
.env
*.env
.env.*
```

C extensions

```
*.so
```

Distribution / packaging

```
.Python
build/
develop-eggs/
dist/
downloads/
eggs/
.eggs/
lib/
lib64/
parts/
sdist/
var/
wheels/
```

```
share/python-wheels/  
*.egg-info/  
config.json
```

33. user (2026-06-10T06:03:27.163Z)

```
1      {  
2          "sport": "badminton",  
3          "ai_provider": "gemini",  
4          "ai_model": "gemini-2.5-flash",  
5          "validation": {  
6              "min_resolution_width": 1280,  
7              "min_resolution_height": 720,  
8              "min_fps": 29.9,  
9              "min_blur_threshold": 20.0,  
10             "max_scene_cuts_per_minute": 0.5,  
11             "relevance": {  
12                 "court_green_lower": [  
13                     35,  
14                     40,  
15                     40  
16                 ],  
17                 "court_green_upper": [  
18                     85,  
19                     255,  
20                     255  
21                 ],  
22                 "court_pixels_min_ratio": 0.05  
23             }  
24         },  
25         "indexing": {  
26             "default_segmenter": "gemini",  
27             "use_gpu": true,  
28             "motion_threshold": 0.08,  
29             "min_segment_duration": 3.0,  
30             "dead_time_merge_gap": 2.0,  
31             "chunk_duration": 90,  
32             "chunk_overlap": 10,  
33             "detector_model": "fasterrcnn_resnet50_fpn",  
34             "detector_score_threshold": 0.5,  
35             "yolo_velocity_threshold": 0.15,  
36             "yolo_frame_skip": 3,  
37             "yolo_tracker": "bytetrack.yaml",  
38             "yolo_parallel_chunks": false,  
39             "yolo_chunk_workers": 2,  
40             "min_action_window_duration": 2.0,  
41             "log_yolo_candidates": true,  
42             "store_yolo_candidates": true,  
43             "ai_handoff_padding": 2.0,  
44             "ai_max_workers": 5,  
45             "tracknet": {  
46                 "wsl_distro": "Ubuntu",  
47                 "repo_dir": "~/models/TrackNetV4",  
48                 "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",  
49                 "conda_env": "TrackNetV4",  
50                 "weights_path": "",  
51                 "queue_length": 5,  
52                 "stage_in_wsl": true,  
53                 "wsl_stage_dir": "~/clips",  
54                 "timeout_sec": 0,  
55                 "velocity_threshold": 0.02  
56             }  
57         },  
58         "output": {  
59             "default_highlights_name": "highlights.mp4",
```

```

60         "db_name": "sports_indexer.db"
61     },
62     "stitching": {
63         "begin_padding": 0.5,
64         "end_padding": 0.5,
65         "use_cache": false,
66         "min_shot_count": 1
67     }
68 }

```

34. user (2026-06-10T06:03:27.518Z)

```

1     import cv2
2     import numpy as np
3     from backend.pipeline.validators.base import VideoValidator, ValidationResult
4     from typing import Dict, Any
5
6     class RelevanceValidator(VideoValidator):
7         @property
8         def name(self) -> str:
9             return "relevance_check"
10
11        @property
12        def description(self) -> str:
13            return "Checks if the video content is relevant to badminton by scanning for
14            court color palettes and geometric features."
15
16        def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
17        -> ValidationResult:
18            cap = cv2.VideoCapture(video_path)
19            if not cap.isOpened():
20                return ValidationResult(
21                    validator_name=self.name,
22                    passed=False,
23                    message="Failed to open video file for relevance validation."
24                )
25
26            total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
27            if total_frames <= 0:
28                cap.release()
29                return ValidationResult(
30                    validator_name=self.name,
31                    passed=False,
32                    message="Invalid frame count."
33                )
34
35            # Sample 10 frames across the video
36            num_samples = 10
37            step = max(1, total_frames // num_samples)
38
39            green_ratios = []
40
41            # Get sport profile config
42            sport_name = config.get("sport", "badminton")
43            from backend.sports import sport_registry
44            try:
45                sport_profile = sport_registry.get(sport_name)
46                sport_rel_cfg = sport_profile.get_relevance_config()
47            except KeyError:
48                sport_rel_cfg = {}
49
50            val_cfg = config.get("validation", {})
51            rel_cfg = val_cfg.get("relevance", {})

```

```

52         hsv_lower = np.array(rel_cfg.get("court_green_lower",
sport_rel_cfg.get("court_green_lower", [35, 40, 40])))
53         hsv_upper = np.array(rel_cfg.get("court_green_upper",
sport_rel_cfg.get("court_green_upper", [85, 255, 255])))
54         min_court_ratio = rel_cfg.get("court_pixels_min_ratio",
sport_rel_cfg.get("court_pixels_min_ratio", 0.05))
55
56         for i in range(num_samples):
57             frame_idx = i * step
58             cap.set(cv2.CAP_PROP_POS_FRAMES, frame_idx)
59             ret, frame = cap.read()
60             if not ret:
61                 continue
62
63             # Downscale frame for speed
64             h, w = frame.shape[:2]
65             small_frame = cv2.resize(frame, (320, 240))
66
67             # Convert to HSV color space
68             hsv = cv2.cvtColor(small_frame, cv2.COLOR_BGR2HSV)
69
70             # Mask court green pixels
71             mask = cv2.inRange(hsv, hsv_lower, hsv_upper)
72             green_pixels = cv2.countNonZero(mask)
73             total_pixels = mask.shape[0] * mask.shape[1]
74
75             ratio = green_pixels / total_pixels
76             green_ratios.append(ratio)
77
78         cap.release()
79
80         if not green_ratios:
81             return ValidationResult(
82                 validator_name=self.name,
83                 passed=False,
84                 message="Failed to extract frames for relevance checking."
85             )
86
87         avg_green_ratio = sum(green_ratios) / len(green_ratios)
88
89         details = {
90             "avg_green_court_ratio": round(avg_green_ratio, 4),
91             "max_green_court_ratio": round(max(green_ratios), 4),
92             "min_green_court_ratio": round(min(green_ratios), 4),
93             "green_threshold": min_court_ratio
94         }
95
96         # If a certain amount of court color is visible, consider it relevant
97         if avg_green_ratio < min_court_ratio:
98             return ValidationResult(
99                 validator_name=self.name,
100                 passed=False,
101                 message=f"Video rejected: Relevance check failed. The scene does not
contain a recognizable {sport_name} court (court area found: {round(avg_green_ratio * 100, 2)}%
vs min required {round(min_court_ratio * 100, 2)}%).",
102                 details=details
103             )
104
105         return ValidationResult(
106             validator_name=self.name,
107             passed=True,
108             message=f"Video identified as relevant to {sport_name} ({sport_name} court
structures found).",
109             details=details
110         )

```

35. user (2026-06-10T06:03:39.853Z)

"""CLI to evaluate detected rallies against hand-annotated ground truth.

Scores predictions already stored in the SQLite DB (run the pipeline first) ? no
API calls, fully deterministic. See docs/EVALUATION.md for the workflow.

Examples

Create an empty annotation file to fill in:

```
python run_eval.py --scaffold --annotations rallies.csv
```

Score a video's detected rallies (identify it by file or by stored video_id):

```
python run_eval.py --video "C:/path/match.mp4" --annotations rallies.csv
```

```
python run_eval.py --video-id <md5> --annotations rallies.csv --stage both --iou 0.5
```

Show which rallies were missed / spurious, and dump a JSON report:

```
python run_eval.py --video-id <md5> --annotations rallies.csv --show-failures --json  
report.json  
"""
```

```
import os  
import sys  
import json  
import argparse  
import logging
```

```
from backend.infrastructure.database import Database  
from backend.config import load_config  
from backend.utils.hashing import compute_video_id as get_file_md5 # canonical video_id  
from backend.eval.annotations import load_annotations, write_scaffold  
from backend.eval.harness import evaluate, format_report_text, report_to_dict, STAGES
```

```
def _default_db_path() -> str:  
    """Resolve the DB path from the typed config (output.output_dir / output.db_name)."""  
    cfg = load_config()  
    return os.path.join(cfg.output.output_dir, cfg.output.db_name)
```

```
_EPILOG = """\
```

Prerequisite:

The harness scores predictions read from the SQLite DB ? it does NOT run the pipeline. So you must first PROCESS the video (e.g. via run_process_and_stitch.py) so its detections land in output/sports_indexer.db. Then point this tool at the same video file (--video) or its stored MD5 (--video-id). If you get "database not found" or zero predictions, the video hasn't been processed yet.

See docs/EVALUATION.md for the annotation format and how to read the output.

```
"""
```

```
def build_parser() -> argparse.ArgumentParser:  
    p = argparse.ArgumentParser(  
        description="Evaluate detected rallies vs ground-truth annotations.",  
        epilog=_EPILOG,  
        formatter_class=argparse.RawDescriptionHelpFormatter,  
    )  
    p.add_argument("--annotations", help="Path to the ground-truth rally CSV (start,end per row).")  
    p.add_argument("--video", help="Path to the video file (its MD5 becomes the video_id).")  
    p.add_argument("--video-id", help="Stored video_id (MD5) to evaluate, instead of --video.")  
    p.add_argument("--db", default=None, help="SQLite DB path (default: from config.json).")
```



```

    p.add_argument("--stage", choices=("candidates", "final", "both"), default="both",
                    help="Which prediction stage(s) to score (default: both).")
    p.add_argument("--detector", default=None,
                    help="Only score candidates from this detector (e.g. yolo_hybrid,
tracknet_hybrid).")
    p.add_argument("--iou", type=float, default=0.5, help="IoU match threshold (default: 0.5).")
    p.add_argument("--pr-curve", action="store_true", help="Include an IoU sweep (0.1..0.9) in
the report.")
    p.add_argument("--show-failures", action="store_true",
                    help="List missed (FN) and spurious (FP) rallies with timestamps.")
    p.add_argument("--json", dest="json_out", default=None, help="Write a JSON report to this
path.")
    p.add_argument("--scaffold", action="store_true",
                    help="Write an empty annotation CSV to --annotations and exit.")
    return p

```

```

def main(argv=None) -> int:
    logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
    args = build_parser().parse_args(argv)

```

```

    # Scaffold mode: just write the template and exit.
    if args.scaffold:
        if not args.annotations:

```

```

=====

```

```

"""CLI: regression-test the production tool against golden-set ground truth.

```

```

Obtains ground truth from an annotation provider (a tier), scores the video's
stored predictions, and compares to a snapshotted baseline. Exit code 1 on a
detected regression so it can gate CI. See docs/GOLDEN_SET_IMPLEMENTATION.md.

```

Examples

```

-----

```

```

Snapshot the current numbers as the baseline (first run / after a real improvement):

```

```

    python run_regression.py --video-id <md5> --tier file --annotations rallies.csv --update-
baseline

```

```

Check for a regression (exit 1 if precision/recall dropped beyond tolerance):

```

```

    python run_regression.py --video-id <md5> --tier file --annotations rallies.csv

```

```

"""

```

```

import os
import sys
import json
import argparse
import logging

```

```

from backend.infrastructure.database import Database
from backend.config import load_config
from backend.annotations import annotation_registry
from backend.eval import regression

```

```

def _default_db_path() -> str:
    cfg = load_config()
    return os.path.join(cfg.output.output_dir, cfg.output.db_name)

```

```

def build_parser() -> argparse.ArgumentParser:
    p = argparse.ArgumentParser(description="Regression-test the tool vs golden-set ground
truth.")
    p.add_argument("--video-id", required=True, help="Stored video_id (MD5) whose predictions to
score.")
    p.add_argument("--tier", default="file",
                    help=f"Annotation provider tier. Available:
{annotation_registry.list_available()}")

```

```

    p.add_argument("--annotations", help="video_ref for the provider (e.g. CSV path for the
'file' tier).")
    p.add_argument("--stage", choices=("candidates", "final"), default="final")
    p.add_argument("--detector", default=None, help="Restrict candidate scoring to one
detector.")
    p.add_argument("--iou", type=float, default=0.5)
    p.add_argument("--tolerance", type=float, default=regression.DEFAULT_TOLERANCE)
    p.add_argument("--db", default=None)
    p.add_argument("--baseline", default=regression.DEFAULT_BASELINE_PATH, help="Baseline JSON
path.")
    p.add_argument("--update-baseline", action="store_true",
                    help="Snapshot the current numbers as the new baseline instead of checking.")
    p.add_argument("--json", dest="json_out", default=None)
    return p

```

```

def main(argv=None) -> int:
    logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
    args = build_parser().parse_args(argv)

    db_path = args.db or _default_db_path()
    if not os.path.exists(db_path):
        print(f"ERROR: database not found at {db_path}. Run the pipeline first.",
file=sys.stderr)
        return 2
    db = Database(db_path)

```

```

    result = regression.regress_with_provider(
=====
#!/usr/bin/env python
"""Run the FULL test suite for the badminton-highlight-indexer.

```

A single, discoverable entry point so you (or a reviewer / CI) can verify the whole suite before opening a PR:

```

python run_all_tests.py          # full suite, quiet
python run_all_tests.py -v       # verbose (passthrough to pytest)
python run_all_tests.py -k config # filter by keyword (passthrough)
python run_all_tests.py --cov=backend --cov-report=term-missing # coverage

```

Anything after the script name is passed straight through to pytest.

Notes

```

-----
- The suite is fully mocked (no GPU, no network, no WSL) and runs offline.
- The observability **reporting** tests skip cleanly if the optional ``obsreport``
  package isn't installed (they use ``pytest.importorskip``). To exercise them too:
  pip install -e ../sports-obsreport --no-build-isolation
- Exit code is pytest's own, so this is CI / pre-PR friendly (0 = all green).
"""

```

```

from __future__ import annotations

```

```

import importlib.util
import subprocess
import sys

```

```

def _installed(module: str) -> bool:
    try:
        return importlib.util.find_spec(module) is not None
    except (ImportError, ValueError):
        return False

```

```

def main() -> int:

```

```

args = list(sys.argv[1:])
# Default to quiet unless the caller chose a verbosity.
if not any(a in ("-q", "-v", "-vv", "-vvv", "--quiet", "--verbose") for a in args):
    args = ["-q", *args]

```

36. user (2026-06-10T06:03:41.304Z)

```

6:> Status legend: ? blocks something . ? do when triggered . ? nice-to-have/cleanup.
30:- ? **Annotation guideline** ? **DONE (2026-06-02):** authored as the Roora brief +
illustrated guide in the collector repo `docs/annotation/` ? rally CSV schema; ignore dead time
/ include every rally (incl. faults & lets) / shot = one contact; `ending_reason`
(forced/unforced-error taxonomy, shared across net-separated sports); singles & doubles for
badminton & pickleball.
31:- ? **Internal gold key** ? hand-annotate 5?10 licensed clips to near-perfect as the vendor
answer key. **Unblocked.** **Revisit:** anytime.
36:- ? **Phase-1 video collection** ? gather the round-1 golden corpus per
`docs/GOLDEN_SET_PHASE1_VIDEOS.md`: **static self-recorded** footage (mirror deployment, not
broadcast), 3 sports incl. singles & doubles, across the diversity matrix; ~12 clips/sport with
**? held out by match**. **Unblocked.**
59:- ? **Docs-only checkpoint PR** ? This update + the refreshed `docs/LOW_REGRET_MOVES_PLAN.md`
formally record the approved path before any code changes. The initial review docs (PR #37)
introduced the findings; this entry locks the execution decisions.

```

37. user (2026-06-10T06:03:54.685Z)

```

backend\eval\gemini_refine.py:17:Run (GEMINI_API_KEY in env):
backend\eval\gemini_refine.py:71:def refine_window(api_key: str, model: str, sport_profile,
video: str, window: Interval,
backend\eval\gemini_refine.py:76:     provider = provider_registry.get("gemini", api_key=api_key,
model_name=model)
backend\eval\gemini_refine.py:105:     api_key = os.environ.get("GEMINI_API_KEY")
backend\eval\gemini_refine.py:106:     if not api_key:
backend\eval\gemini_refine.py:107:         raise SystemExit("GEMINI_API_KEY not set in env")
backend\eval\gemini_refine.py:122:     futs = [ex.submit(refine_window, api_key, model,
sport_profile, video, w, pad, duration,
backend\eval\gemini_refine.py:156:     api_key = os.environ.get("GEMINI_API_KEY")
backend\eval\gemini_refine.py:157:     if not api_key:
backend\eval\gemini_refine.py:158:         raise SystemExit("GEMINI_API_KEY not set in env")
backend\eval\gemini_refine.py:174:     provider = provider_registry.get("gemini",
api_key=api_key, model_name=model)

```

38. user (2026-06-10T06:03:55.456Z)

```

__init__.py
__pycache__
test_annotation_providers.py
test_automated_provider.py
test_base.py
test_calibrate_local.py
test_calibrate_wasb.py
test_calibration.py
test_config.py
test_database.py
test_distill_local.py
test_eval_annotations.py
test_eval_batch.py
test_eval_classifier.py
test_eval_harness.py
test_eval_metrics.py
test_eval_partial_labels.py
test_eval_regression.py
test_eval_training.py
test_export_wasb_segments.py
test_gemini.py

```

```

test_gemini_refine.py
test_geometry.py
test_hashing.py
test_hit_detector.py
test_import_smoke.py
test_provider_gemini.py
test_providers.py
test_rally_gate.py
test_rally_seq_proto.py
test_rally_slicer.py
test_reporting.py
test_reporting_harvest.py
test_reporting_spec_faq.py
test_sport_profiles.py
test_tracknet_hybrid.py
test_tracknet_runner.py
test_validators_base.py
test_video.py
test_wasb_cache.py
test_yolo_hybrid.py
    171 tests/test_wasb_cache.py
    320 tests/test_yolo_hybrid.py
    3622 total

```

39. user (2026-06-10T06:04:03.887Z)

```
53:     """predict_fn(cfg) -> rally windows, from a cached WASB trajectory (parsed once)."""
```

40. user (2026-06-10T06:04:08.625Z)

```

1     """Resumable-cache layer of wasb_infer (pure-Python parts; no WSL/GPU).
2
3     Covers the reboot-resume guarantees: per-window incremental cache, cross-window
4     detection accumulation on replay, crash-truncation recovery, manifest
5     compatibility/invalidation, and atomic trajectory writes.
6     """
7     import json
8     import os
9     import os.path as osp
10
11     import numpy as np
12
13     from backend.pipeline.detectors import wasb_infer as wi
14
15
16     def _win(idx, frames):
17         """A window record: frames = [(fid, [[x,y,score,scale], ...]), ...]."""
18         return {"w": idx, "f": [[fid, dets] for fid, dets in frames]}
19
20
21     def _write_windows(cache_dir, windows):
22         det_jsonl, _ = wi._cache_paths(cache_dir)
23         with open(det_jsonl, "a") as f:
24             wi._append_windows(f, windows)
25
26
27     # ----- #
28     # detection (de)serialization
29     # ----- #
30     def test_det_plain_and_back_roundtrip():
31         det = {"xy": np.array([439.0, 64.5]), "score": np.float32(0.87), "scale": 0}
32         plain = wi._det_plain(det)
33         assert plain == [439.0, 64.5, float(np.float32(0.87)), 0]
34
35         back = wi._dets_to_tracker([plain])

```

```

36     assert len(back) == 1
37     assert isinstance(back[0]["xy"], np.ndarray)          # tracker does vector math on xy
38     assert back[0]["xy"].tolist() == [439.0, 64.5]
39     assert back[0]["scale"] == 0
40
41
42     # ----- #
43     # cache replay + resume index
44     # ----- #
45     def test_replay_accumulates_detections_across_windows(tmp_path):
46         cache = str(tmp_path / "c")
47         os.makedirs(cache)
48         # frame 1 appears in BOTH windows -> its candidates must accumulate in window order;
49         # frame 2 is empty in window 0 but detected in window 1.
50         _write_windows(cache, [
51             _win(0, [(0, [[10, 20, 0.9, 0]]), (1, [[11, 21, 0.8, 0]]), (2, [])]),
52             _win(1, [(1, [[12, 22, 0.7, 0]]), (2, [[13, 23, 0.6, 0]]), (3, [])]),
53         ])
54         windows_done, det = wi.load_and_clean_cache(cache)
55         assert windows_done == 2
56         assert det[0] == [[10, 20, 0.9, 0]]
57         assert det[1] == [[11, 21, 0.8, 0], [12, 22, 0.7, 0]] # window-0 then window-1
58         assert det[2] == [[13, 23, 0.6, 0]]
59         assert det[3] == [] # seen but never detected
60
61
62     def test_resume_index_equals_window_count(tmp_path):
63         cache = str(tmp_path / "c")
64         os.makedirs(cache)
65         _write_windows(cache, [_win(i, [(i, [[i, i, 0.5, 0]])]) for i in range(5)])
66         windows_done, _ = wi.load_and_clean_cache(cache)
67         assert windows_done == 5
68
69
70     def test_empty_cache(tmp_path):
71         cache = str(tmp_path / "c")
72         os.makedirs(cache)
73         windows_done, det = wi.load_and_clean_cache(cache)
74         assert windows_done == 0 and dict(det) == {}
75
76
77     # ----- #
78     # crash robustness
79     # ----- #
80     def test_truncated_trailing_line_recovered_and_cleaned(tmp_path):
81         cache = str(tmp_path / "c")
82         os.makedirs(cache)
83         _write_windows(cache, [_win(0, [(0, [[1, 1, 0.9, 0]])]),
84                                 _win(1, [(1, [[2, 2, 0.8, 0]])])])
85         det_jsonl, _ = wi._cache_paths(cache)
86         with open(det_jsonl, "a") as f: # simulate a crash mid-flush
87             f.write('{ "w":2, "f":[[2, [[3, 3, 0.7, 0]]]]')
88         windows_done, det = wi.load_and_clean_cache(cache)
89         assert windows_done == 2 # only the two clean windows survive
90
91         assert det[1] == [[2, 2, 0.8, 0]]
92         # file is rewritten to the clean prefix -> appending window 2 stays contiguous
93         with open(det_jsonl) as f:
94             lines = [json.loads(l) for l in f if l.strip()]
95         assert [o["w"] for o in lines] == [0, 1]
96         _write_windows(cache, [_win(2, [(2, [[3, 3, 0.7, 0]])])])
97         assert wi.load_and_clean_cache(cache)[0] == 3
98
99     def test_noncontiguous_window_index_stops_replay(tmp_path):

```

```

100     cache = str(tmp_path / "c")
101     os.makedirs(cache)
102     _write_windows(cache, [_win(0, [(0, [[1, 1, 0.9, 0]])]),
103                          _win(5, [(5, [[2, 2, 0.8, 0]])])) # gap -> stop after w=0
104     windows_done, det = wi.load_and_clean_cache(cache)
105     assert windows_done == 1
106     assert 5 not in det
107
108
109     # ----- #
110     # manifest: atomic write + compatibility / invalidation
111     # ----- #
112     def test_manifest_roundtrip_and_compat(tmp_path):
113         cache = str(tmp_path / "c")
114         os.makedirs(cache)
115         m = {
116             "version": wi.CACHE_VERSION,
117             "frames_dir": osp.abspath("/x/frames"),
118             "weights": "w.pth.tar",
119             "sport": "badminton",
120             "frames_in": 3,
121             "frames_total": 100,
122             "windows_total": 98,
123             "windows_done": 42,
124         }
125         wi.write_manifest(cache, m)
126         assert not osp.exists(osp.join(cache, wi._MANIFEST_FILE + ".tmp")) # atomic, no
tmp left
127         assert wi.read_manifest(cache) == m
128
129         base = dict(frames_dir="/x/frames", weights="w.pth.tar", sport="badminton",
130                     frames_in=3, frames_total=100)
131         assert wi.manifest_compatible(m, **base)
132         assert not wi.manifest_compatible(m, **{**base, "frames_total": 101}) # more
frames
133         assert not wi.manifest_compatible(m, **{**base, "weights": "other.pth"})
134         assert not wi.manifest_compatible(m, **{**base, "sport": "tennis"})
135         assert not wi.manifest_compatible({}, **base) # no cache
136         assert not wi.manifest_compatible(**m, "version": 999, **base) # schema bump
137
138
139     def test_read_manifest_missing_returns_none(tmp_path):
140         assert wi.read_manifest(str(tmp_path)) is None
141
142
143     def test_reset_cache_removes_files(tmp_path):
144         cache = str(tmp_path / "c")
145         os.makedirs(cache)
146         _write_windows(cache, [_win(0, [(0, [[1, 1, 0.9, 0]])]),
147                              _win(5, [(5, [[2, 2, 0.8, 0]])]))
148         wi.write_manifest(cache, {"version": wi.CACHE_VERSION})
149         wi.reset_cache(cache)
150         det_jsonl, mpath = wi._cache_paths(cache)
151         assert not osp.exists(det_jsonl) and not osp.exists(mpath)
152
153     # ----- #
154     # misc helpers
155     # ----- #
156     def test_default_cache_dir_is_next_to_output():
157         out = osp.join("some", "dir", "sample_long_traj.csv")
158         cd = wi.default_cache_dir(out)
159         assert osp.basename(cd) == "sample_long_traj_wasbcache"
160         assert osp.dirname(cd) == osp.dirname(osp.abspath(out))
161
162

```

```

163     def test_atomic_write_trajectory(tmp_path):
164         out = str(tmp_path / "out" / "traj.csv")
165         wi._atomic_write_trajectory(out, [(0, 1, 439.0, 64.0), (1, 0, -1, -1)])
166         assert not osp.exists(out + ".tmp")
167         with open(out) as f:
168             text = f.read().splitlines()
169         assert text[0] == "Frame,Visibility,X,Y"
170         assert text[1] == "0,1,439.0,64.0"
171         assert text[2] == "1,0,-1,-1"
172

```

41. user (2026-06-10T06:04:35.971Z)

20:[Omitted long matching line]
248:Videos are keyed by the ****MD5 of the file's bytes****. Re-processing the **same** file is treated as a fresh ingest: ``add_video`` is ``INSERT OR REPLACE``, and the foreign-key cascade ****deletes the previous play/candidate segments**** before writing the new ones. So the earlier results are intentionally replaced, not merged. (A ``reingest_replaced_prior`` note in the report flags this.) Note also that re-encoding or re-downloading a video changes its bytes ? a new MD5 ? a separate record.

42. user (2026-06-10T06:04:37.204Z)

173:[Omitted long matching line]
192: `{id, video_id, detector:str, chunk_index, start_time:REAL, end_time:REAL, duration:REAL, confidence:REAL(min(1.0,peak_velocity)), stats:JSON{peak_velocity,mean_velocity,active_frame_count,track_id_count}, detection_params:JSON{...}, confirmed_segment_id, human_label, notes, created_at}`
(stats/detection_params deserialized on read).
250:- ****`add\_video` INSERT OR REPLACE + FK CASCADE**** -> re-ingesting a ``video_id`` cascade-deletes its segments. Use ``clear_video_data`` deliberately. ``_get_connection`` MUST re-issue ``PRAGMA foreign_keys=ON`` per connection.

43. assistant (2026-06-10T06:04:45.206Z)

Let me run the offline test suite to verify current state, and check the wasb_infer cache-healing implementation header.

44. user (2026-06-10T06:05:19.007Z)

```

=====
Full test suite: C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe -m pytest
-q
obsreport (optional reporting dep): available
=====
..... [ 28%]
..... [ 56%]
..... [ 84%]
..... [100%]
256 passed in 28.88s
=====
RESULT: PASS - all tests green
=====

```

45. user (2026-06-10T06:05:19.029Z)

```

1     """WASB shuttle-trajectory inference on an arbitrary frame sequence (runs in WSL).
2
3     WASB-SBDT only ships *dataset* evaluation; this is a thin wrapper that reuses its
4     own building blocks (ImageDataset transform + detector + online tracker) to run on
5     any folder of extracted frames and emit a trajectory CSV in the exact shape the
6     indexer's `tracknet_runner.parse_trajectory_csv` already consumes:
7
8     Frame,Visibility,X,Y

```

```

9
10 It must run INSIDE the WSL `wasb` conda env, from the WASB-SBDT `src/` dir (it
11 imports WASB's `detectors`/`trackers`/`dataloaders`). The Windows-side adapter
12 (`wasb_runner.py`) stages this file + the frames into WSL and invokes it.
13
14 Resumable across reboots
15 -----
16 The GPU detector pass (one forward per sliding window ? ~21k for a 6-min clip) is
17 the slow, expensive part; the CPU tracker pass that turns detections into a
18 trajectory is cheap and *stateful* (it must see every frame in order, so it can't
19 resume mid-stream). We exploit that split:
20
21 * The detector pass writes its per-window raw outputs incrementally to
22 ``<cache>/det_raw.jsonl`` (flushed+fsync'd every ``--flush-every`` windows) and
23 records progress in ``<cache>/manifest.json`` (atomic write). A reboot loses at
24 most ``--flush-every`` windows of GPU work instead of the whole pass.
25 * On restart we replay the cached windows, skip the DataLoader ahead to the first
26 un-cached window, and continue. Once every window is cached, the detector pass
27 is skipped entirely and we just re-run the (cheap, deterministic) tracker over
28 the full ordered cache -> trajectory CSV. So a lost trajectory CSV costs seconds,
29 not the GPU hour.
30
31 The cache is keyed by the output path (``<out>_wasbcache/`` by default), lives next
32 to the output (NOT in %TEMP%), and is invalidated automatically if the frames dir,
33 weights, sport, window size, or frame count change.
34
35 Usage (in WSL, from ~/models/WASB-SBDT/src):
36 python wasb_infer.py --frames_dir /path/frames --weights
37 ../pretrained_weights/wasb_badminton_best.pth.tar \
38     --out /path/traj.csv [--sport badminton] [--limit N] [--cache-dir DIR] [--flush-
39 every 1000] [--fresh]
40 """
41 import os
42 import os.path as osp
43 import csv
44 import glob
45 import json
46 import argparse
47 import logging
48 from collections import defaultdict
49
50 logger = logging.getLogger(__name__)
51
52 CACHE_VERSION = 1
53 _DET_FILE = "det_raw.jsonl"
54 _MANIFEST_FILE = "manifest.json"
55
56 def _build_cfg(weights: str, sport: str):
57     """Compose the WASB eval config via Hydra, overriding model/weights/device."""
58     from hydra import compose, initialize
59     with initialize(config_path="configs", version_base=None):
60         cfg = compose(config_name="eval", overrides=[
61             f"dataset={sport}",
62             "model=wasb",
63             f"detector.model_path={weights}",
64             "runner.device=cuda",
65             "runner.gpus=[0]", # this box has a single GPU; default config assumes
66             ])
67     return cfg
68
69 def _frame_id(path: str) -> int:
70     """Best-effort integer frame id from a frame filename (e.g. frame_000180.png ->

```



```

180)."""
71     stem = osp.splitext(osp.basename(path))[0]
72     digits = "".join(ch for ch in stem if ch.isdigit())
73     return int(digits) if digits else -1
74
75
76     # ----- #
77     # Resumable detector cache (pure-Python, no torch/numpy ? unit-testable)
78     # ----- #
79     def _cache_paths(cache_dir: str):
80         return osp.join(cache_dir, _DET_FILE), osp.join(cache_dir, _MANIFEST_FILE)
81
82
83     def default_cache_dir(out_csv: str) -> str:
84         """Stable cache dir next to the trajectory output (survives reboots; not %TEMP%)."""
85         d = osp.dirname(osp.abspath(out_csv))
86         stem = osp.splitext(osp.basename(out_csv))[0]
87         return osp.join(d, stem + "_wasbcache")
88
89
90     def _atomic_write_text(path: str, text: str) -> None:
91         """Write then rename so a crash mid-write can't leave a half-written file."""
92         tmp = path + ".tmp"
93         with open(tmp, "w") as f:
94             f.write(text)
95             f.flush()
96             os.fsync(f.fileno())
97         os.replace(tmp, path)
98
99
100    def _det_plain(det) -> list:
101        """A detector candidate dict {'xy': np.array([x,y]), 'score', 'scale'} -> JSON-able
list."""
102        xy = det["xy"]
103        return [float(xy[0]), float(xy[1]), float(det["score"]), int(det["scale"])]
104
105
106    def _dets_to_tracker(plain_list):
107        """Inverse of _det_plain: rebuild the dicts the tracker expects (xy MUST be a numpy
array,
108        the tracker does vector math / np.linalg.norm on it)."""
109        import numpy as np
110        return [{"xy": np.array([p[0], p[1]], dtype=float), "score": p[2], "scale": p[3]}
111                for p in plain_list]
112
113
114    def write_manifest(cache_dir: str, manifest: dict) -> None:
115        _, mpath = _cache_paths(cache_dir)
116        _atomic_write_text(mpath, json.dumps(manifest, indent=2))
117
118
119    def read_manifest(cache_dir: str):
120        _, mpath = _cache_paths(cache_dir)

```

46. user (2026-06-10T06:07:56.418Z)

```

49:@app.get("/", response_class=HTMLResponse)
57:# Global variables initialized at startup
87:@app.get("/api/config")
91:@app.post("/api/process")
172:@app.post("/api/query")
183:@app.post("/api/compile")

```

47. user (2026-06-10T06:07:57.801Z)

```

backend\pipeline\segmenters\motion.py:22:     def process_video(self, video_id: str, video_path:
str, player_pool: List[str] = None, max_frames: int = None) -> List[Dict[str, Any]]:
backend\pipeline\segmenters\motion.py:33:         return Failure(message=f"Could not open
video: {video_path}", is_recoverable=False)
backend\pipeline\segmenters\motion.py:226:         return Success(value=segments_data)
backend\pipeline\segmenters\yolo_hybrid.py:102:     def _detect_and_track(self, frame, tracker,
score_thresh: float) -> List[Tuple[int, Tuple[float, float, float]]]:
backend\pipeline\segmenters\yolo_hybrid.py:127:         return []
backend\pipeline\segmenters\yolo_hybrid.py:146:
chunk_index: int = None) -> List[Dict[str, Any]]:
backend\pipeline\segmenters\yolo_hybrid.py:163:         return []
backend\pipeline\segmenters\yolo_hybrid.py:230:         return []
backend\pipeline\segmenters\yolo_hybrid.py:263:     def process_video(self, video_id: str,
video_path: str, player_pool: List[str] = None, max_frames: int = None) -> List[Dict[str, Any]]:
backend\pipeline\segmenters\yolo_hybrid.py:272:         return []
backend\pipeline\segmenters\yolo_hybrid.py:288:         return []
backend\pipeline\segmenters\yolo_hybrid.py:294:         return []
backend\pipeline\segmenters\yolo_hybrid.py:301:         return []
backend\pipeline\segmenters\yolo_hybrid.py:485:     def process_candidate_window(wi: int,
window: Dict[str, Any]) -> List[dict]:
backend\pipeline\segmenters\yolo_hybrid.py:494:         return []
backend\pipeline\segmenters\base.py:24:     def process_video(self, video_id: str, video_path:
str, player_pool: List[str] = None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
backend\pipeline\segmenters\gemini.py:25:     def process_video(self, video_id: str, video_path:
str, player_pool: List[str] = None, max_frames: int = None) -> List[Dict[str, Any]]:
backend\pipeline\segmenters\gemini.py:36:         return []
backend\pipeline\segmenters\gemini.py:52:         return []
backend\pipeline\segmenters\gemini.py:58:         return []
backend\pipeline\segmenters\gemini.py:263:         return []
backend\pipeline\segmenters\wasb_hybrid.py:57:         max_frames: int = None) ->
List[Dict[str, Any]]:
backend\pipeline\segmenters\wasb_hybrid.py:63:         return []
backend\pipeline\segmenters\wasb_hybrid.py:73:         return []
backend\pipeline\segmenters\wasb_hybrid.py:78:         return []
backend\pipeline\segmenters\wasb_hybrid.py:83:         return []
backend\pipeline\segmenters\wasb_hybrid.py:109:         return []
backend\pipeline\segmenters\wasb_hybrid.py:118:         return []
backend\pipeline\segmenters\wasb_hybrid.py:152:         return []
backend\pipeline\segmenters\wasb_hybrid.py:160:     def process_candidate_window(wi: int,
window: Dict[str, Any]) -> List[dict]:
backend\pipeline\segmenters\wasb_hybrid.py:166:         return []
backend\pipeline\segmenters\tracknet_hybrid.py:60:         max_frames: int = None)
-> List[Dict[str, Any]]:
backend\pipeline\segmenters\tracknet_hybrid.py:67:         return []
backend\pipeline\segmenters\tracknet_hybrid.py:81:         return []
backend\pipeline\segmenters\tracknet_hybrid.py:86:         return []
backend\pipeline\segmenters\tracknet_hybrid.py:91:         return []
backend\pipeline\segmenters\tracknet_hybrid.py:120:         return []
backend\pipeline\segmenters\tracknet_hybrid.py:131:         return []
backend\pipeline\segmenters\tracknet_hybrid.py:172:         return []
backend\pipeline\segmenters\tracknet_hybrid.py:180:     def process_candidate_window(wi: int,
window: Dict[str, Any]) -> List[dict]:
backend\pipeline\segmenters\tracknet_hybrid.py:186:         return []

```

48. assistant (2026-06-10T06:08:07.504Z)

All findings confirmed. The base class declares `Result[List[...]]` but only `motion.py` honors it; the four production AI segmenters all return bare `[]` on every failure path (missing key, healthcheck fail, provider error, WSL timeout, no candidates), so `main.py`'s `isinstance(result, Failure)` check is dead code for them, and there is no FastAPI startup/lifespan hook so globals are only initialized on the CLI path.

49. user (2026-06-10T06:09:38.912Z)

Structured output provided successfully

50. assistant (2026-06-10T06:09:44.451Z)

Audit complete. I deep-read the API/DB/detector/provider/compiler/reporting layers with a failure-mode lens, ran the offline suite (256 passed), and returned the structured findings. The headline: the indexer is careful as a local tool but has a coherent cluster of hosted-API gaps the deferral docs don't cover ? chiefly a half-applied Result contract that turns every AI failure into a silent `[]`, unsafe SQLite/working-path concurrency, arbitrary file-path read/write surfaces on `/api/process` and `/api/compile`, and several uncaptured resource leaks (orphaned Gemini uploads, never-GC'd WSL copies, `timeout\_sec=0` hangs).